QUT Centre for Robotics



ENN583 – Foundations of Robotic Vision

Sem 2, 2026

Prof Niko Suenderhauf

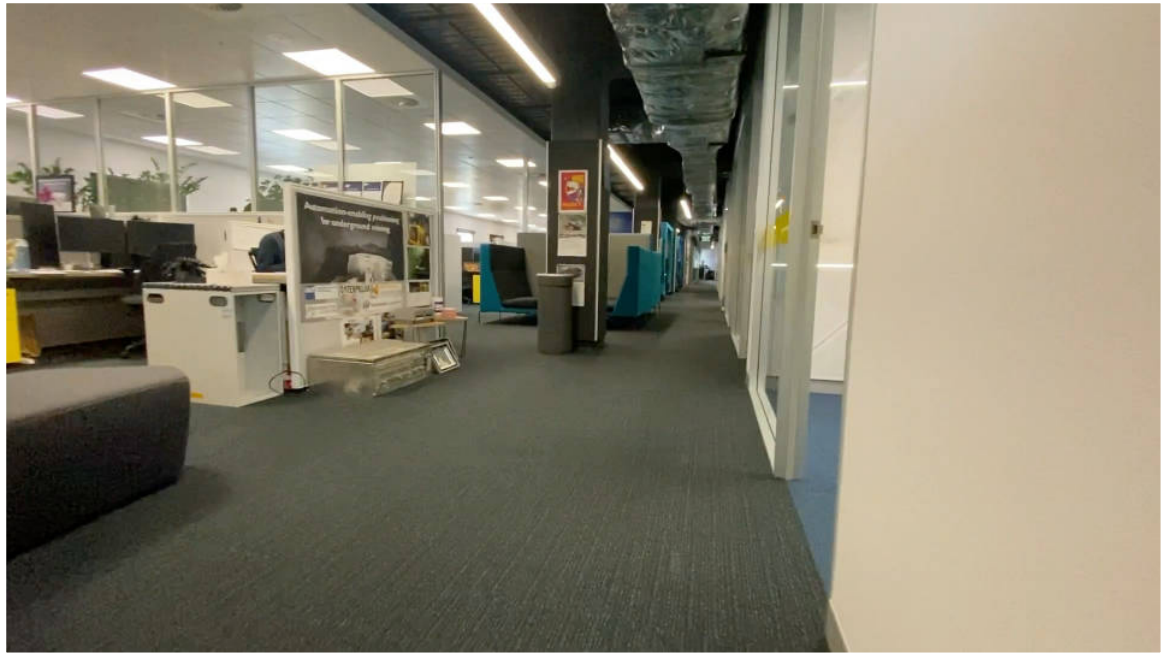
Welcome to Week 2!



CRICOS No.00213J

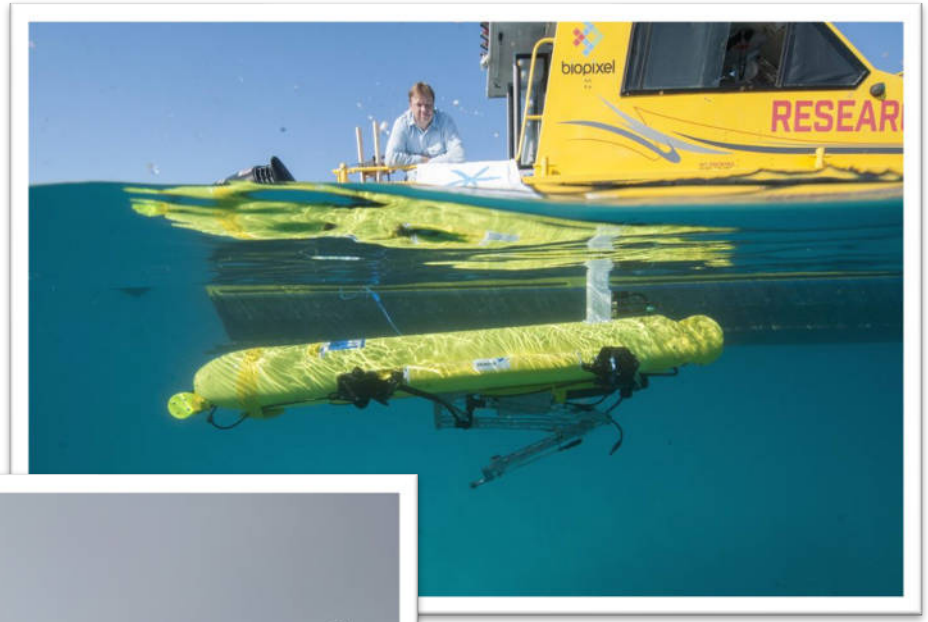
Motivation: How can robots measure their movement through an environment?





Wheel encoders is all you need?

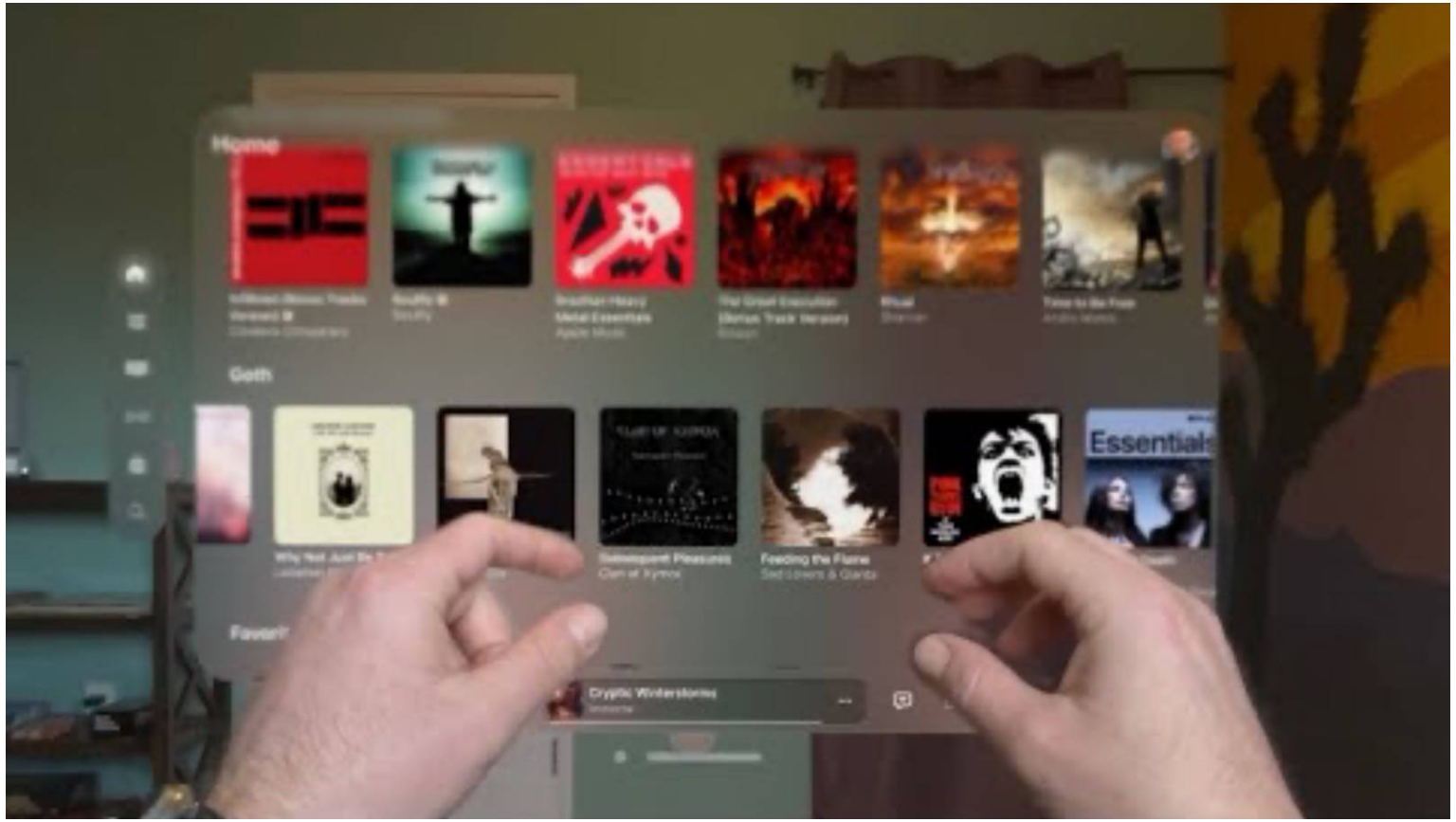






CRICOS No.00213J

Augmented Reality Devices

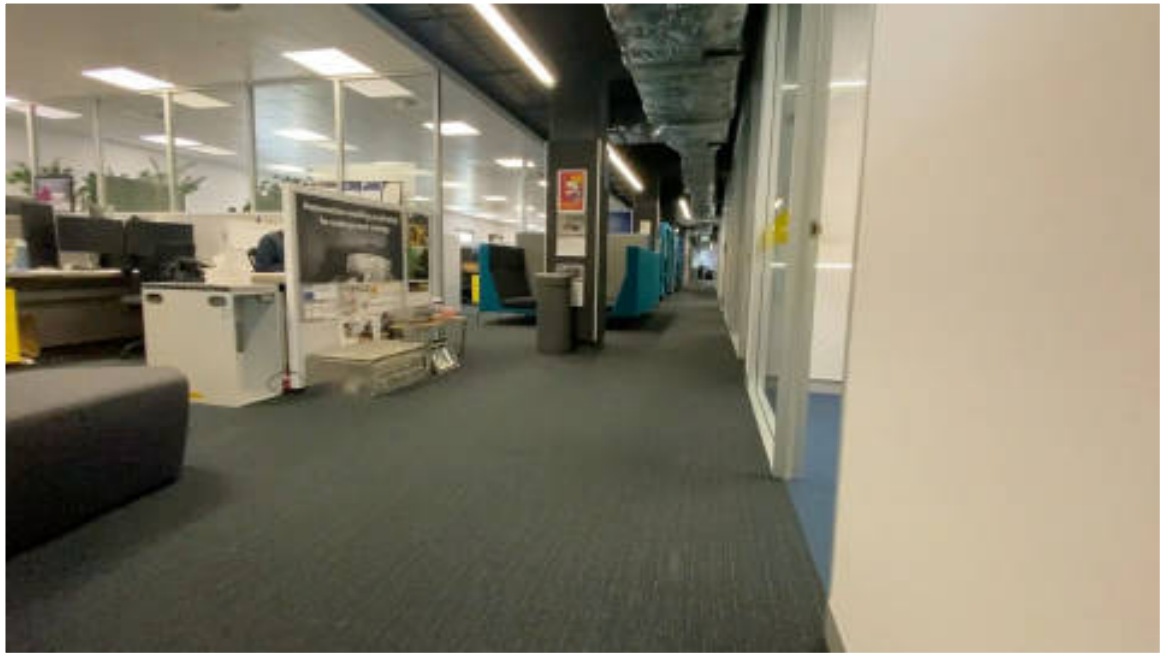


https://www.youtube.com/watch?v=nSC6_sEA6JM

Visual Odometry

Determine the motion of a camera by analysing the camera images.

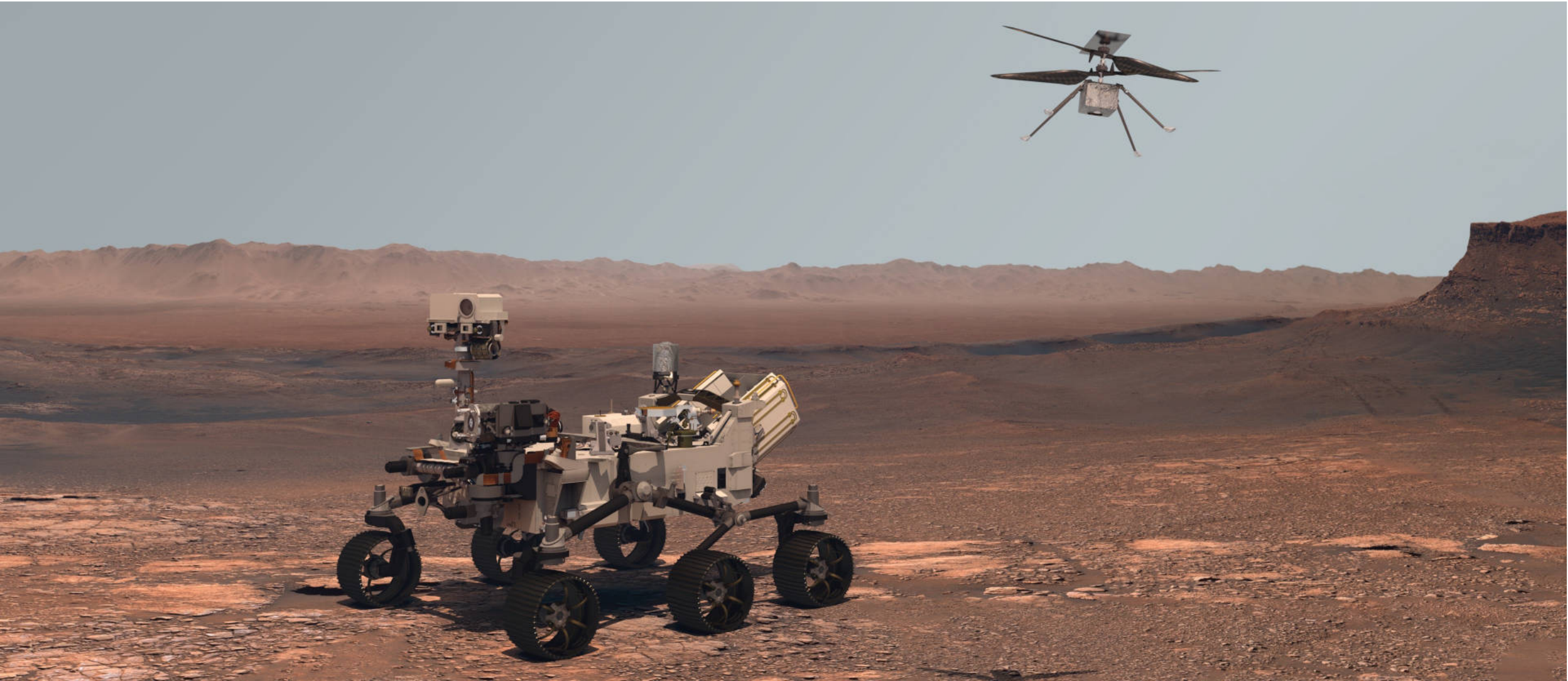
- *Odometry* from Greek *odos* (meaning "route") and *metron* (meaning "measure")
- Commercial applications in
  - Robotics
  - autonomous systems
  - quadcopters and other UAVs
  - augmented reality





CRICOS No.00213J

Ingenuity Mars UAV



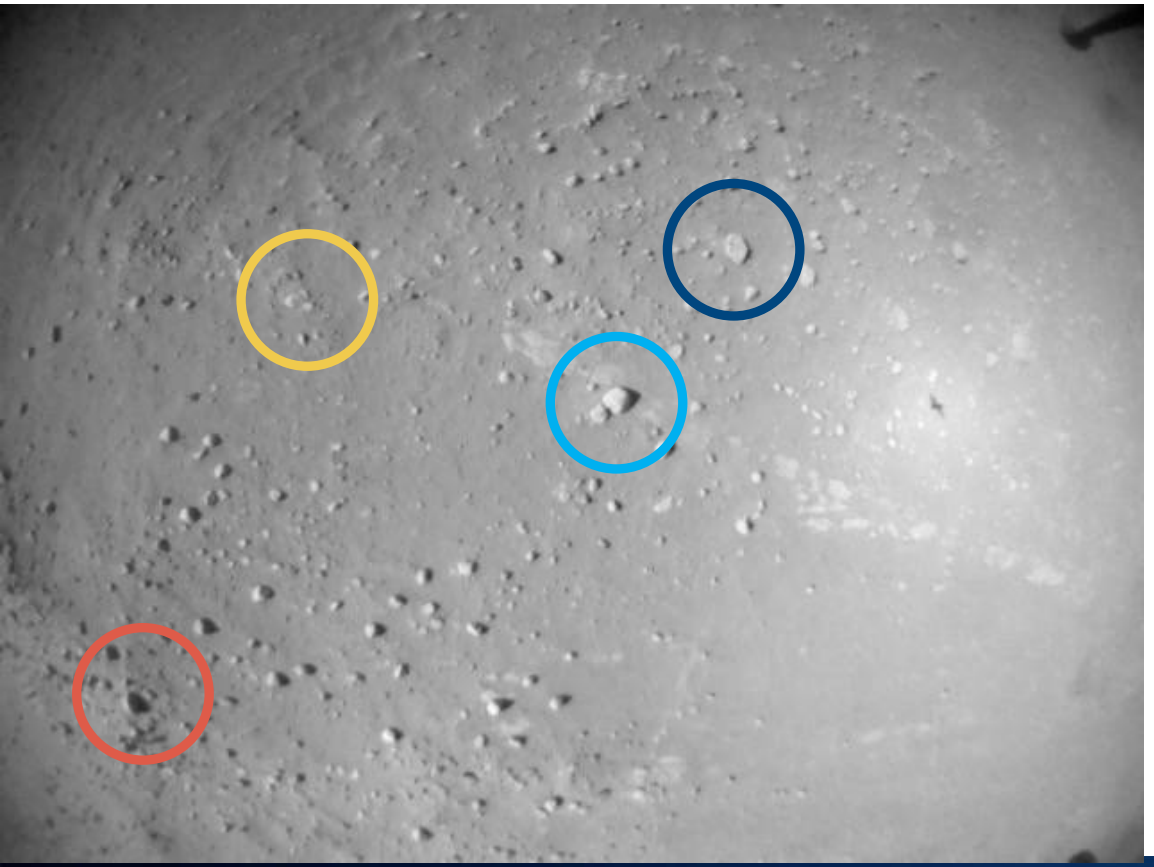
Which way is it moving?



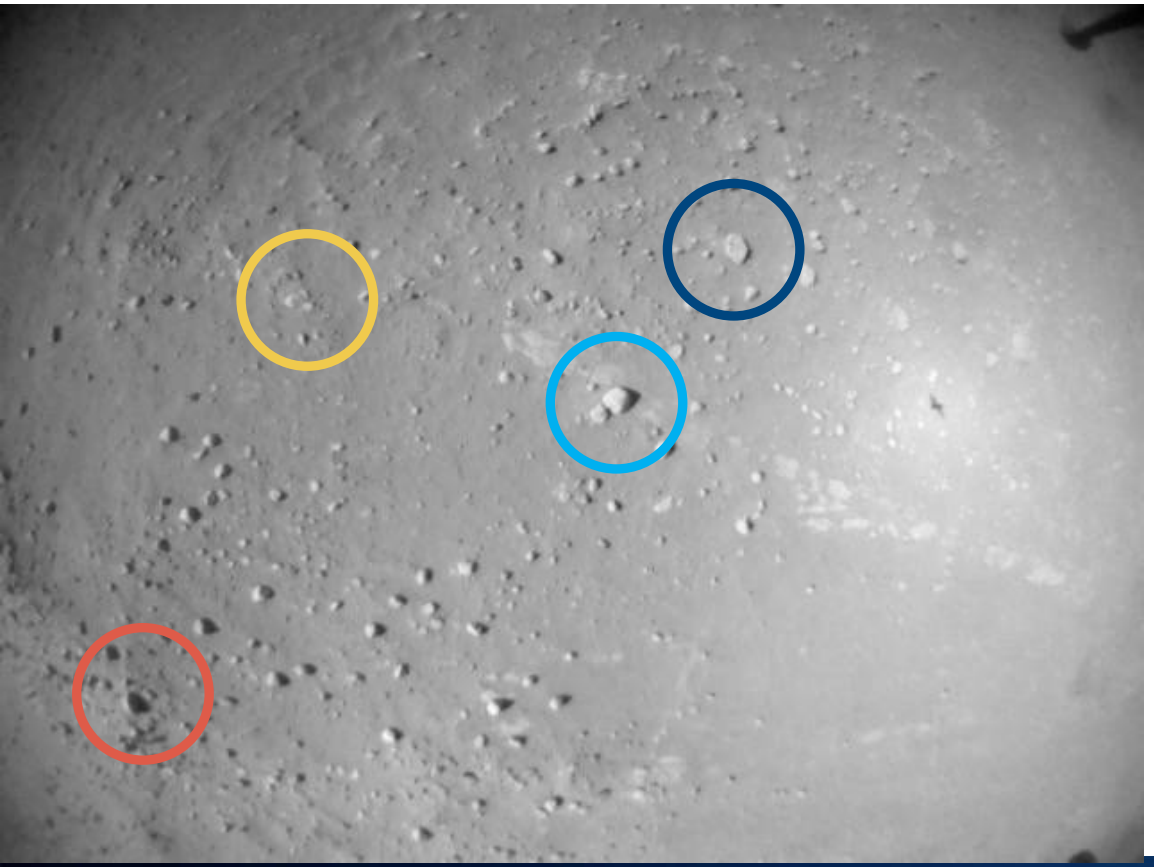


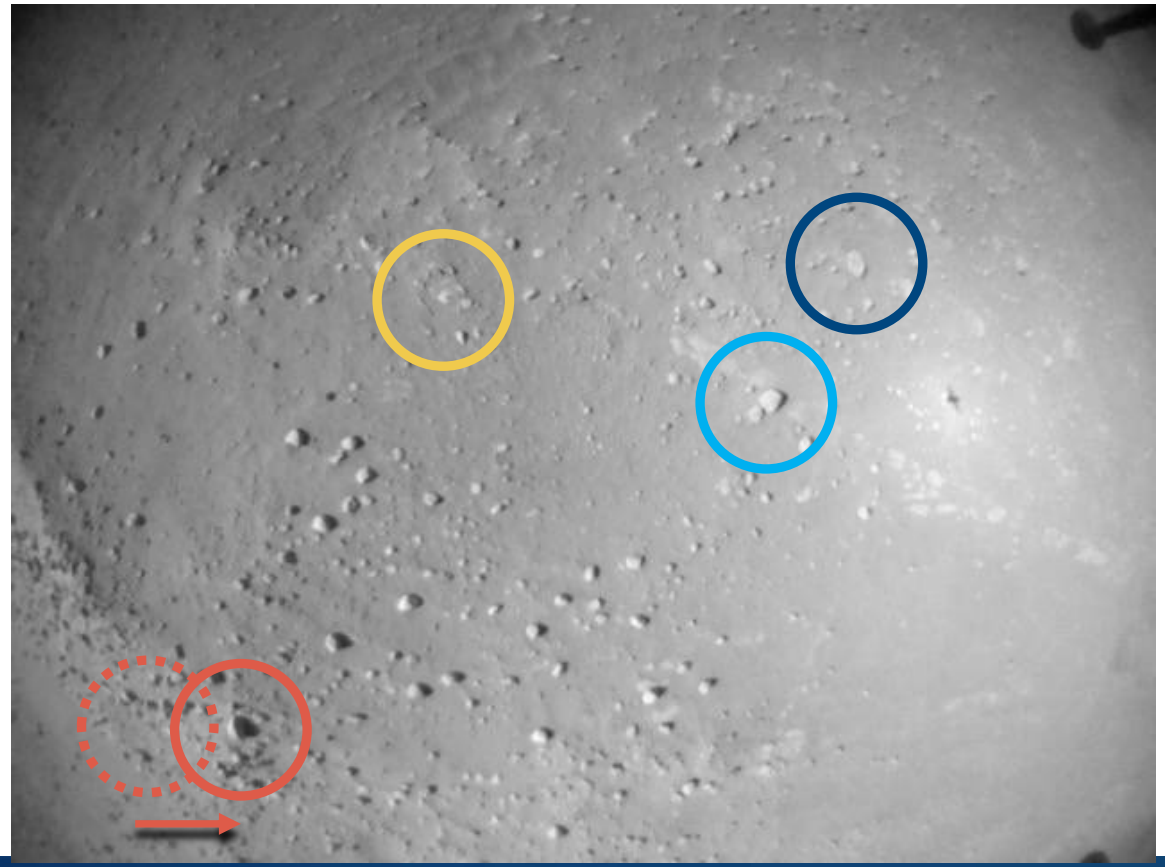
CRICOS No.00213J

How did you do it?



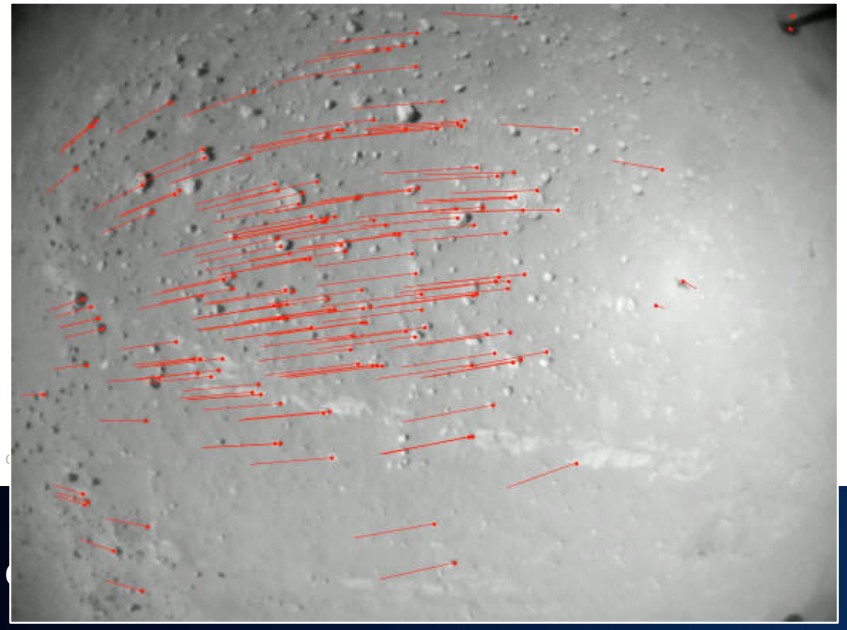
How did you do it?





The Visual Odometry Pipeline

1. **Establish visual correspondences:** Find distinctive and interesting image regions and identify the corresponding region in the next image
2. **Motion Estimation:** Estimate the camera motion based on these correspondences





The Visual Odometry Pipeline

1. **Establish visual correspondences:** Find distinctive and interesting image regions and identify the corresponding region in the next image
2. **Motion Estimation:** Estimate the camera motion based on these correspondences

- Today: Visual Correspondences: Interest point detectors, descriptors, matching
- Week 4-5: Motion Estimation

CRICOS No.00213J

Establishing Visual Correspondences: Feature Detection and Matching

Establishing Visual Correspondences: Feature Detection and Matching

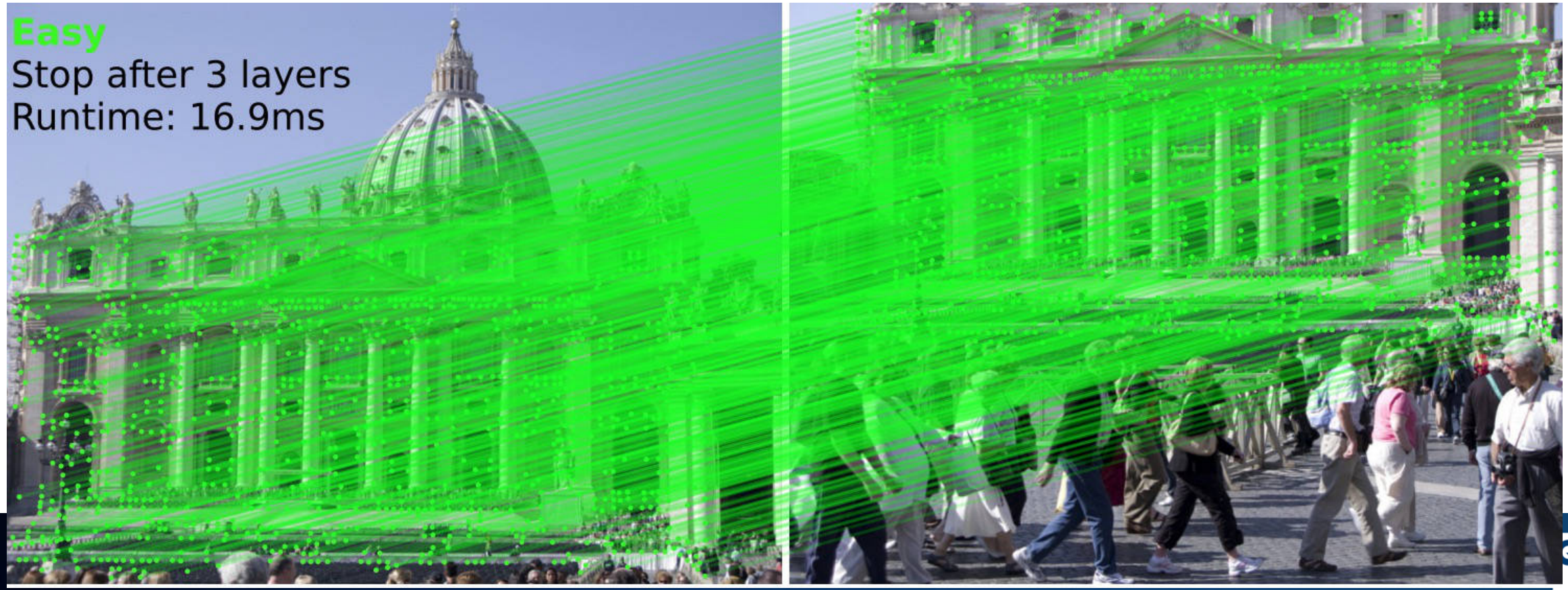
Given two images, how do we know which pixels refer to the same physical point?



LightGlue: Local Feature Matching at Light Speed. Philipp Lindenberger, Paul-Edouard Sarlin, Marc Pollefeys. ICCV 2023

Establishing Visual Correspondences: Feature Detection and Matching

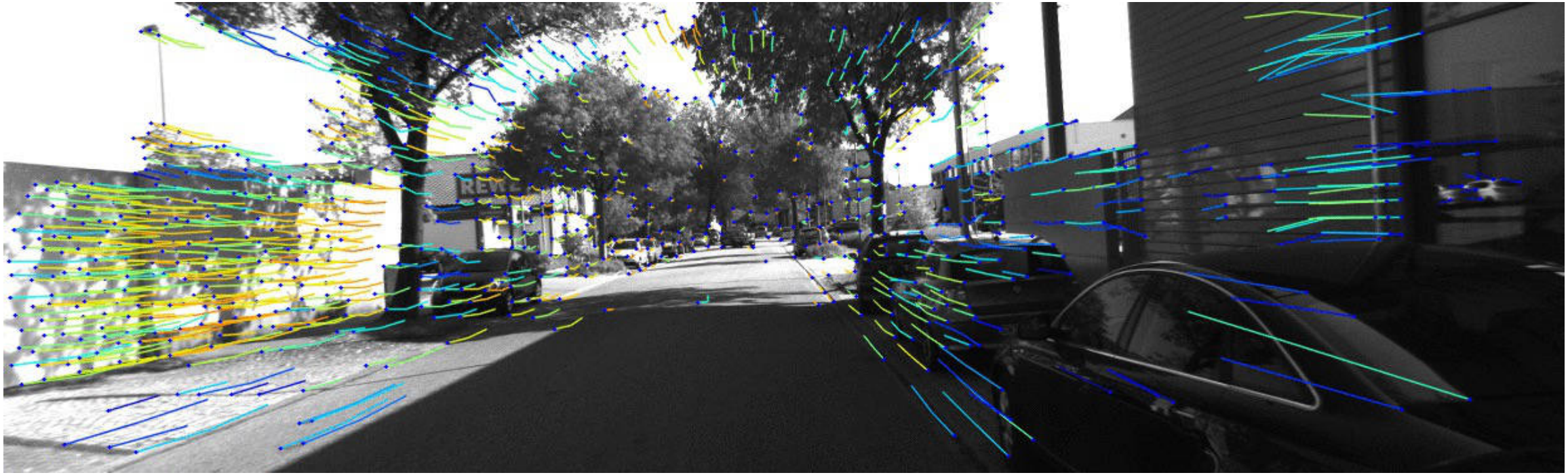
Given two images, how do we know which pixels refer to the same physical point?



LightGlue: Local Feature Matching at Light Speed. Philipp Lindenberger, Paul-Edouard Sarlin, Marc Pollefeys. ICCV 2023

Establishing Visual Correspondences: Feature Detection and Matching

Given two images, how do we know which pixels refer to the same physical point?



CRICOS No.00213J

LightGlue: Local Feature Matching at Light Speed. Philipp Lindenberger, Paul-Edouard Sarlin, Marc Pollefeys. ICCV 2023

Challenges

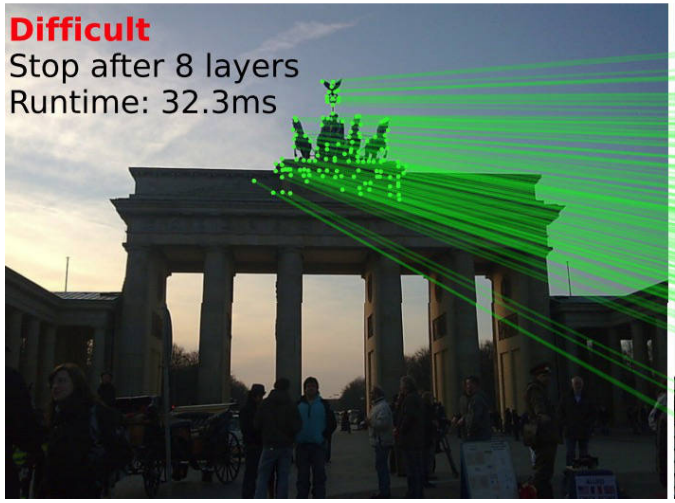
CRICOS No.00213J

LightGlue: Local Feature Matching at Light Speed. Philipp Lindenberger, Paul-Edouard Sarlin, Marc Pollefeys. ICCV 2023

Challenges

Geometric changes due to viewpoint change:

- scale change
- rotation
- perspective distortion

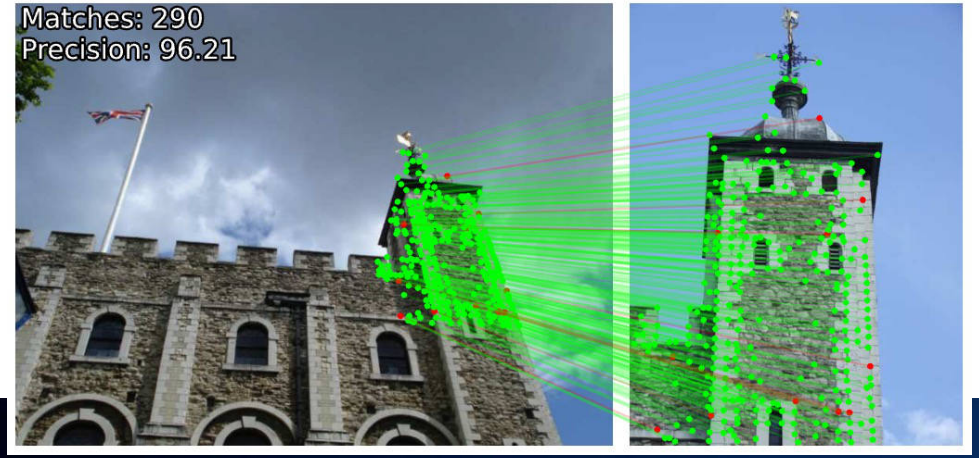








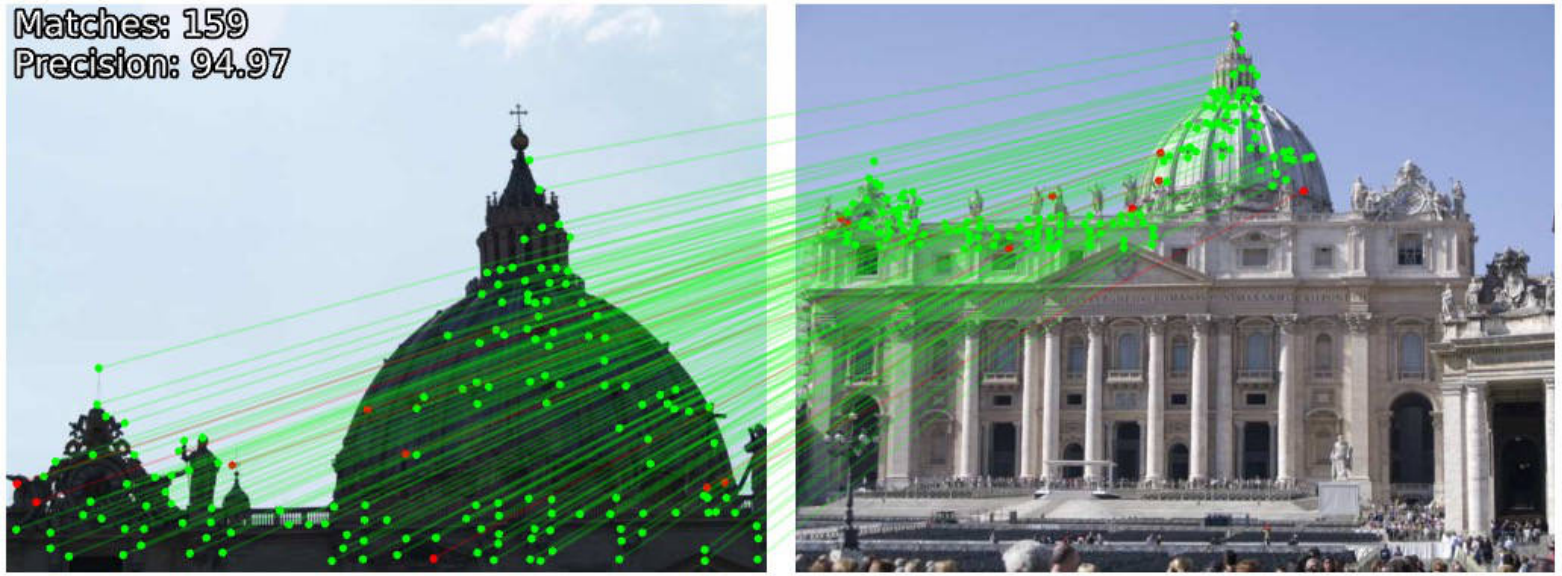
Mikolajczyk et al., A Comparison of Affine Region Detectors.
International Journal of Computer Vision, 2004.



Challenges

Appearance Changes

- Light and shadows
- Season and weather
- Exposure, white-balance
- Blur, sensor noise







LightGlue: Local Feature Matching at Light Speed. Philipp Lindenberger, Paul-Edouard Sarlin, Marc Pollefeys. ICCV 2023

Challenges

Ambiguity

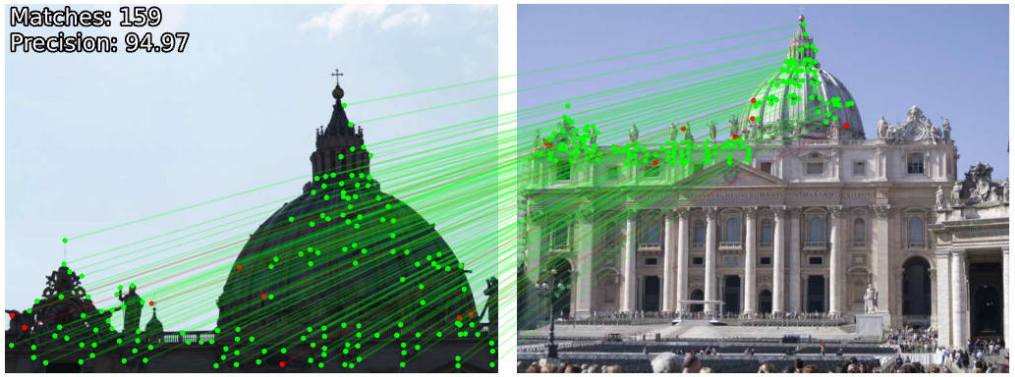
- Repeated patterns
- Visual aliasing (things look the same but are not the same)
- Low-texture regions



LightGlue: Local Feature Matching at Light Speed. Philipp Lindenberger, Paul-Edouard Sarlin, Marc Pollefeys. ICCV 2023

Establishing Visual Correspondences

Designing methods that are robust or invariant to these challenges is still an area of active research.



General pipeline:

- Feature **detection**
  - Find good features in the image.
- Feature **description**
  - Each feature gets a “fingerprint” that identifies it.
- Feature **matching**
  - Use the descriptors to establish correspondence across both images

CRICOS No.00213J

LightGlue: Local Feature Matching at Light Speed. Philipp Lindenberger, Paul-Edouard Sarlin, Marc Pollefeys. ICCV 2023

Establishing Visual Correspondences



- There are **many different methods**, especially for detectors and descriptors
  - They each have their advantages and disadvantages
- Sometimes a method proposes multiple components
  - e.g. SIFT is a detector and descriptor
- Some methods are **combinations** of others
  - e.g. ORB is the FAST detector + BRIEF descriptor
- There are certain standard approaches that the community has settled on

CRICOS No.002130

LightGlue: Local Feature Matching at Light Speed. Philipp Lindenberger, Paul-Edouard Sarlin, Marc Pollefeys. ICCV 2023

Some noteworthy methods

Stage	Classical methods	Learned methods
Detection	Harris, FAST, Difference-of-Gaussian (SIFT), SURF, AKAZE	SuperPoint, DISK, R2D2
Description	SIFT, SURF, BRIEF, ORB, BRISK, AKAZE	SuperPoint, DISK, D2-Net, R2D2
Matching	Nearest neighbour, brute force, ratio test, cross-check, approximate nearest neighbour	SuperGlue, LightGlue, LoFTR

CRICOS No.00213J

Visual Correspondences – Applications beyond Visual Odometry

- **Image stitching and panoramas:** Match overlapping regions between images, then align and blend them into a larger view.
- **3D reconstruction / Structure from Motion:** Match points across many images to estimate camera poses and reconstruct 3D scene points.
- **Stereo vision and depth estimation:** Match the same point in the left and right camera images to estimate depth.
- **Object tracking:** Track distinctive points on an object across video frames to estimate its motion, deformation, or pose.
- **Image registration**: Align images from different times, sensors, or viewpoints, such as satellite images, medical scans, or before-and-after inspection images.
- **Augmented reality:** Match features to a known scene or object so virtual content remains correctly anchored.
- **Object recognition and instance matching:** Determine whether a particular object, building, artwork, or product appears in another image.
- **Place recognition and loop closure:** Identify that two images show the same location, even when captured at different times or from different viewpoints.
- **Camera calibration:** Match known calibration-pattern points to estimate camera parameters.

CRICOS No.00213J



Building Rome in a Day, Sameer Agarwal, Noah Snavely, Ian Simon, Steven M. Seitz and Richard Szeliski. International Conference on Computer Vision, 2009.



TAPIR: Tracking Any Point with per-frame Initialization and temporal Refinement. Carl Doersch, Yi Yang, Mel Vecerik, Dilara Gokay, Ankush Gupta, Yusuf Aytar, Joao Carreira, Andrew Zisserman. ICCV 2023

Feature Detection



What makes a good feature and a good feature detector?

Let’s think about it!

CRICOS No.00213J

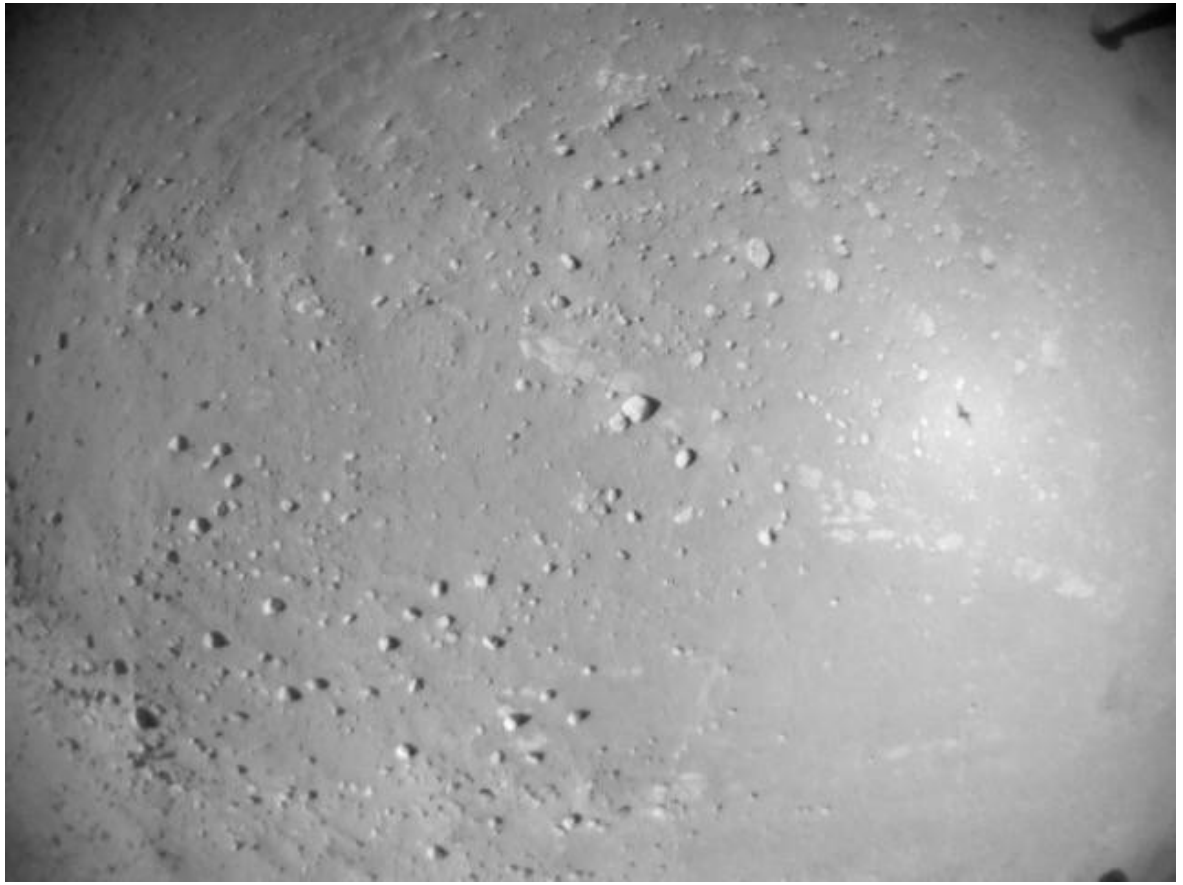
What makes a good feature and a good feature detector?

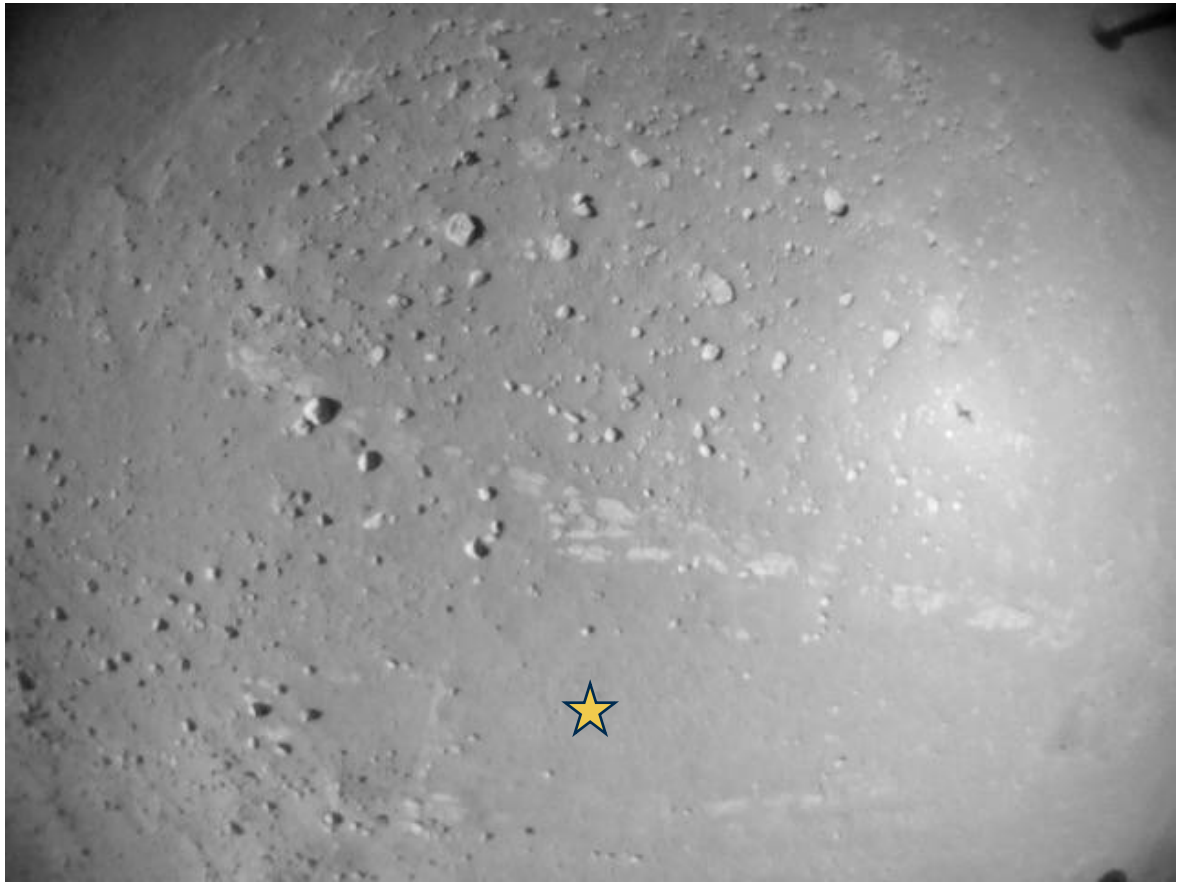
Let’s think about it!

You might say good features should be in an image region that is interesting, somewhat unique, and easy to find again after the camera moved.

CRICOS No.00213J



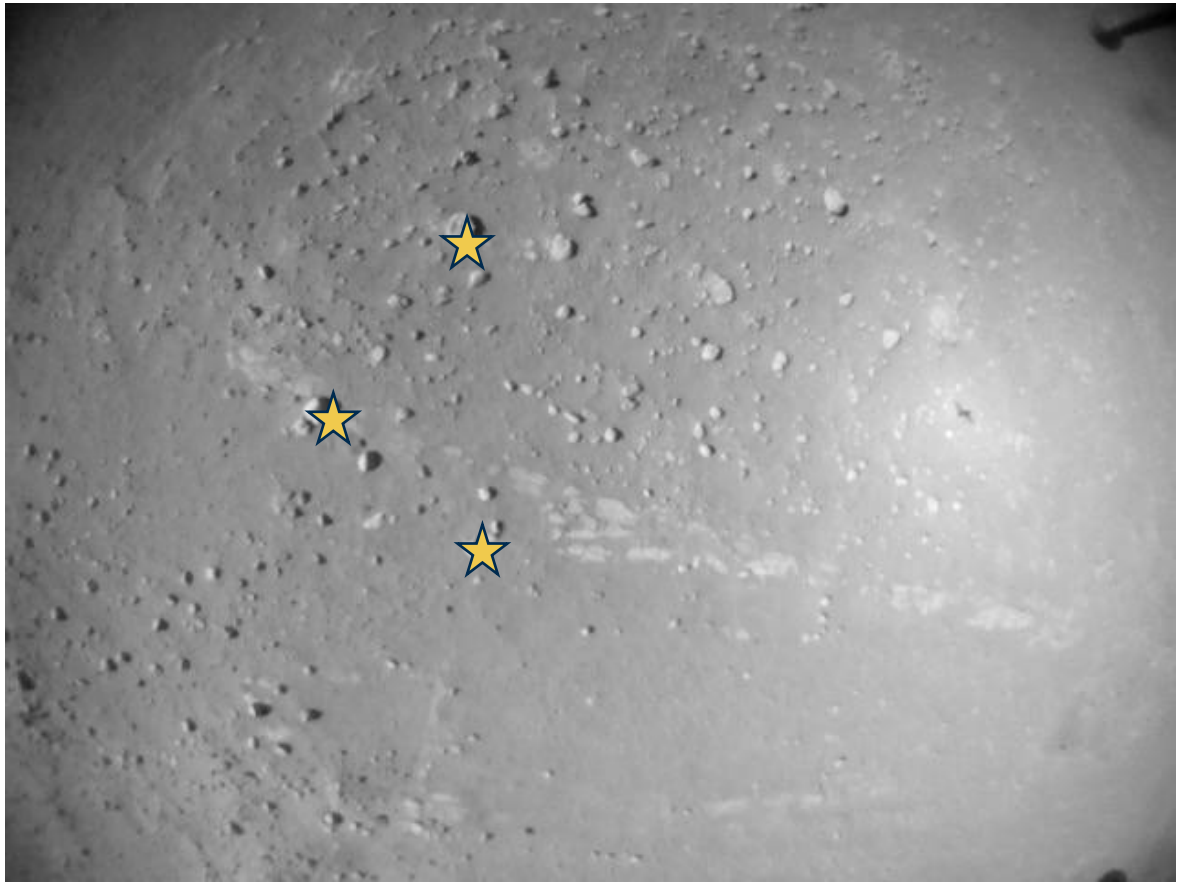


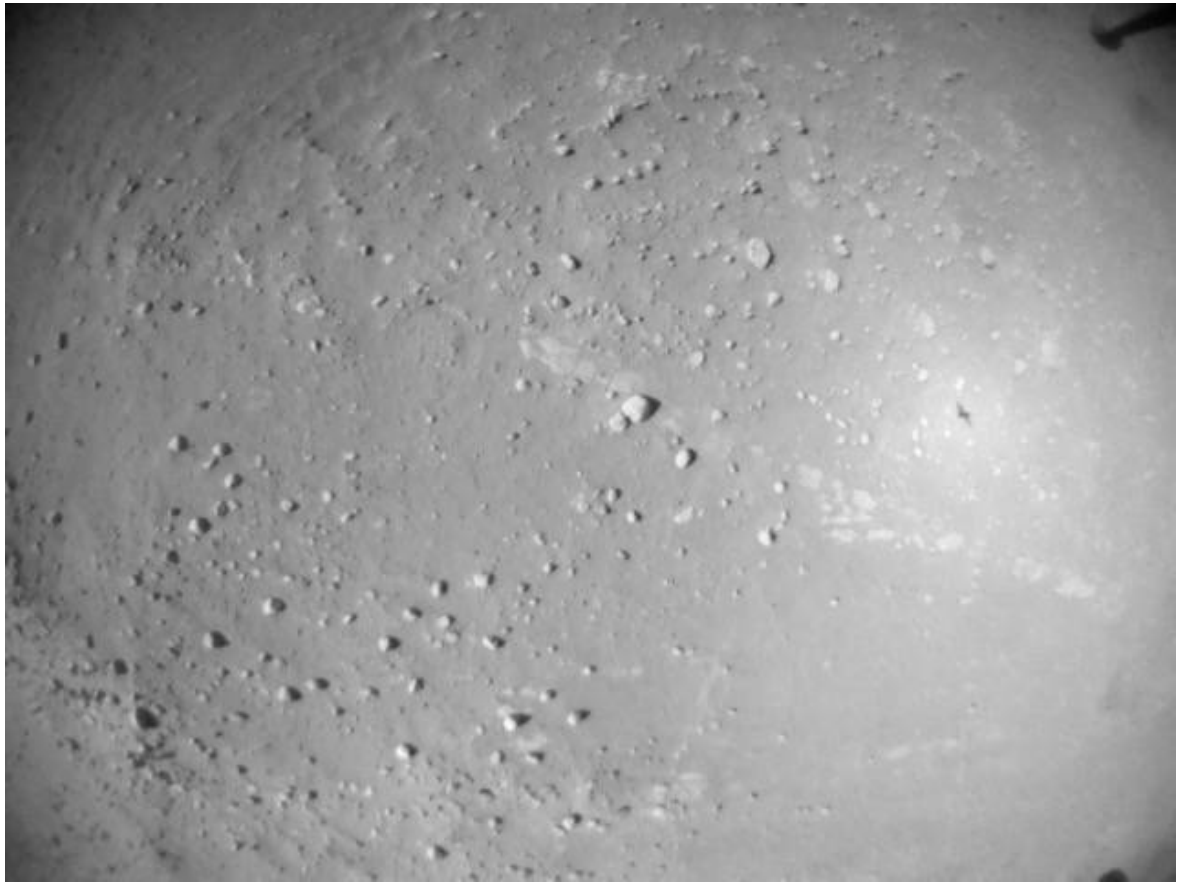


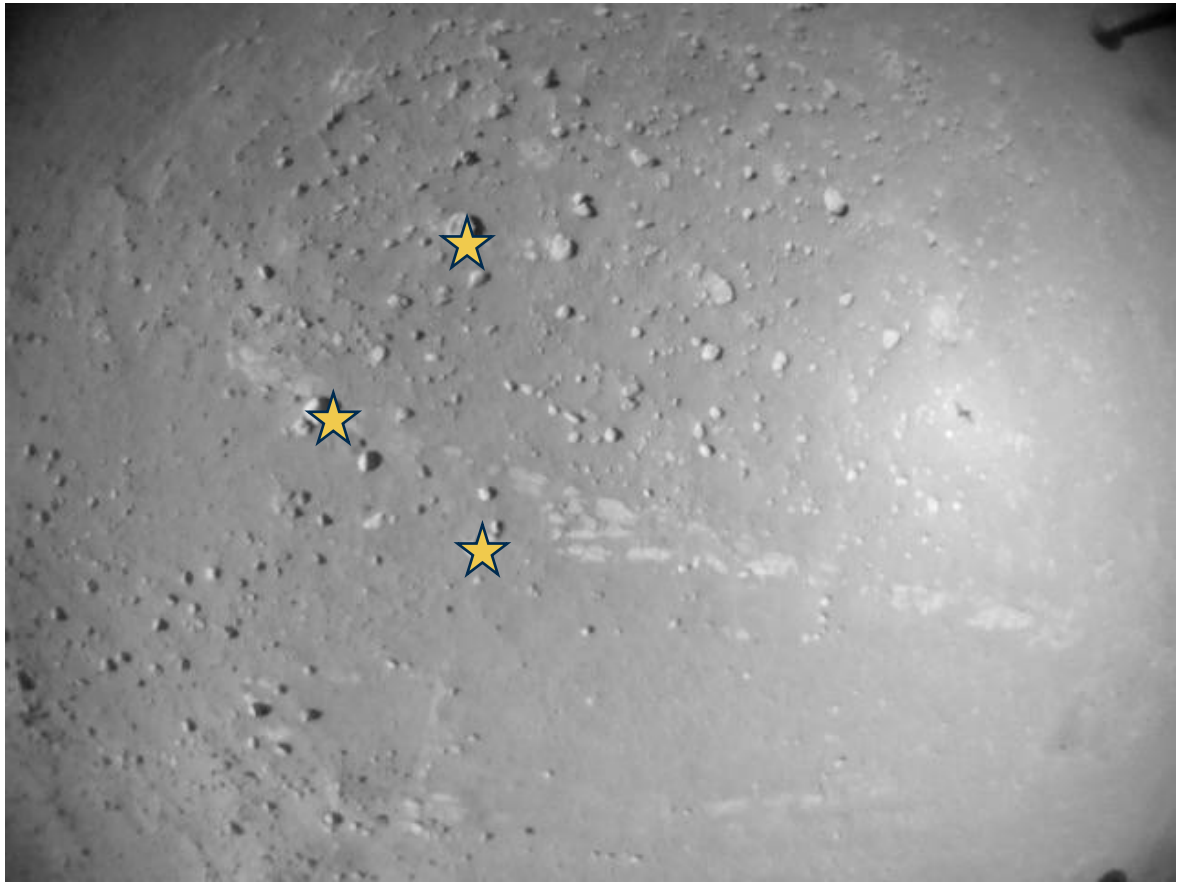


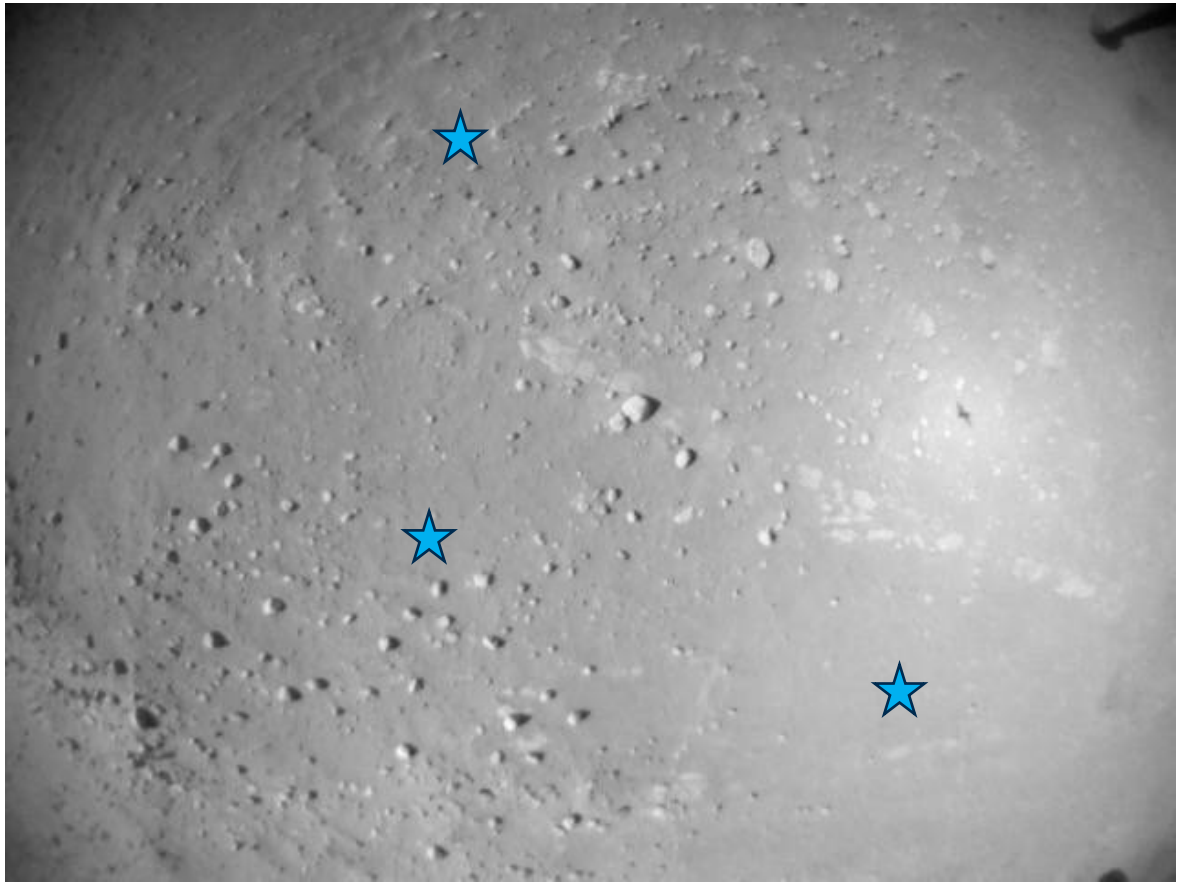
Distinctive?

CRICOS No.00213J

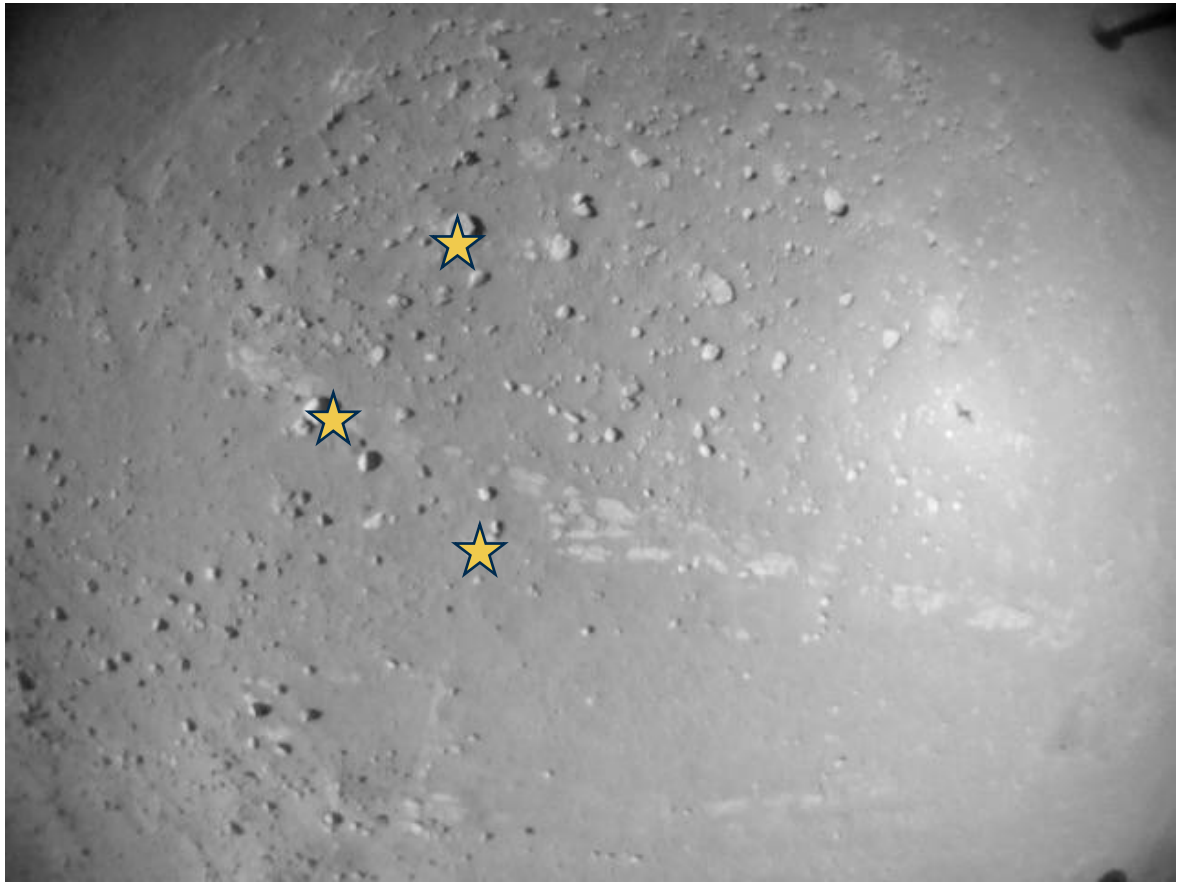


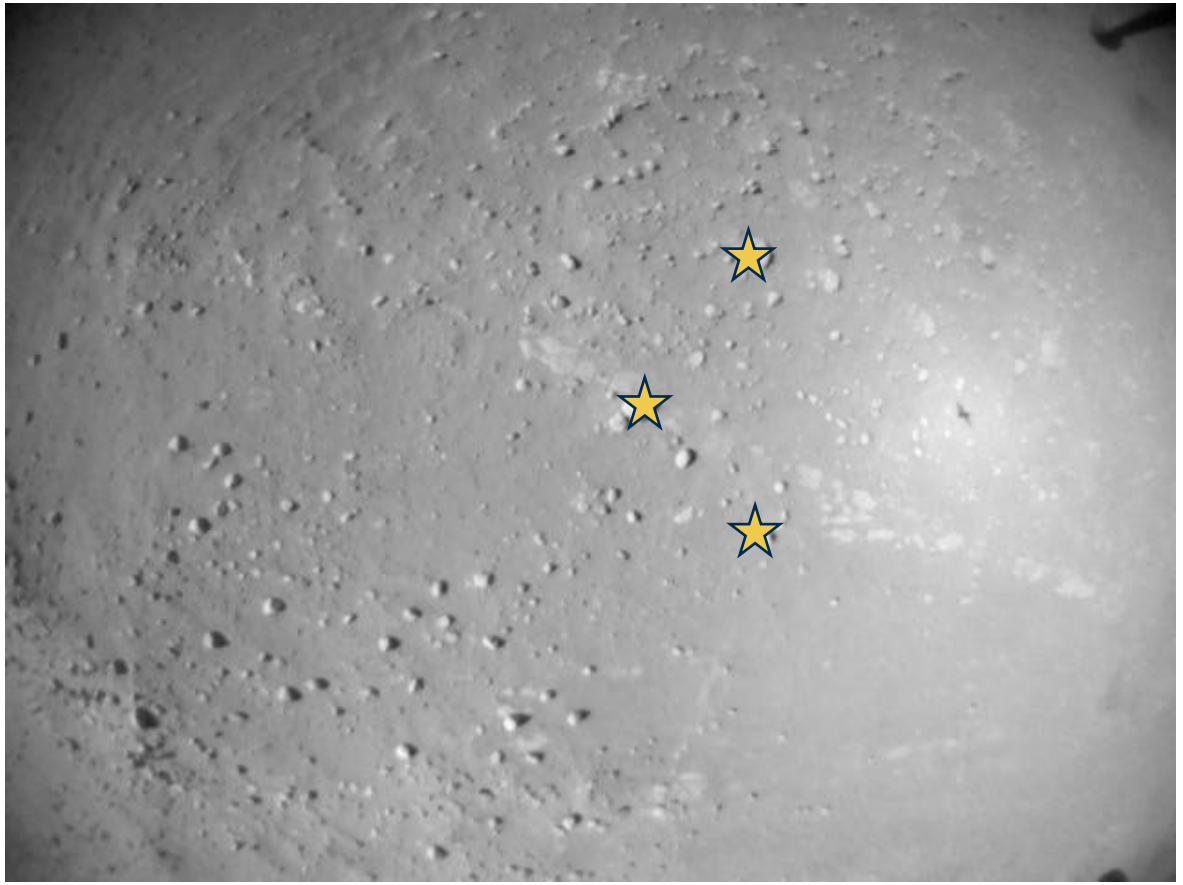






Repeatable?





Repeatable?

What makes a good feature?

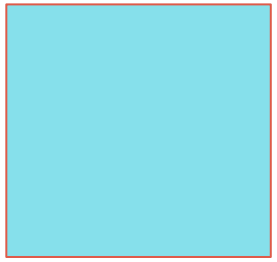
- **Distinctive**
  - The appearance around the point should be different enough from other regions that it can be matched unambiguously.
- **Repeatable**
  - The same physical point should be detected again when the viewpoint, scale, illumination, or image noise changes slightly.

CRICOS No.00213J

What makes a good feature?

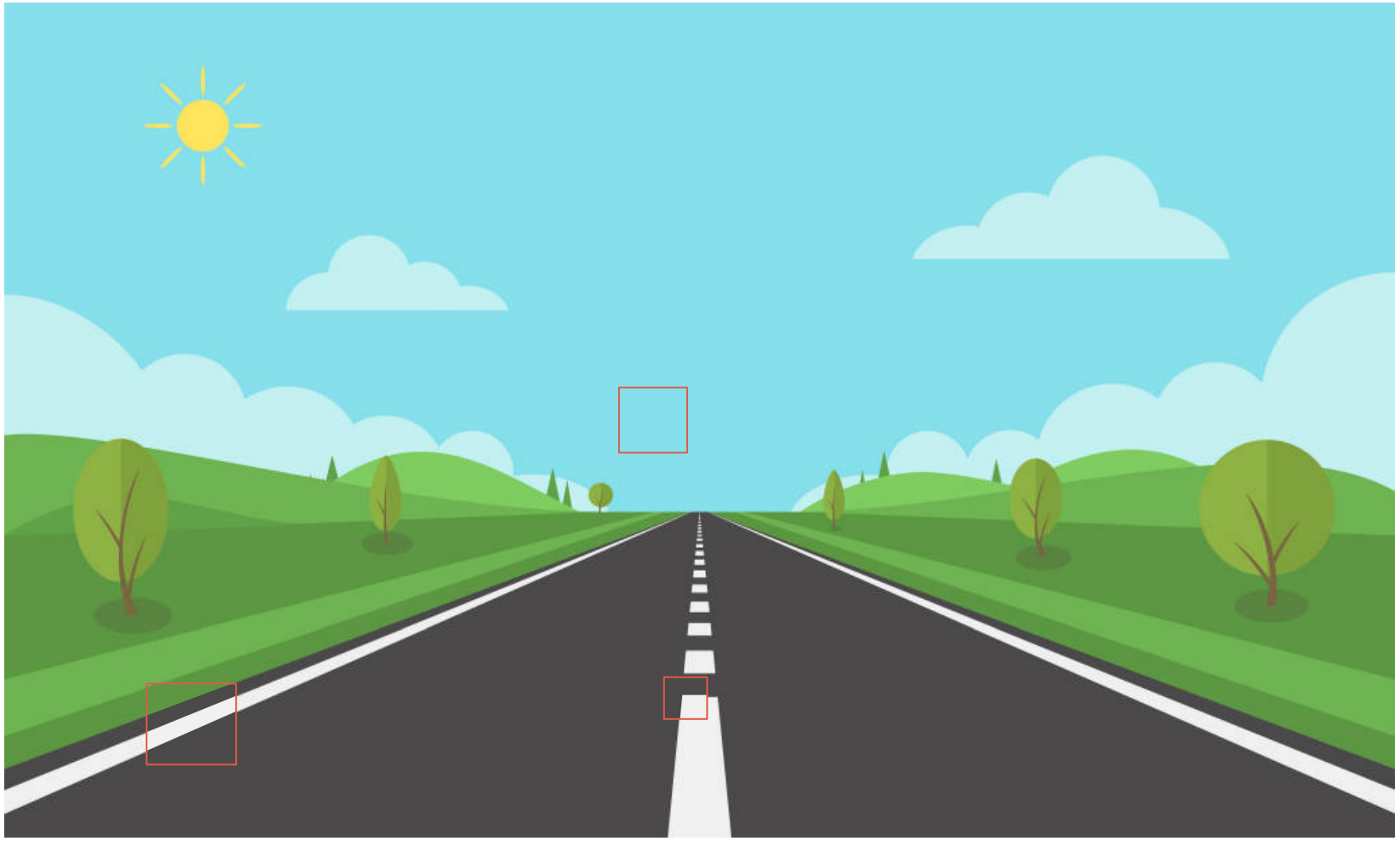
- **Distinctive**
  - The appearance around the point should be different enough from other regions that it can be matched unambiguously.
- **Repeatable**
  - The same physical point should be detected again when the viewpoint, scale, illumination, or image noise changes slightly.
- **Well localised**
  - The point should have a well-defined image location, so small changes in the image do not cause a large shift in where it is detected.

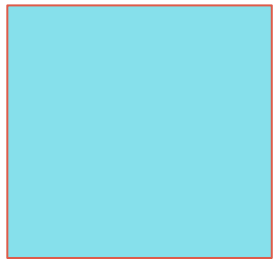
CRICOS No.00213J





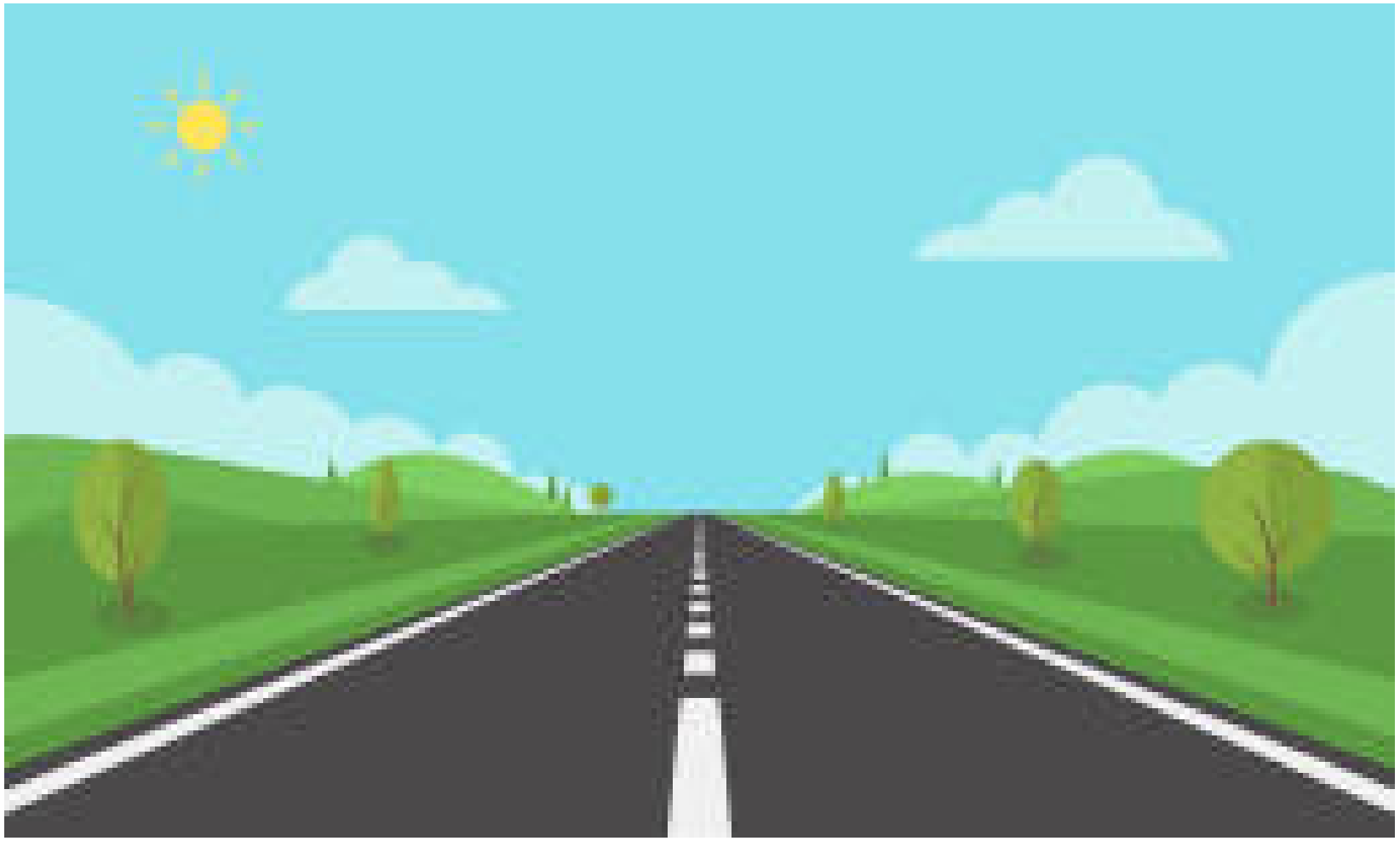


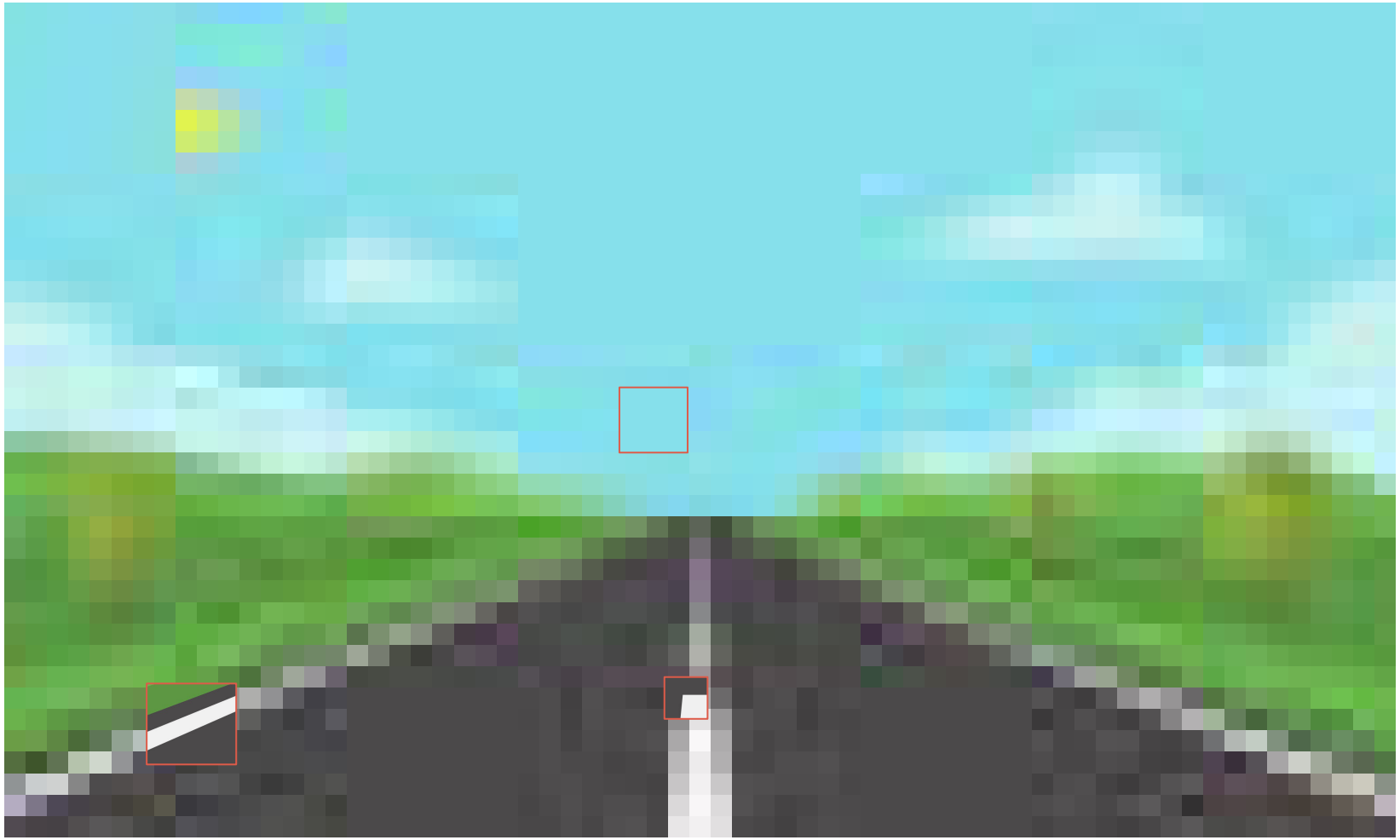


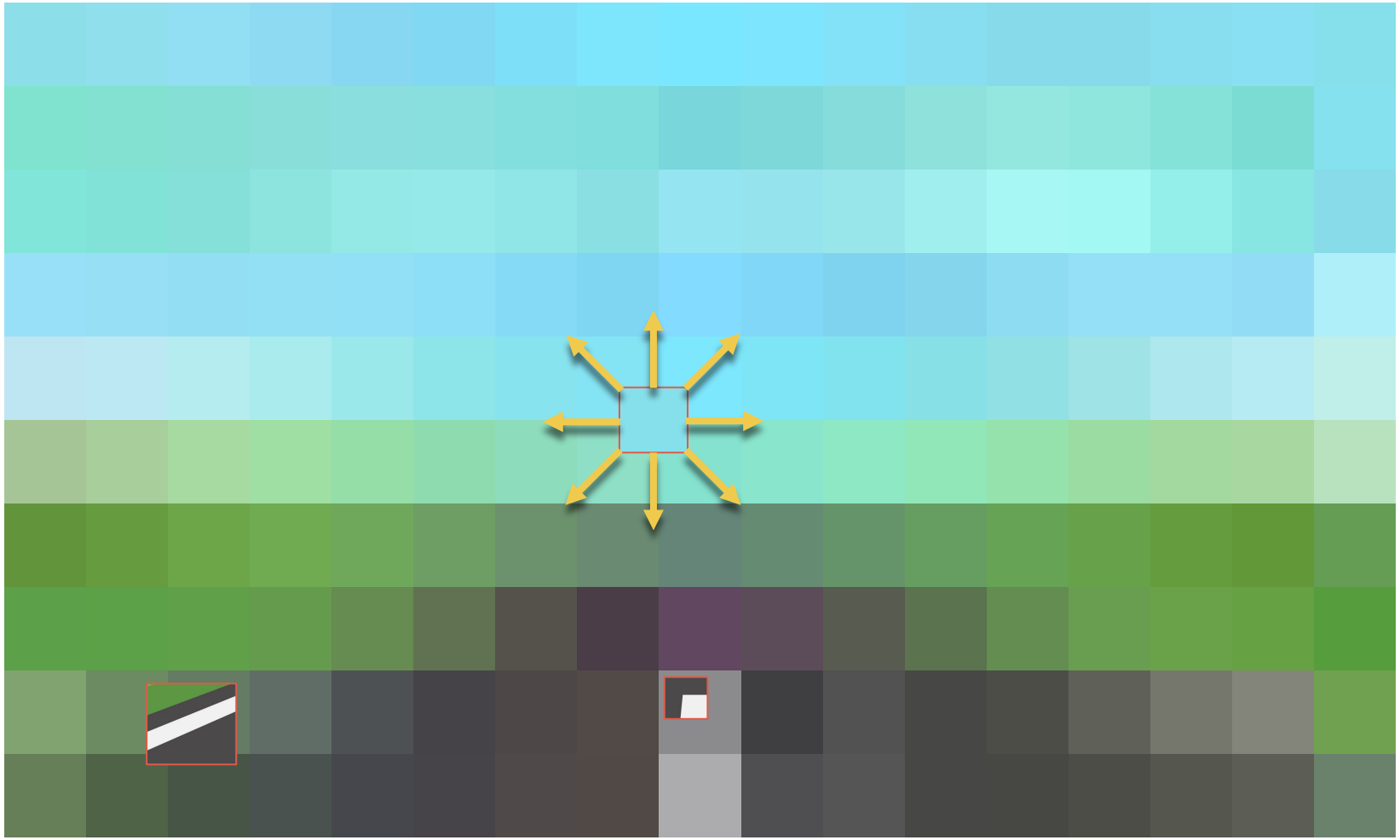


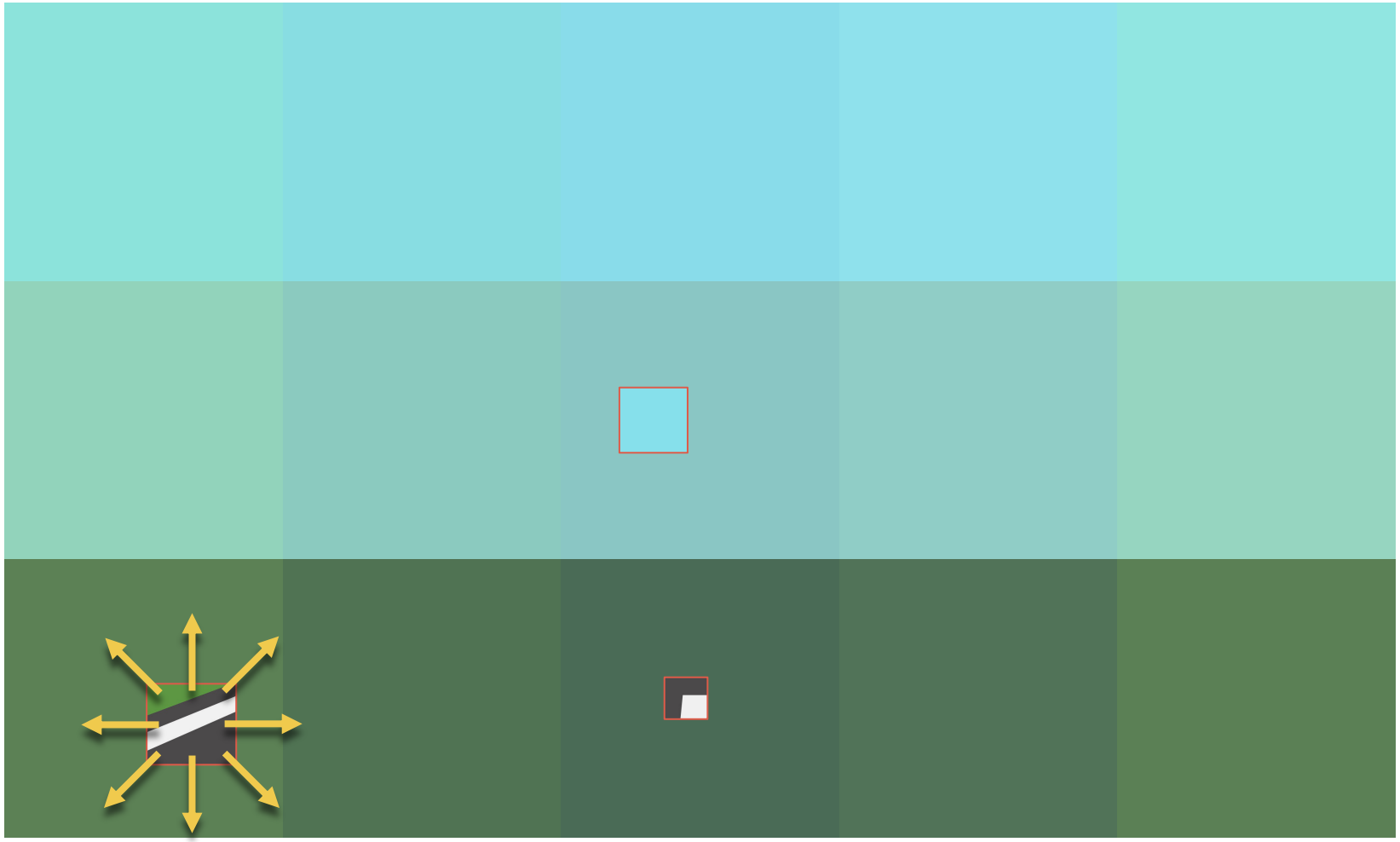


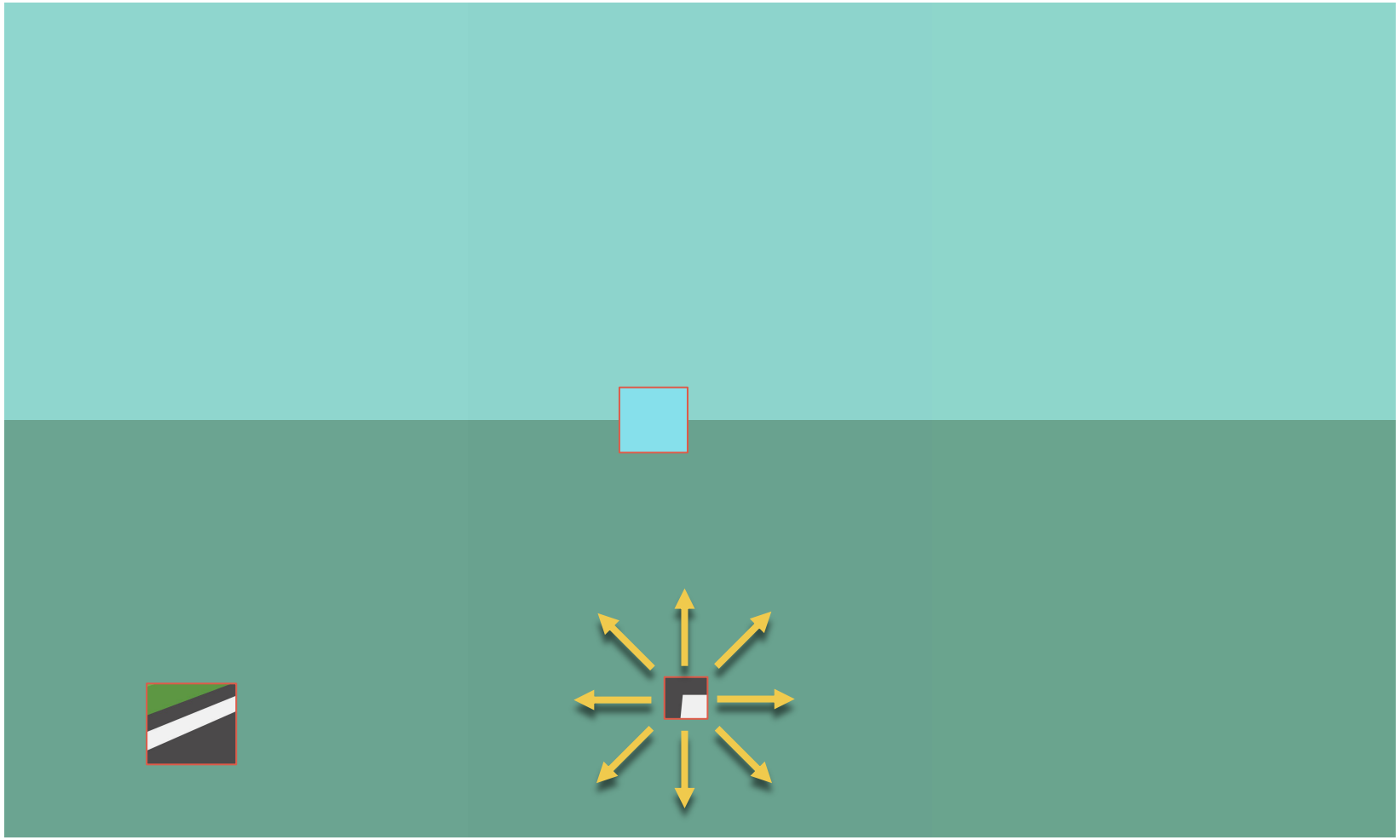












Moravec & Harris

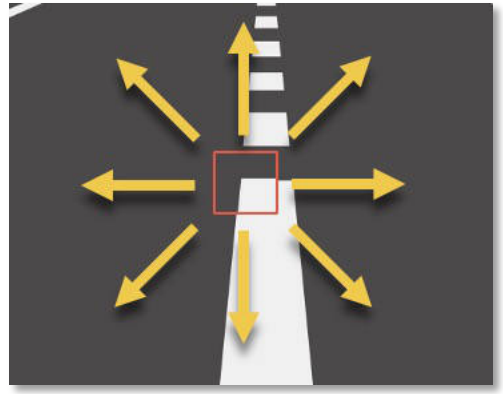
The first detectors



Moravec: The first detector (1980)

Idea:

- Find pixels where the image content changes significantly in all directions.
- These are corners and are good features.

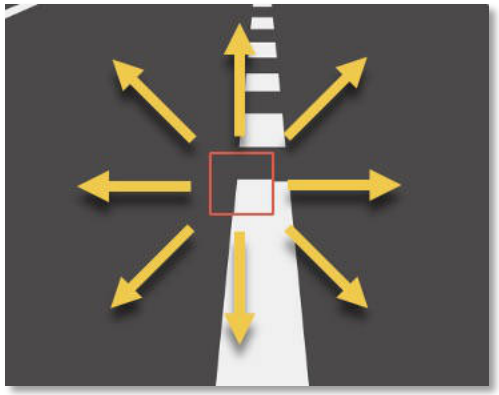


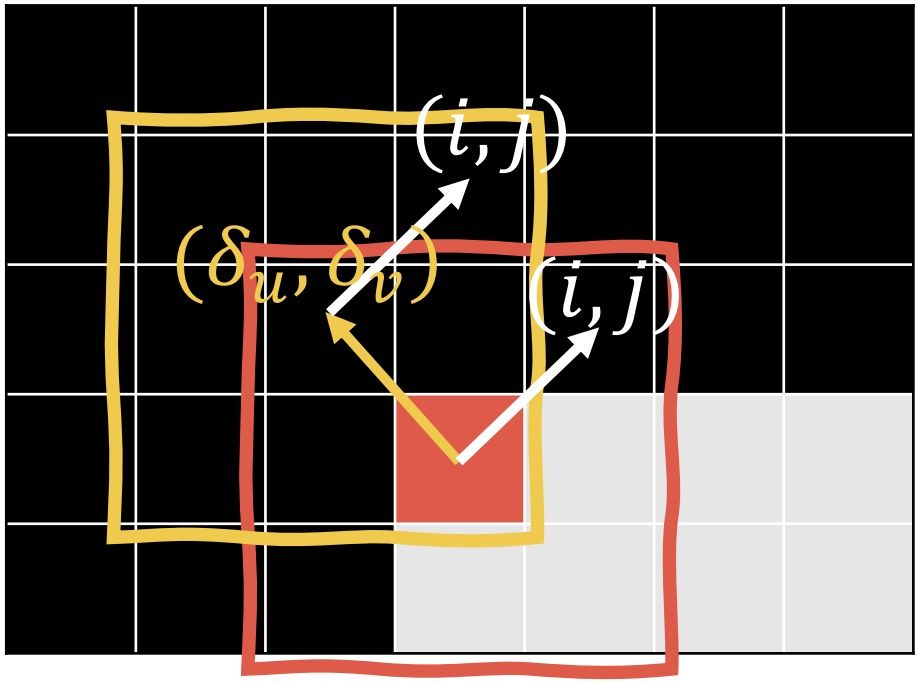
Method:

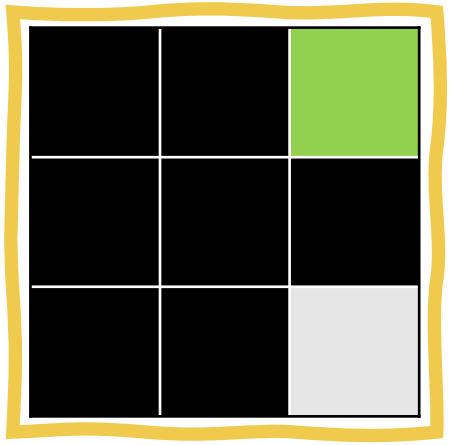
- **For each pixel:**
  - Take a small neighbourhood around the pixel.
  - Shift that neighbourhood slightly in the up, down, left, right, diagonal directions.
  - Measure how much the image content changes for each shift.
  - Use the smallest change as the feature score.
  - If the content changes strongly in every tested direction, the pixel receives a high score.

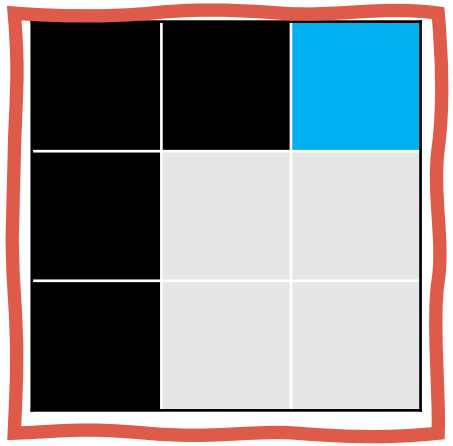
CRICOS No.00213J

Moravec Detector (1980)





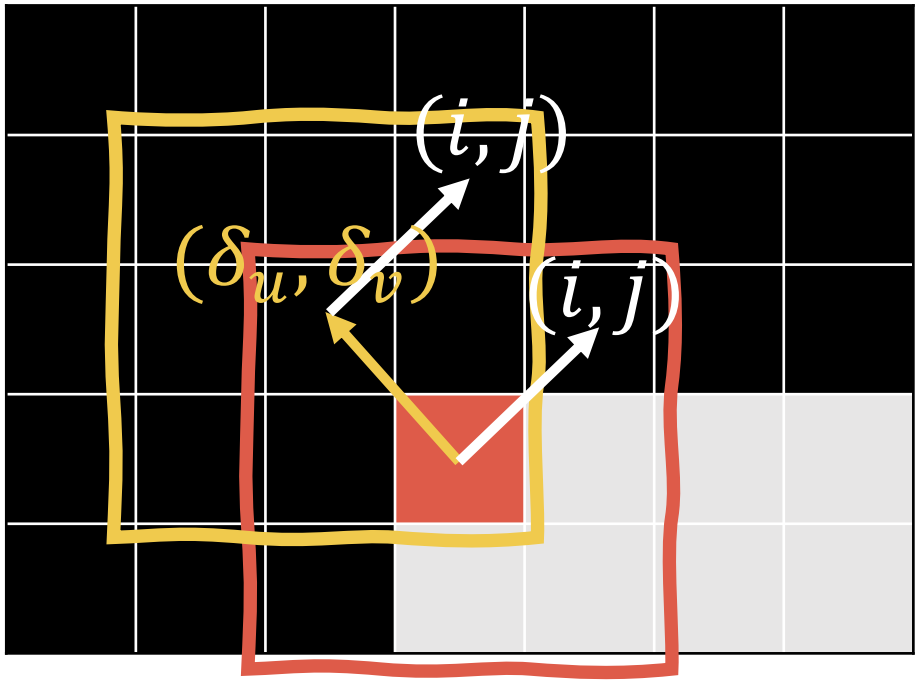




$$s(u,v,\delta_u,\delta_v) = \sum_{(i,j)\in W} (I(u+\delta_u+i, v+\delta_v+j) - I(u+i,v+j))^2$$

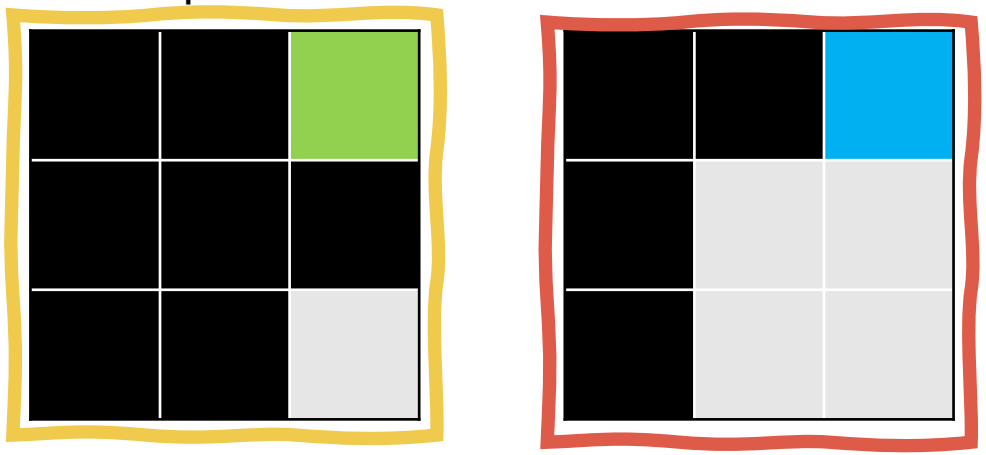
$$C(u,v) = \min_{(\delta_u,\delta_v)\in D} s(u,v,\delta_u,\delta_v)$$

Moravec Detector (1980)



Ok, but:

- Tends to return high score for pixels on edges that are not horizontal, vertical, or 45˚
  - Why? → comparisons along 45deg directions!
- Not very efficient: calculate squared differences between same pixel pairs multiple times

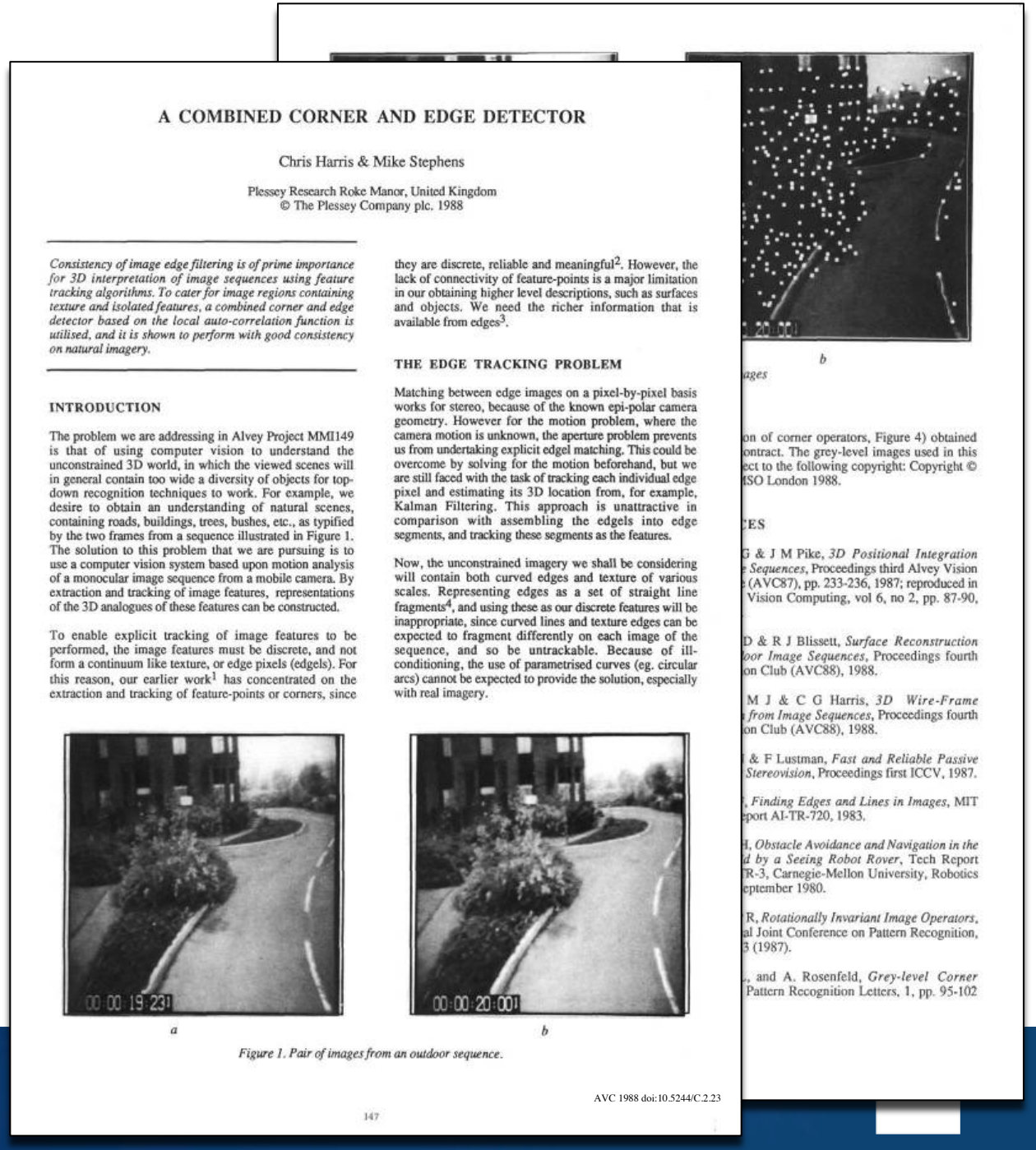


$$s(u, v, \delta_u, \delta_v) = \sum_{(i,j) \in W} (I(u + \delta_u + i, v + \delta_v + j) - I(u + i, v + j))^2$$

$$C(u, v) = \min_{(\delta_u, \delta_v) \in D} s(u, v, \delta_u, \delta_v)$$

Improvement: (1988) Harris Detector

- Addresses both problems of Moravec's detector
- Powerful idea: using image derivatives and **gradients** instead of the image itself



A COMBINED CORNER AND EDGE DETECTOR

Chris Harris & Mike Stephens

Plessey Research Roke Manor, United Kingdom
© The Plessey Company plc. 1988

Consistency of image edge filtering is of prime importance for 3D interpretation of image sequences using feature tracking algorithms. To cater for image regions containing texture and isolated features, a combined corner and edge detector based on the local auto-correlation function is utilised, and it is shown to perform with good consistency on natural imagery.

INTRODUCTION

The problem we are addressing in Alvey Project MMI149 is that of using computer vision to understand the unconstrained 3D world, in which the viewed scenes will in general contain too wide a diversity of objects for top-down recognition techniques to work. For example, we desire to obtain an understanding of natural scenes, containing roads, buildings, trees, bushes, etc., as typified by the two frames from a sequence illustrated in Figure 1. The solution to this problem that we are pursuing is to use a computer vision system based upon motion analysis of a monocular image sequence from a mobile camera. By extraction and tracking of image features, representations of the 3D analogues of these features can be constructed.

To enable explicit tracking of image features to be performed, the image features must be discrete, and not form a continuum like texture, or edge pixels (edgels). For this reason, our earlier work[1] has concentrated on the extraction and tracking of feature-points or corners, since they are discrete, reliable and meaningful[2]. However, the lack of connectivity of feature-points is a major limitation in our obtaining higher level descriptions, such as surfaces and objects. We need the richer information that is available from edges[3].

THE EDGE TRACKING PROBLEM

Matching between edge images on a pixel-by-pixel basis works for stereo, because of the known epi-polar camera geometry. However for the motion problem, where the camera motion is unknown, the aperture problem prevents us from undertaking explicit edgel matching. This could be overcome by solving for the motion beforehand, but we are still faced with the task of tracking each individual edge pixel and estimating its 3D location from, for example, Kalman Filtering. This approach is unattractive in comparison with assembling the edgels into edge segments, and tracking these segments as the features.

Now, the unconstrained imagery we shall be considering will contain both curved edges and texture of various scales. Representing edges as a set of straight line fragments[4], and using these as our discrete features will be inappropriate, since curved lines and texture edges can be expected to fragment differently on each image of the sequence, and so be untrackable. Because of ill-conditioning, the use of parametrised curves (eg. circular arcs) cannot be expected to provide the solution, especially with real imagery.

a b

Figure 1. Pair of images from an outdoor sequence.

AVC 1988 doi:10.5244/C.2.23

147

CRICOS No.00213J

Harris Corner Detector – Improve Harris (1988)

Idea:

- Find pixels where the image content changes significantly in all directions.
- These are corners and are good features.

Method:

- Calculate image derivatives in u and v directions

$$\boldsymbol{A} = w * \begin{pmatrix} J_u^2 & J_u J_v \\ J_u J_v & J_v^2 \end{pmatrix}$$

- **For each pixel:**
  - Combine derivatives into a “structure tensor” matrix
  - Investigate Eigenvalues of this matrix
  - Two large Eigenvalues: it is a corner, give it a high score.

CRICOS No.00213J

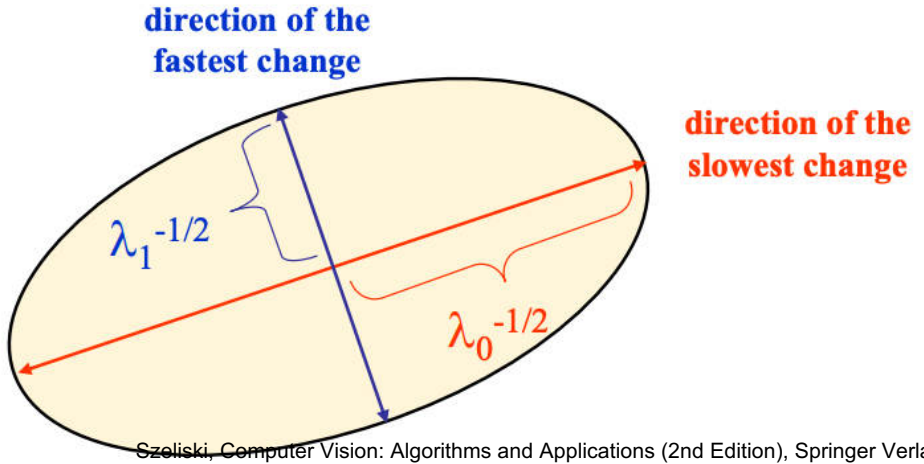
Structure Tensor: Key Property

$$\boldsymbol{A} = w * \begin{pmatrix} J_u^2 & J_u J_v \\ J_u J_v & J_v^2 \end{pmatrix}$$

Eigenvalues and eigenvectors of the structure tensor reveal important information about the direction and magnitude of local image gradients!

Intuitively, they point along the **steepest** and **flattest** directions of the local gradient.

- Eigenvectors: directions of fastest and slowest change in image intensity
- Eigenvalues: proportional to magnitude of change along those directions

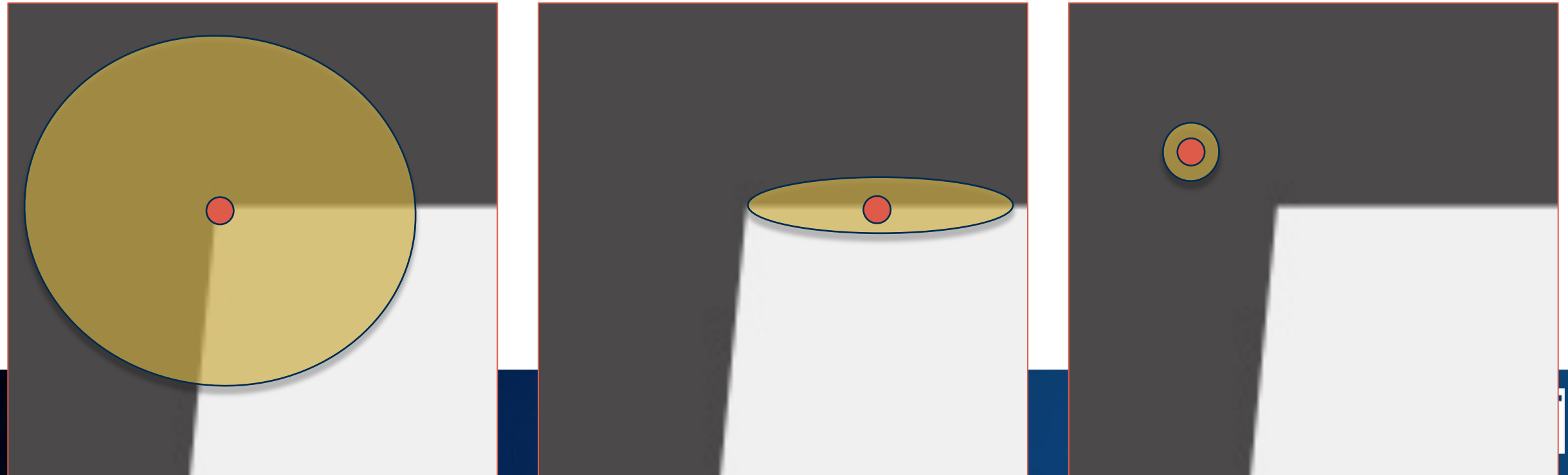


Szeliski, Computer Vision: Algorithms and Applications (2nd Edition), Springer Verlag

Structure Tensor: Key Property

$$\boldsymbol{A} = w * \begin{pmatrix} J_u^2 & J_u J_v \\ J_u J_v & J_v^2 \end{pmatrix}$$

- **Two large eigenvalues**: the pixel is at a **corner**, where intensity changes significantly in both directions.
- **One large, one small eigenvalue**: the pixel lies on an **edge**, where intensity changes in one direction only.
- **Two small eigenvalues**: the pixel is in a **flat region** with little to no intensity change.

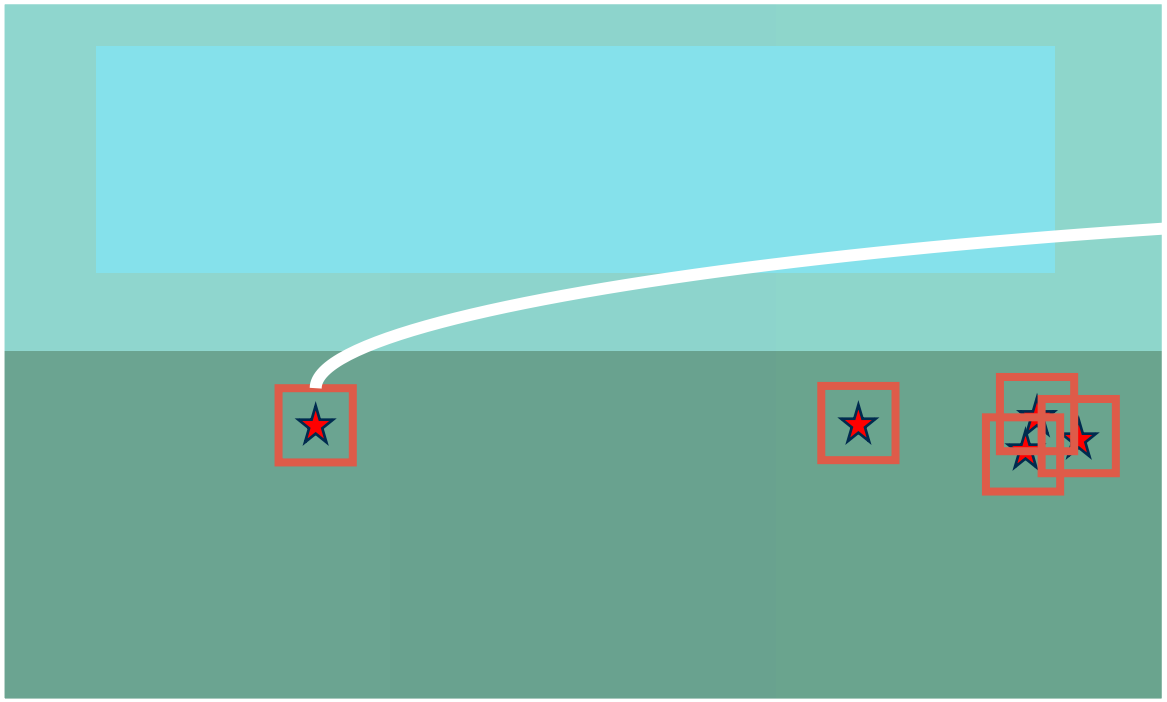


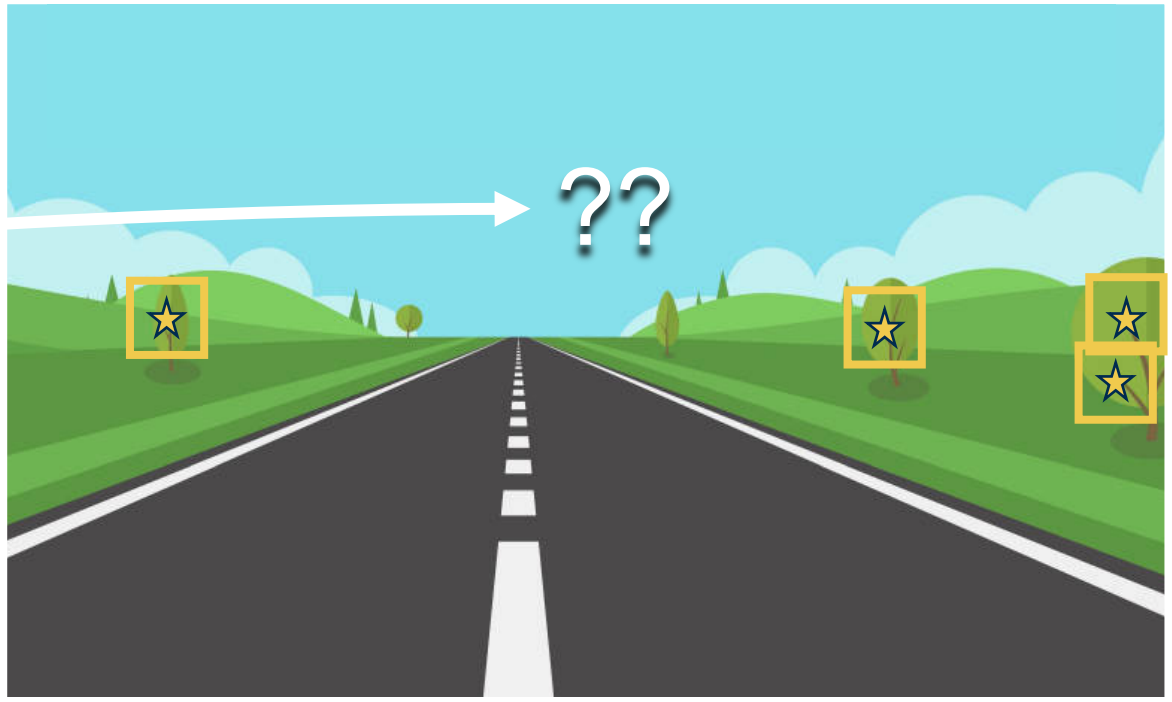
Feature Descriptors

How do we recognise it’s the same feature?

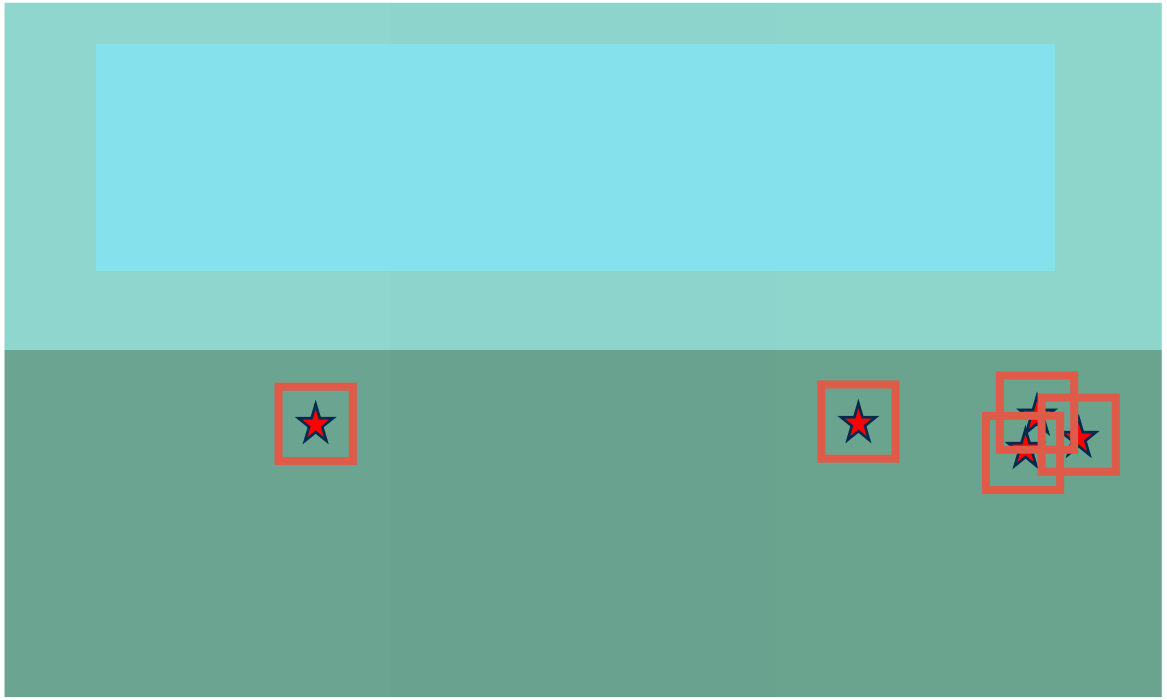


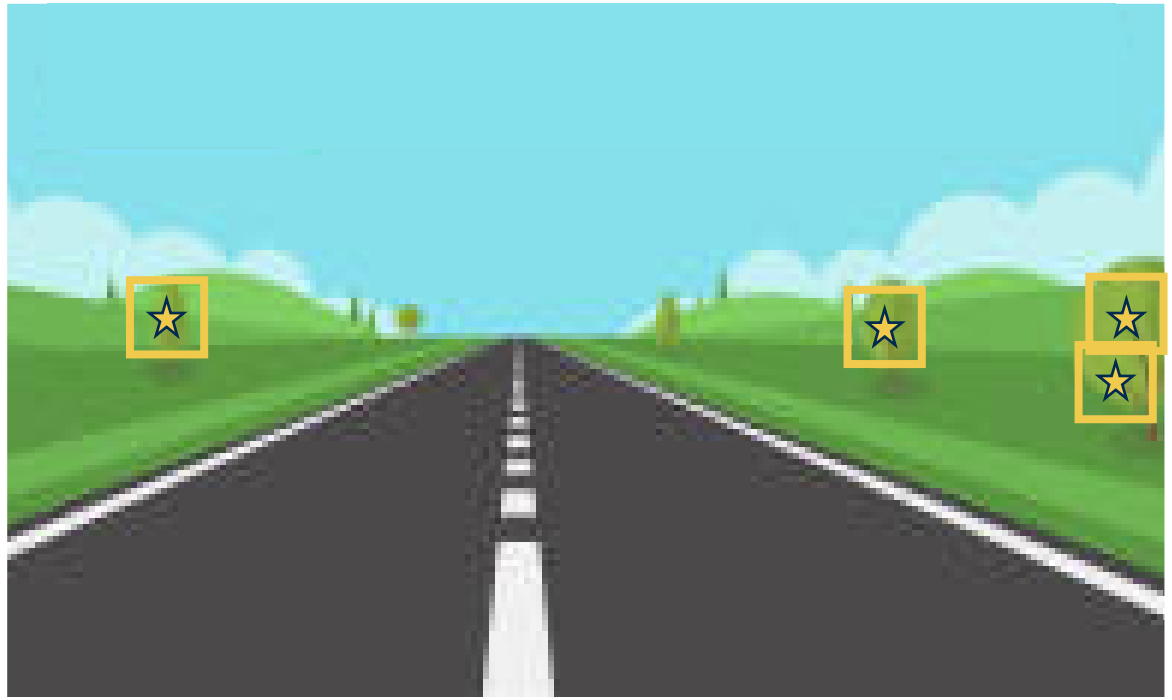
How can we match features?



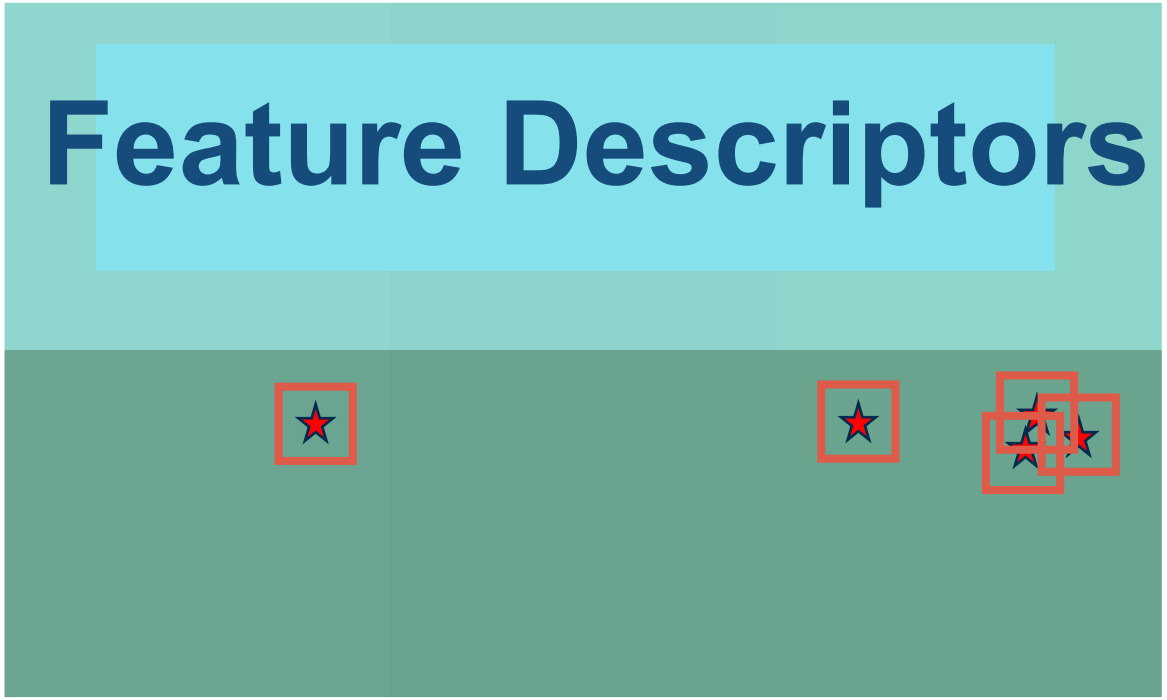


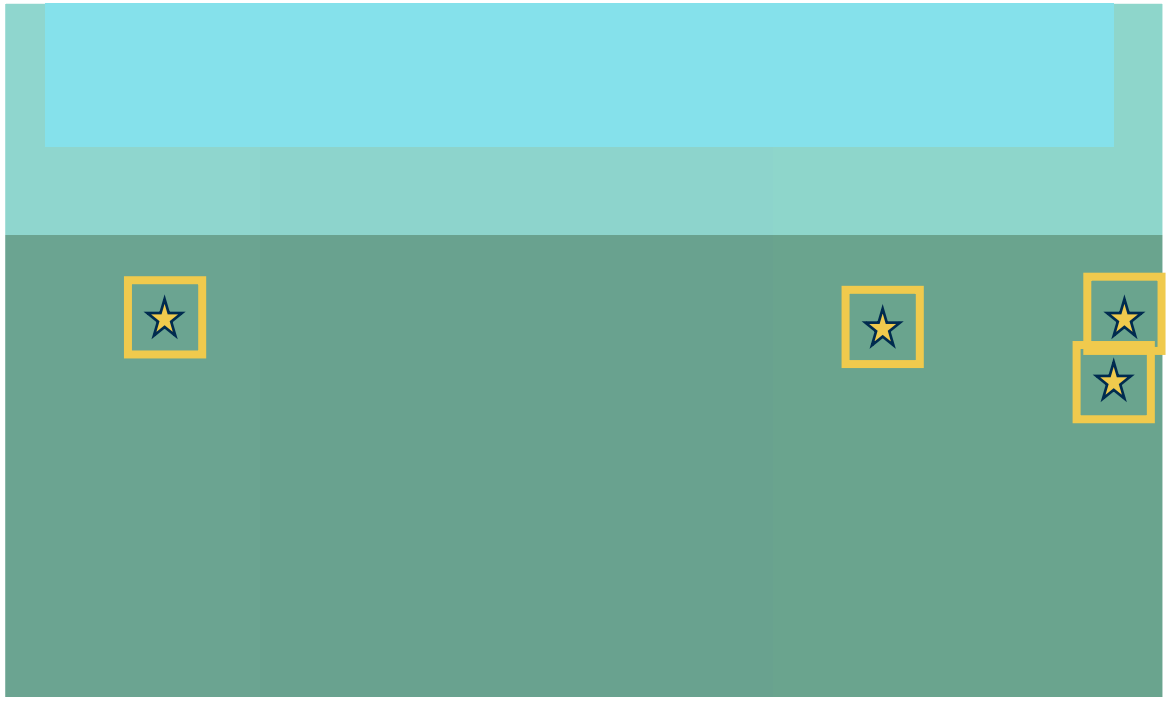
CRICOS No.00213J



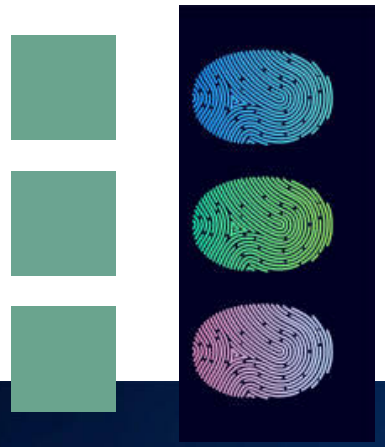


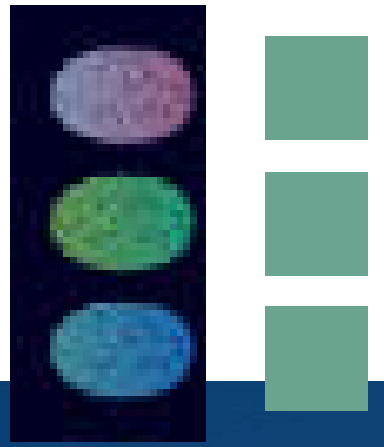
Feature Descriptors





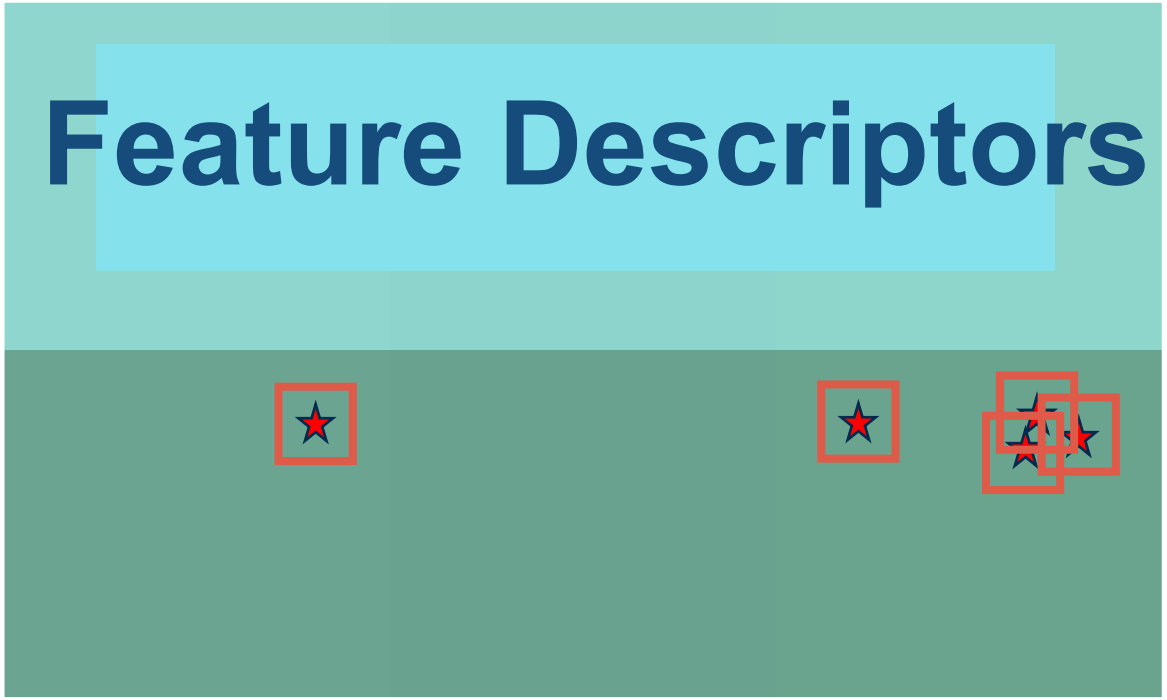
Idea: give each feature a descriptor – a “fingerprint” – that is a unique identifier

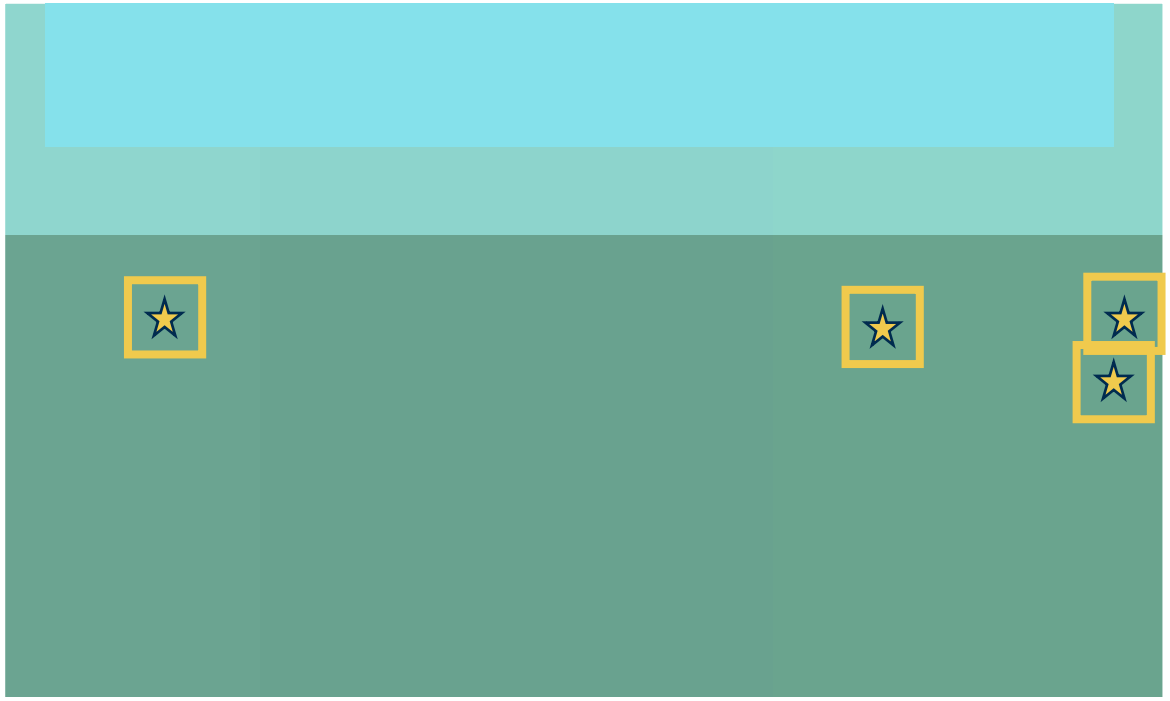




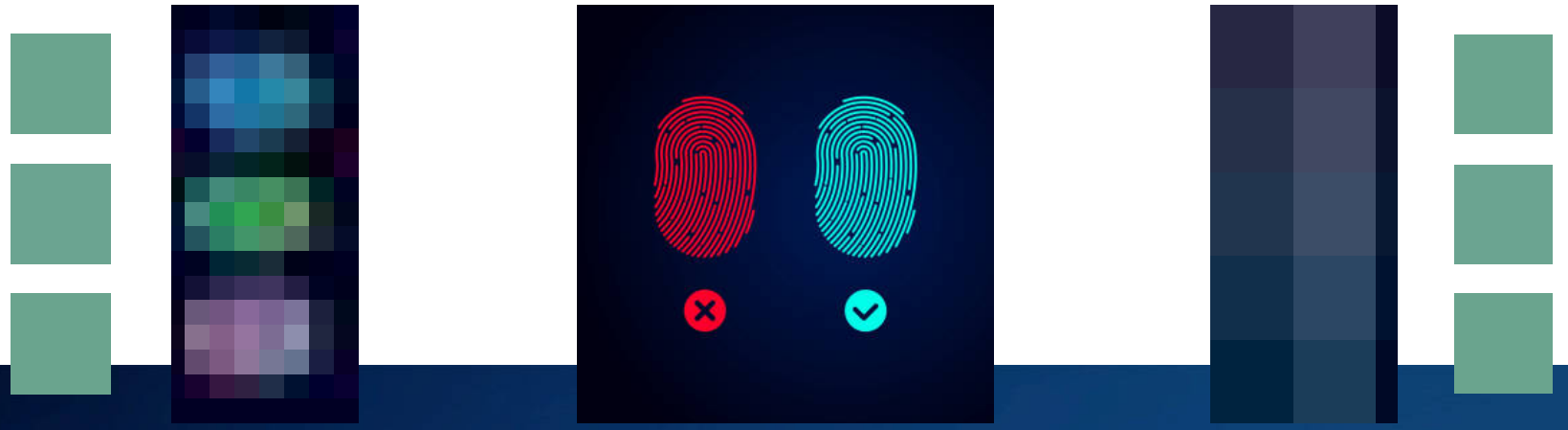
CRICOS No.00213J

Feature Descriptors

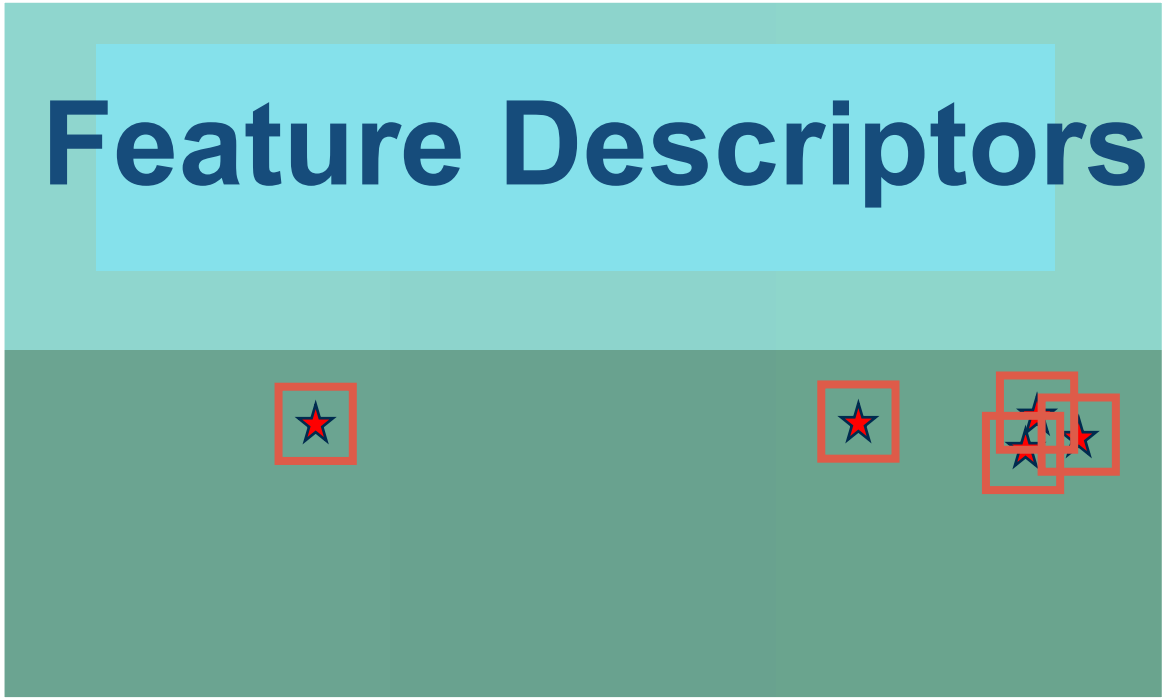


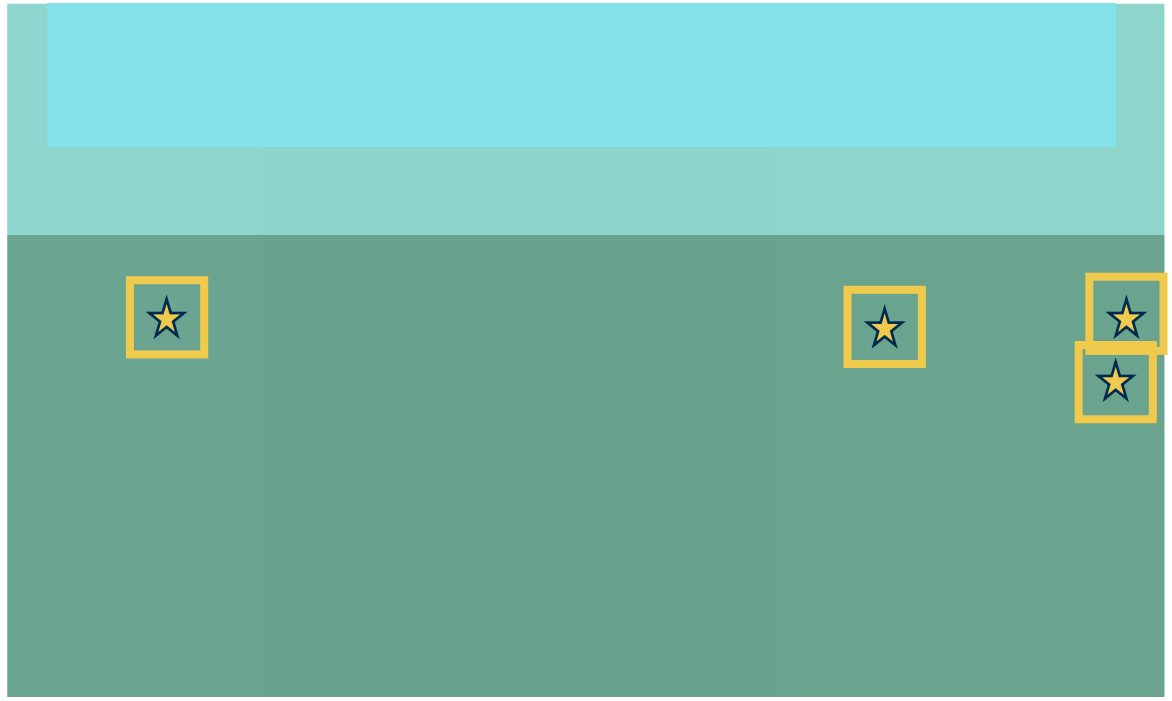


Idea: give each feature a descriptor and then match them based on similarity

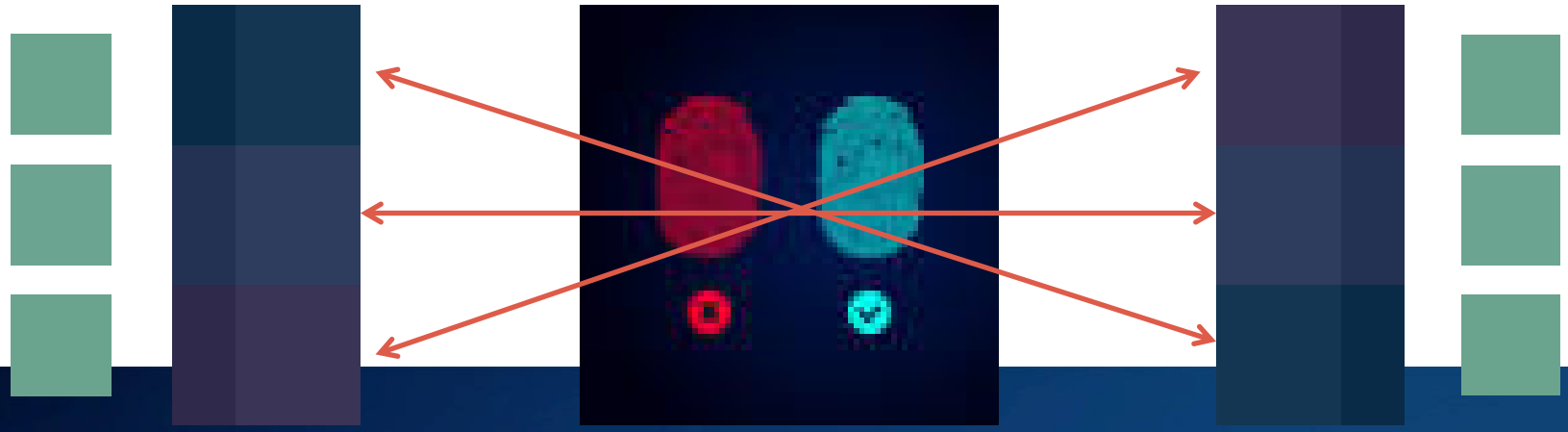


Feature Descriptors



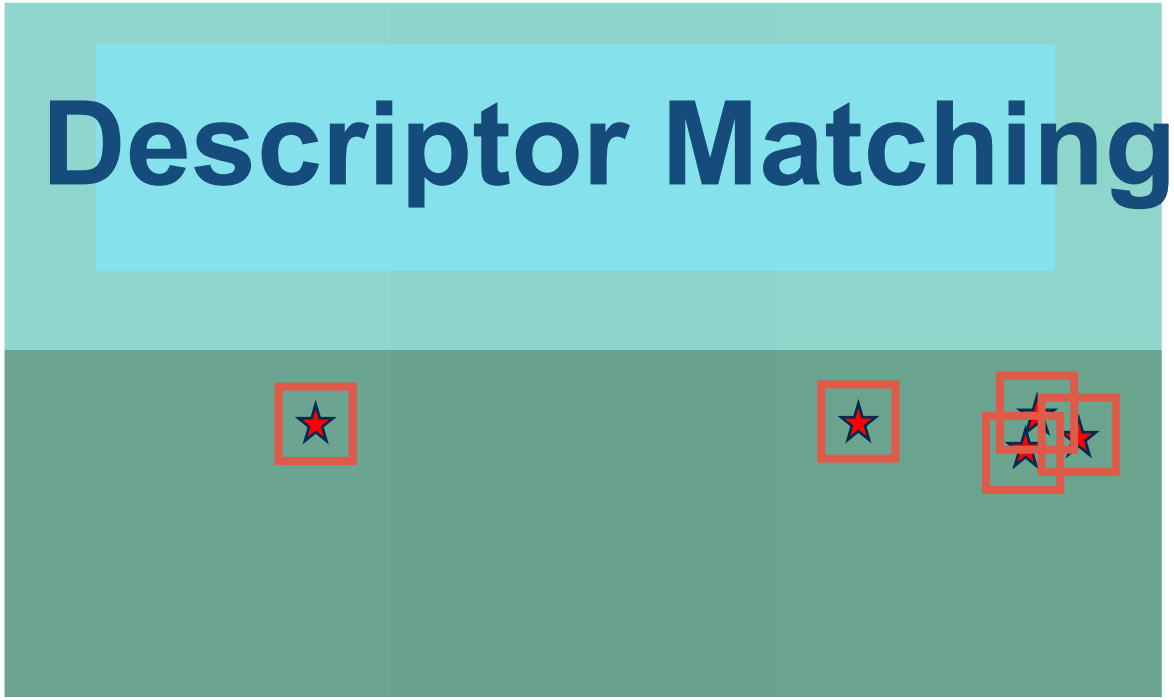


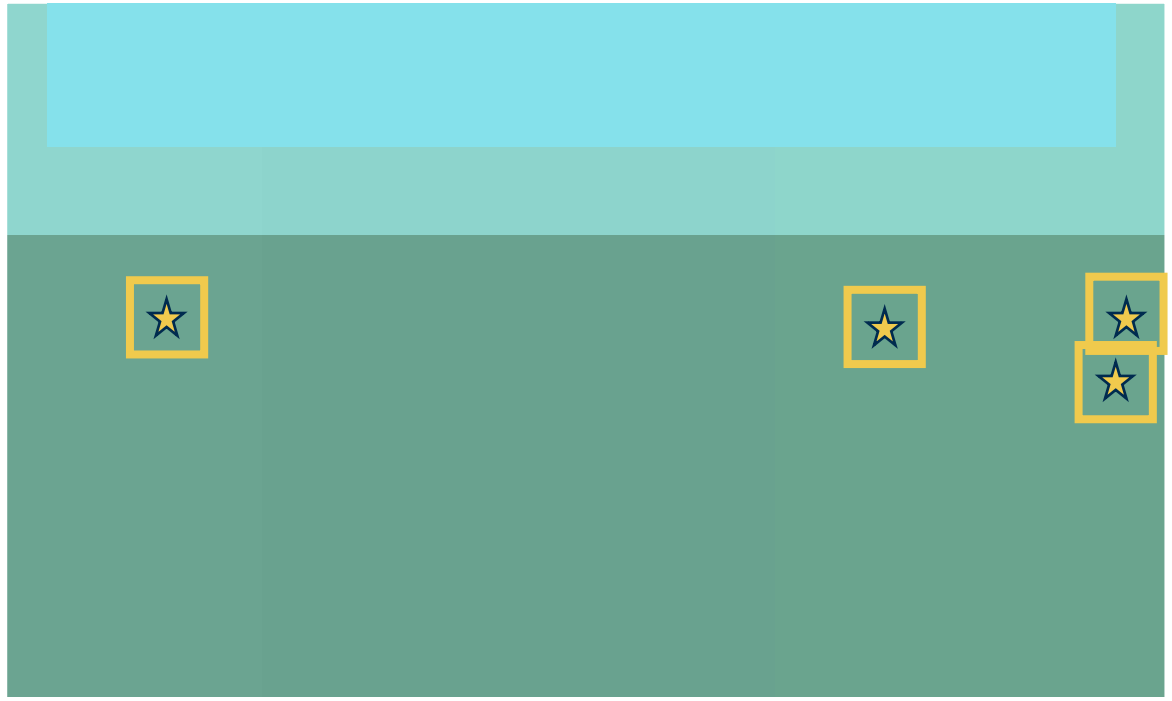
Idea: give each feature a descriptor and then match them based on similarity



CRICOS No.00213J

Descriptor Matching





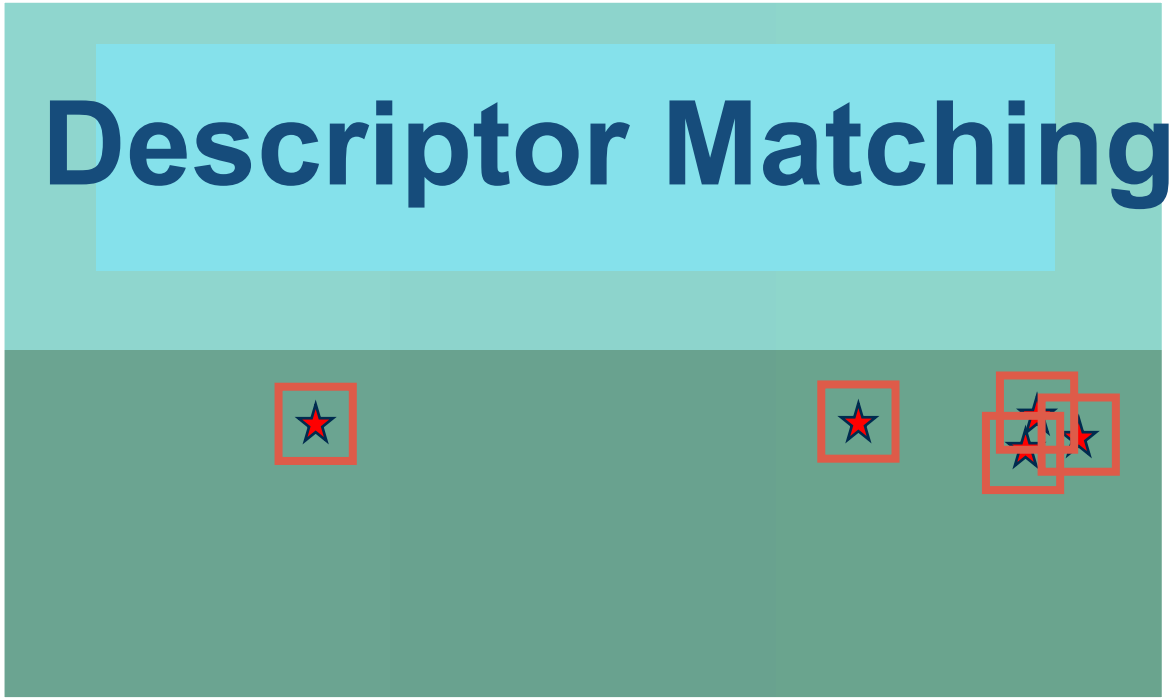
Naïve idea: simply use the pixel values from the patch pixels around the feature and compare the pixel values

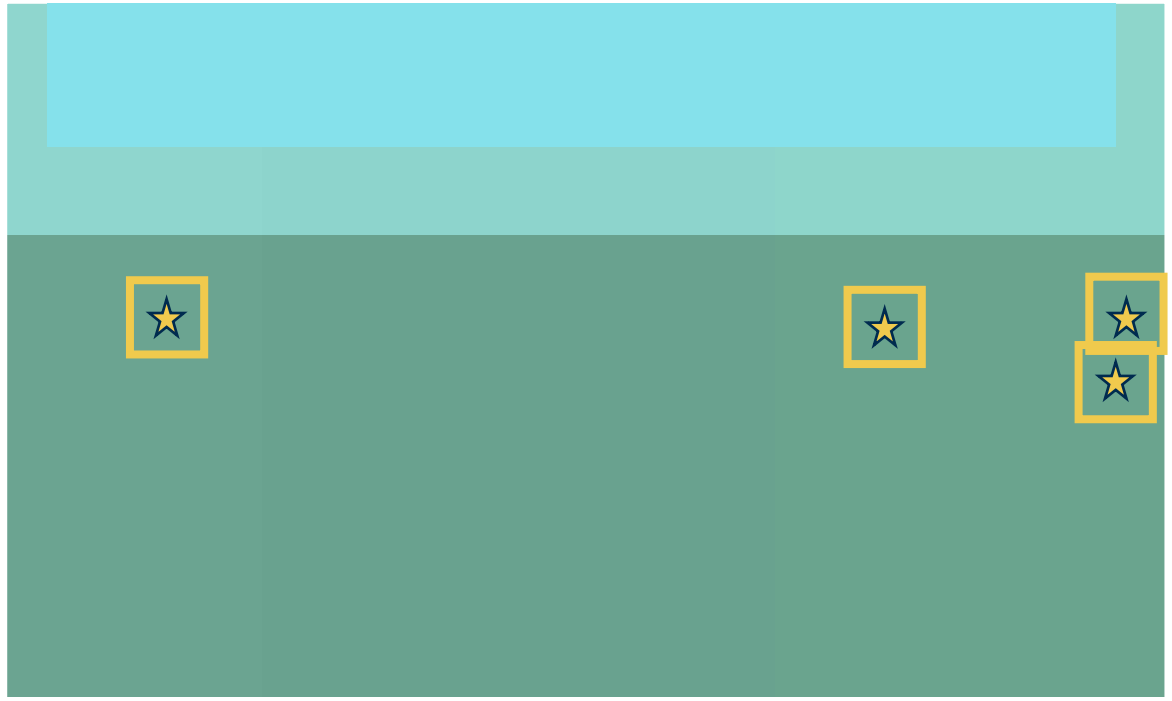
Sum of Squared Differences (SSD) $\sum(\blacksquare - \blacksquare)^2$

Sum of Absolute Differences (SAD) $\sum \mathrm{abs}(\blacksquare - \blacksquare)$

CRICOS No.00213J

Descriptor Matching





Naïve idea: simply use the pixel values from the patch pixels around the feature and compare the pixel values.

Why is this not a great idea?

Sum of Squared Differences (SSD)

$$\sum(\blacksquare - \blacksquare)^2$$

Sum of Absolute Differences (SAD)

$$\sum \text{abs}(\blacksquare - \blacksquare)$$

SIFT: Scale Invariant Feature Transform

A Detector and Descriptor



SIFT – Scale Invariant Feature Transform (1999, 2004)

- Goal: produce features that remain reliable despite changes in:
  - Scale
  - Rotation
  - Moderate perspective distortions
  - Illumination
  - Noise

Idea:

- Use gradient information to establish principal orientation, and achieve orientation invariance.
- Use “scale space” to achieve scale invariance.

CRICOS No.00213J

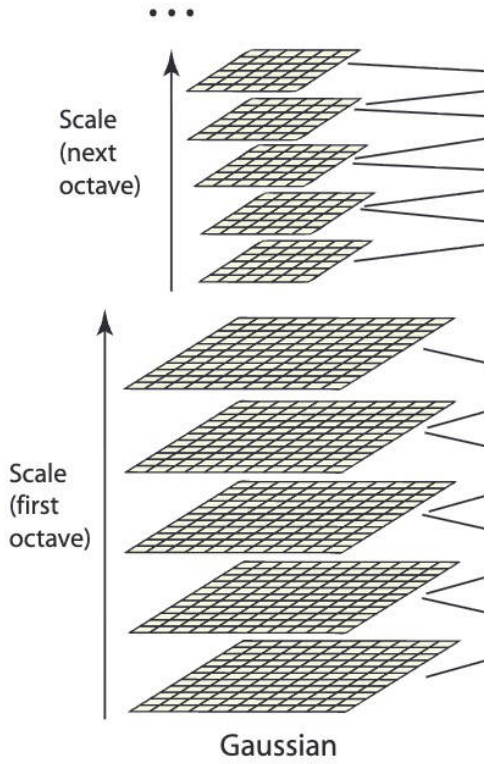
SIFT – Scale Space Idea

Idea: To be scale-invariant, don't just search good features across (u,v) but also across different image **scales**!

This "simulates" seeing the scene from further and further away.



CRICOS No.00213J



Lowe, David G. (2004). "Distinctive Image Features from Scale-Invariant Keypoints". International Journal of Computer Vision.

SIFT – Scale Space Idea

Subtract the blurred images → **Difference of Gaussian**

$$D(u, v, \sigma) = L(u, v, k\sigma) - L(u, v, \sigma)$$

The resulting DoG image highlights structures that stand out at a particular scale.



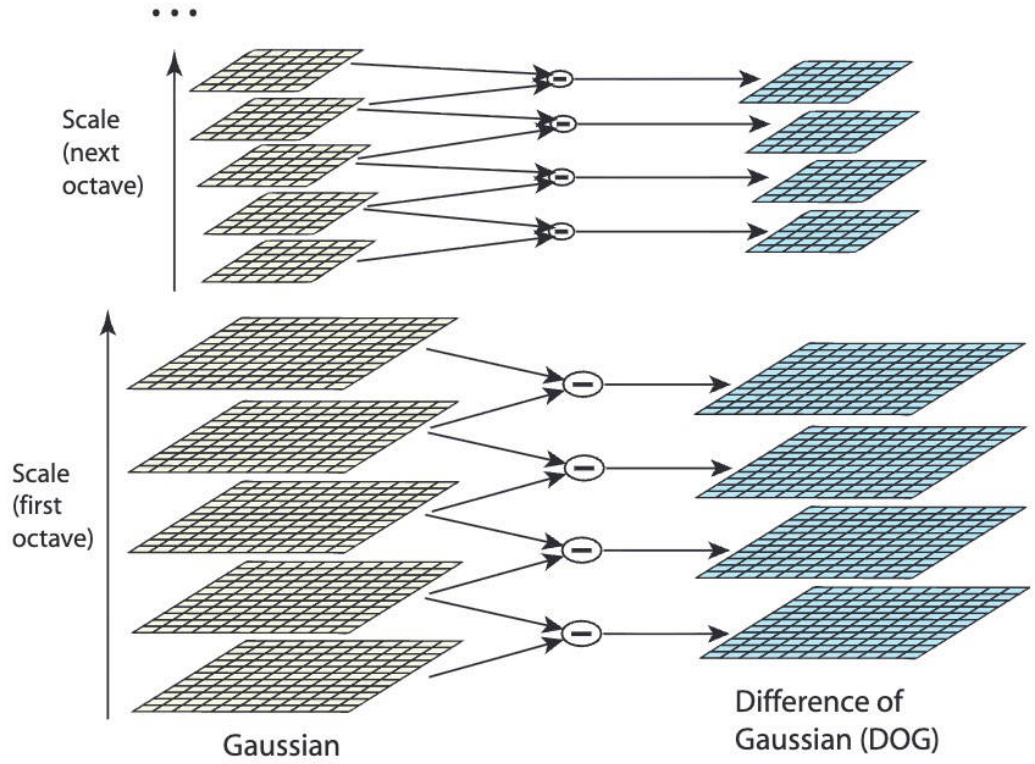
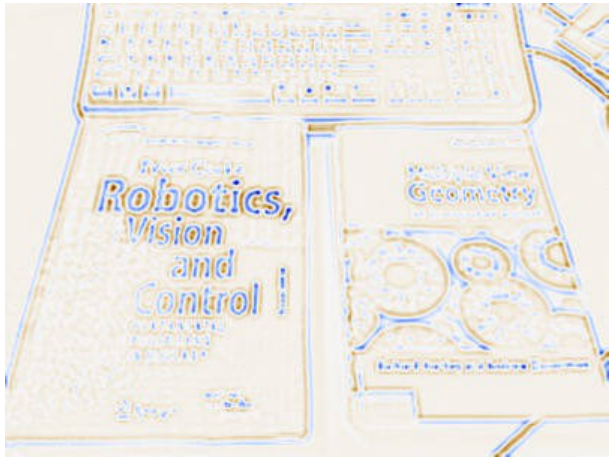


Figure 1: For each octave of scale space, the initial image is repeatedly convolved with Gaussians to produce the set of scale space images shown on the left. Adjacent Gaussian images are subtracted to produce the difference-of-Gaussian images on the right. After each octave, the Gaussian image is down-sampled by a factor of 2, and the process repeated.

SIFT – Scale Space Idea

Find candidate feature points as local maxima or minima by comparing each pixel with its neighbours in the same image, and in the scale above and below.

A candidate must be locally distinctive in both **space and scale**.



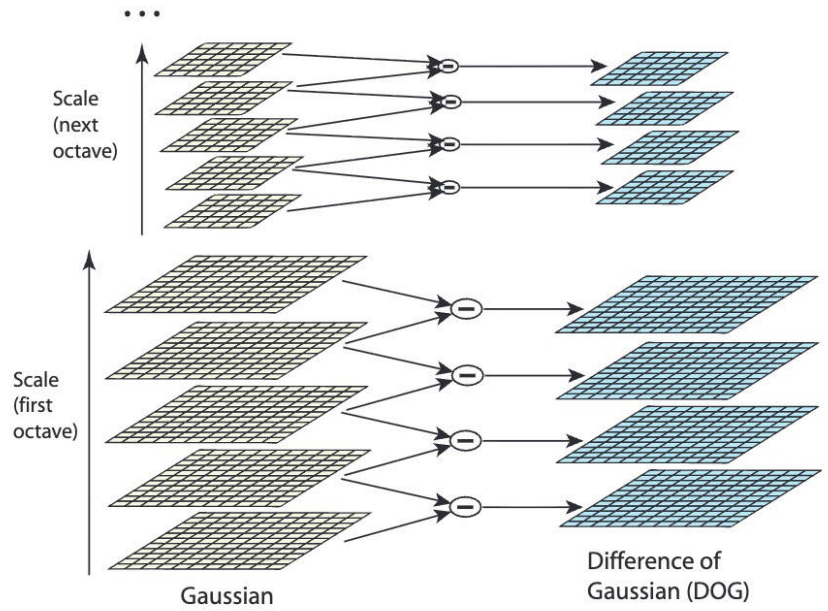
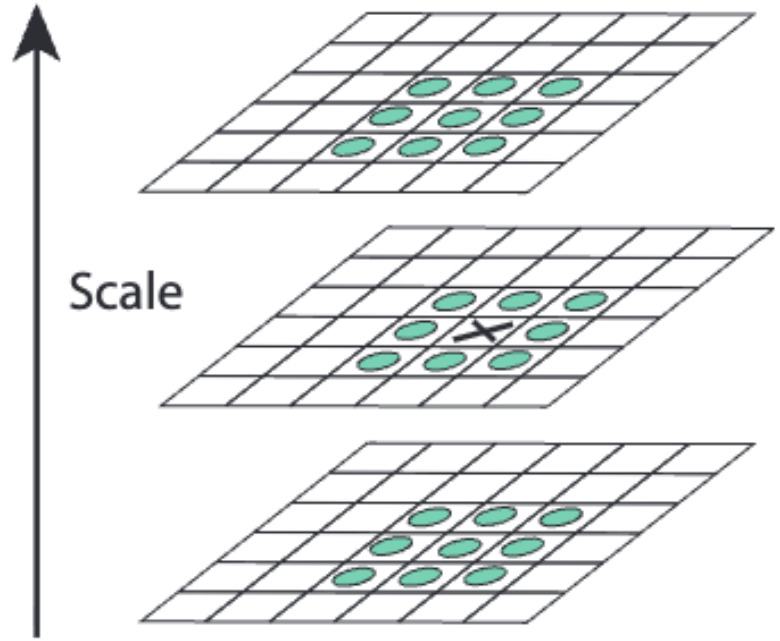


Figure 1: For each octave of scale space, the initial image is repeatedly convolved with Gaussians to produce the set of scale space images shown on the left. Adjacent Gaussian images are subtracted to produce the difference-of-Gaussian images on the right. After each octave, the Gaussian image is down-sampled by a factor of 2, and the process repeated.



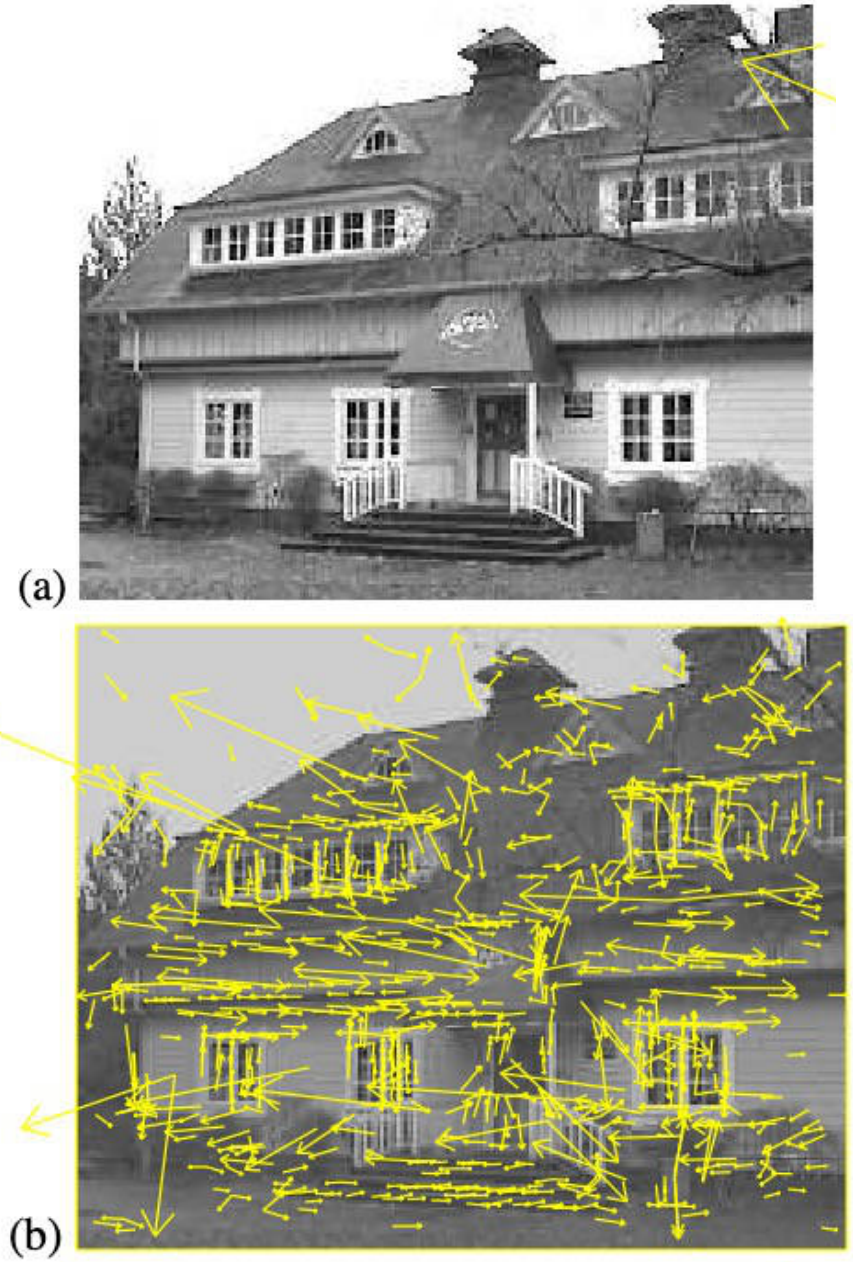
CRICOS No.00213J

Lowe, David G. (2004). "Distinctive Image Features from Scale-Invariant Keypoints". International Journal of Computer Vision.

SIFT – Orientation

Assign an **orientation** to each candidate

- For nearby pixels, calculate:
  - gradient magnitude
  - gradient orientation
- Accumulate orientations in a histogram
- Strongest orientation is the keypoint's assigned orientation.



CRICOS No.00213J

Lowe, David G. (2004). "Distinctive Image Features from Scale-Invariant Keypoints". International Journal of Computer Vision.

SIFT – Clean Up

Remove potentially **bad features**:

- edge-like
- Low-contrast
- Badly localised

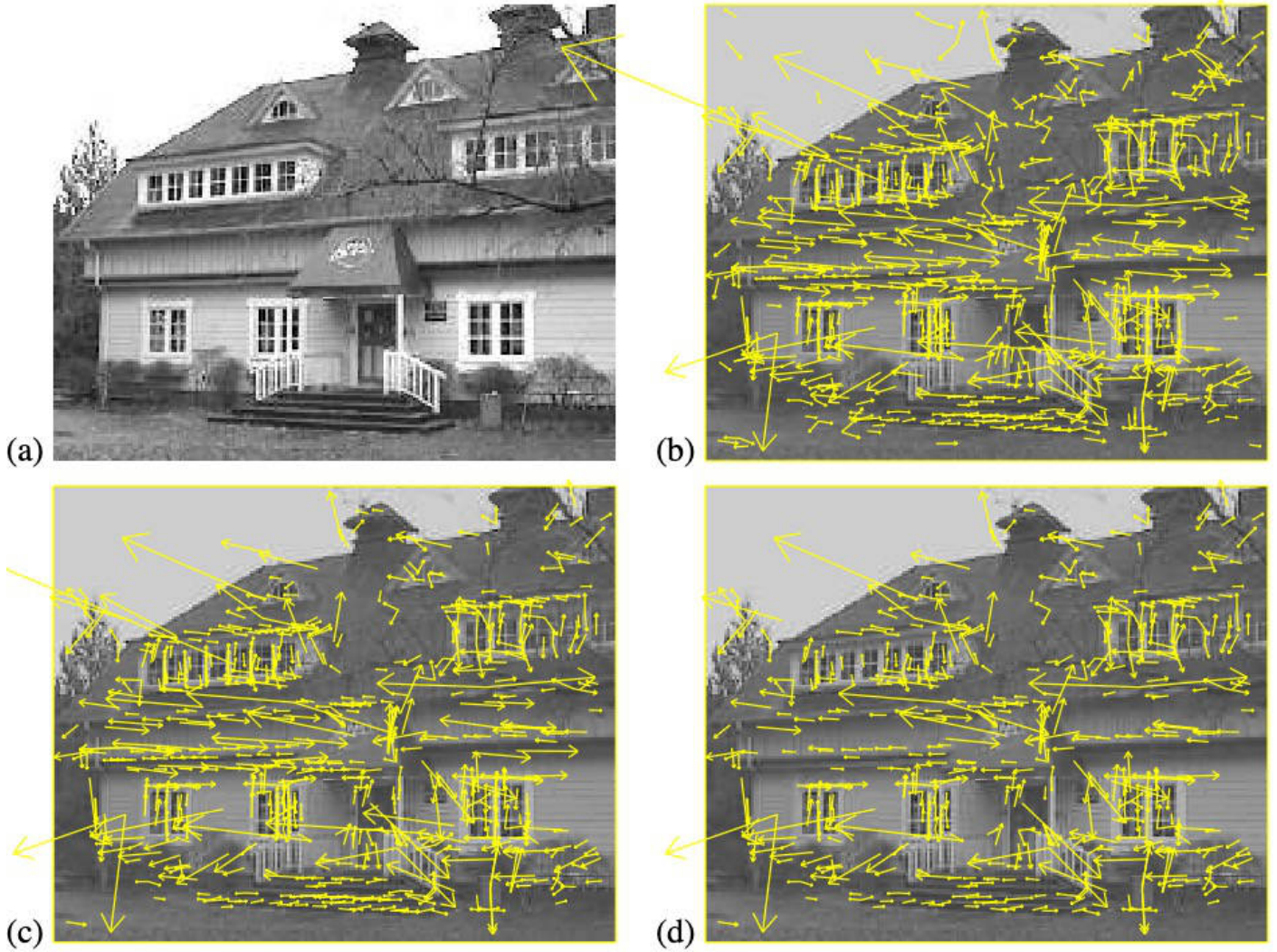


Figure 5: This figure shows the stages of keypoint selection. (a) The 233x189 pixel original image. (b) The initial 832 keypoints locations at maxima and minima of the difference-of-Gaussian function. Keypoints are displayed as vectors indicating scale, orientation, and location. (c) After applying a threshold on minimum contrast, 729 keypoints remain. (d) The final 536 keypoints that remain following an additional threshold on ratio of principal curvatures.

CRICOS No.00213J

Lowe, David G. (2004). "Distinctive Image Features from Scale-Invariant Keypoints". International Journal of Computer Vision.

SIFT – Descriptor

- For pixels around the keypoint:
  - Calculate gradient magnitude and orientation.
  - Divide the region into 4×4 subregions.
  - Build an orientation histogram within each subregion.
- Concatenate the 16 histograms to form the descriptor.

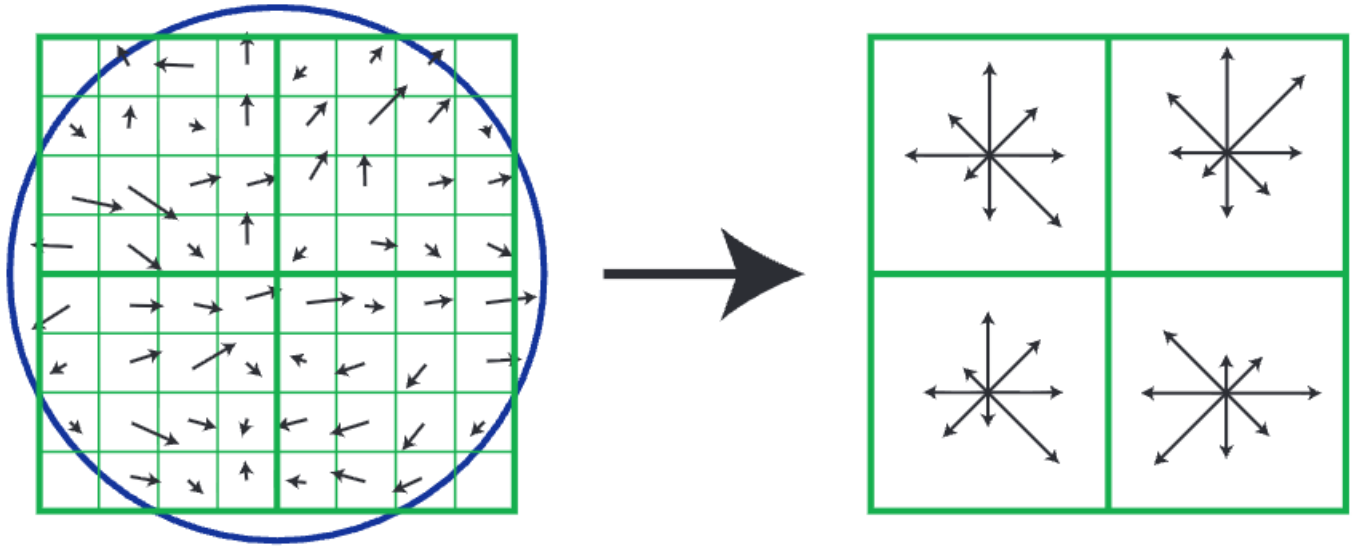
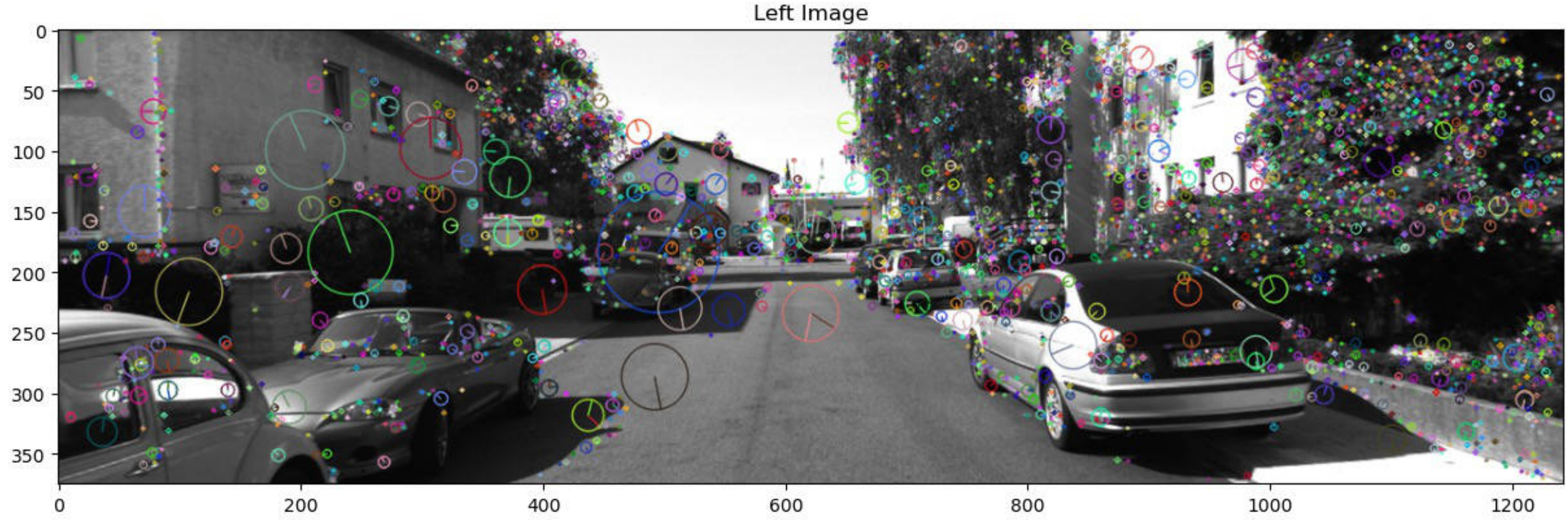


Figure 7: A keypoint descriptor is created by first computing the gradient magnitude and orientation at each image sample point in a region around the keypoint location, as shown on the left. These are weighted by a Gaussian window, indicated by the overlaid circle. These samples are then accumulated into orientation histograms summarizing the contents over 4x4 subregions, as shown on the right, with the length of each arrow corresponding to the sum of the gradient magnitudes near that direction within the region. This figure shows a 2x2 descriptor array computed from an 8x8 set of samples, whereas the experiments in this paper use 4x4 descriptors computed from a 16x16 sample array.

CRICOS No.00213J

Lowe, David G. (2004). "Distinctive Image Features from Scale-Invariant Keypoints". International Journal of Computer Vision.

Keypoints detected with SIFT



CRICOS No.00213J

BRIEF: Binary Robust Independent Elementary Features

A descriptor built for speed



BRIEF (2010)

Idea:

- Build a binary descriptor that is highly efficient, by chaining the results of simple pixel brightness comparisons



BRIEF: Binary Robust Independent Elementary Features*

Michael Calonder, Vincent Lepetit, Christoph Strecha, and Pascal Fua

CVLab, EPFL, Lausanne, Switzerland
e-mail: firstname.lastname@epfl.ch

Abstract. We propose to use binary strings as an efficient feature point descriptor, which we call BRIEF. We show that it is highly discriminative even when using relatively few bits and can be computed using simple intensity difference tests. Furthermore, the descriptor similarity can be evaluated using the Hamming distance, which is very efficient to compute, instead of the L_2 norm as is usually done.
As a result, BRIEF is very fast both to build and to match. We compare it against SURF and U-SURF on standard benchmarks and show that it yields a similar or better recognition performance, while running in a fraction of the time required by either.

1 Introduction

Feature point descriptors are now at the core of many Computer Vision technologies, such as object recognition, 3D reconstruction, image retrieval, and camera localization. Since applications of these technologies have to handle ever more data or to run on mobile devices with limited computational resources, there is a growing need for local descriptors that are fast to compute, fast to match, and memory efficient.

One way to speed up matching and reduce memory consumption is to work with short descriptors. They can be obtained by applying dimensionality reduction, such as PCA [1] or LDA [2], to an original descriptor such as SIFT [3] or SURF [4]. For example, it was shown in [5–7] that floating point values of the descriptor vector could be quantized using very few bits per value without loss of recognition performance. An even more drastic dimensionality reduction can be achieved by using hash functions that reduce SIFT descriptors to binary strings, as done in [8]. These strings represent binary descriptors whose similarity can be measured by the Hamming distance.

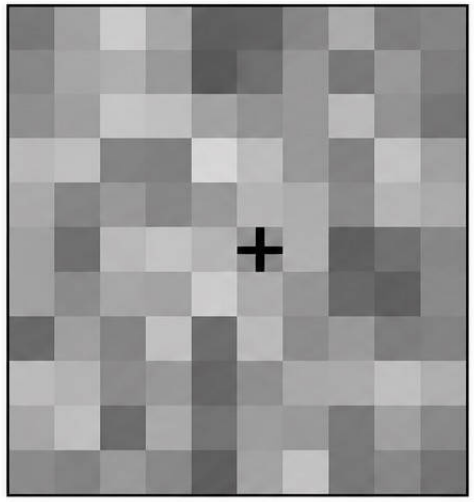
While effective, these approaches to dimensionality reduction require first computing the full descriptor before further processing can take place. In this paper, we show that this whole computation can be shortcut by *directly* computing binary strings from image patches. The individual bits are obtained by comparing the intensities of pairs of points along the same lines as in [9] but without requiring a training phase. We refer to the resulting descriptor as BRIEF.

*This work has been supported in part by the Swiss National Science Foundation.

Calonder, M., Lepetit, V., Strecha, C., & Fua, P. (2010).
Brief: Binary robust independent elementary features. European Conference on Computer Vision.

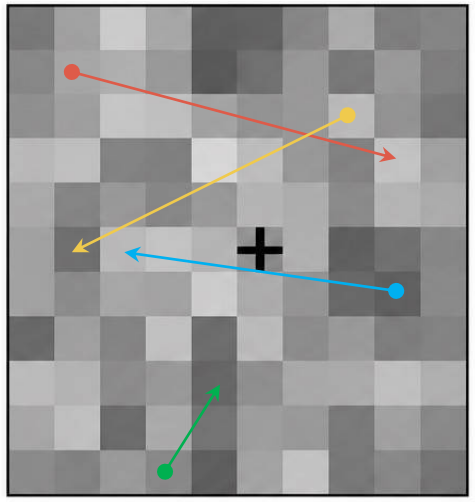
BRIEF – A simple idea

Patch around keypoint



Take a small image patch around the feature

Pixel intensity comparisons



Compare intensities of random pairs of pixels

Each comparison gives 1 bit: if I(p) < I(q), then 1, else 0

Binary descriptor

Comparison	Bit
I(p) < I(q)	0
I(p) > I(q)	1
I(p) < I(q)	0
I(p) > I(q)	1

[0,1,0,1,…]

Concatenate all comparisons to form the binary descriptor.

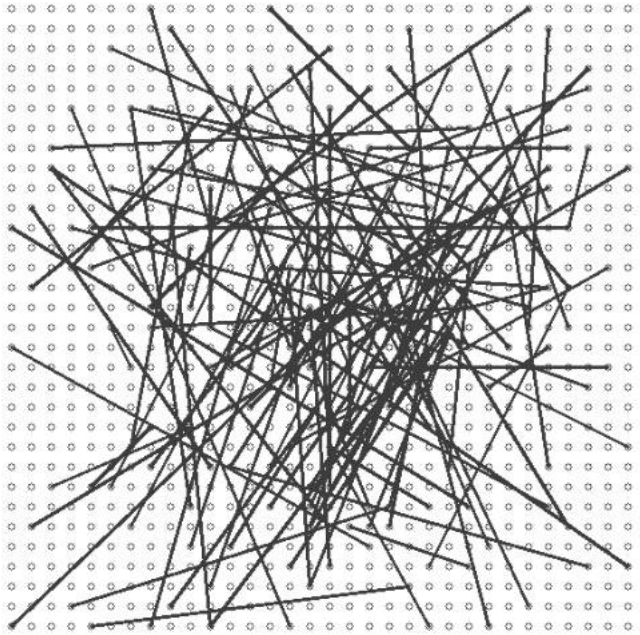
E.g. 256 bits long

CRICOS No.00213J

BRIEF – Advantages

- Very fast!

- Binary string matching = XOR + bitcount
- No gradients or floating-point ops required
- Descriptors are short (e.g., 32 bytes)
- Very fast matching with Hamming distance – suitable for real-time



CRICOS No.00213J

Calonder, M., Lepetit, V., Strecha, C., & Fua, P. (2010).
Brief: Binary robust independent elementary features. European Conference on Computer Vision.

Limitations of BRIEF

- Not rotation invariant – sensitive to in-plane rotation
- Not scale invariant – fixed patch size
- Must be paired with a stable keypoint detector (e.g., FAST, ORB)

CRICOS No.00213J

Calonder, M., Lepetit, V., Strecha, C., & Fua, P. (2010).
Brief: Binary robust independent elementary features. European Conference on Computer Vision.

FAST

A detector built for speed



FAST (2006)

- Idea:
  - Speed up the detection of feature points by **learning** to detect them

Machine learning for high-speed corner detection

Edward Rosten and Tom Drummond

Department of Engineering, Cambridge University, UK
{er258, twd20}@cam.ac.uk

Abstract Where feature points are used in real-time frame-rate applications, a high-speed feature detector is necessary. Feature detectors such as SIFT (DoG), Harris and SUSAN are good methods which yield high quality features, however they are too computationally intensive for use in real-time applications of any complexity. Here we show that machine learning can be used to derive a feature detector which can fully process live PAL video using less than 7% of the available processing time. By comparison neither the Harris detector (120%) nor the detection stage of SIFT (300%) can operate at full frame rate.
Clearly a high-speed detector is of limited use if the features produced are unsuitable for downstream processing. In particular, the same scene viewed from two different positions should yield features which correspond to the same real-world 3D locations[1]. Hence the second contribution of this paper is a comparison corner detectors based on this criterion applied to 3D scenes. This comparison supports a number of claims made elsewhere concerning existing corner detectors. Further, contrary to our initial expectations, we show that despite being principally constructed for speed, our detector significantly outperforms existing feature detectors according to this criterion.

1 Introduction

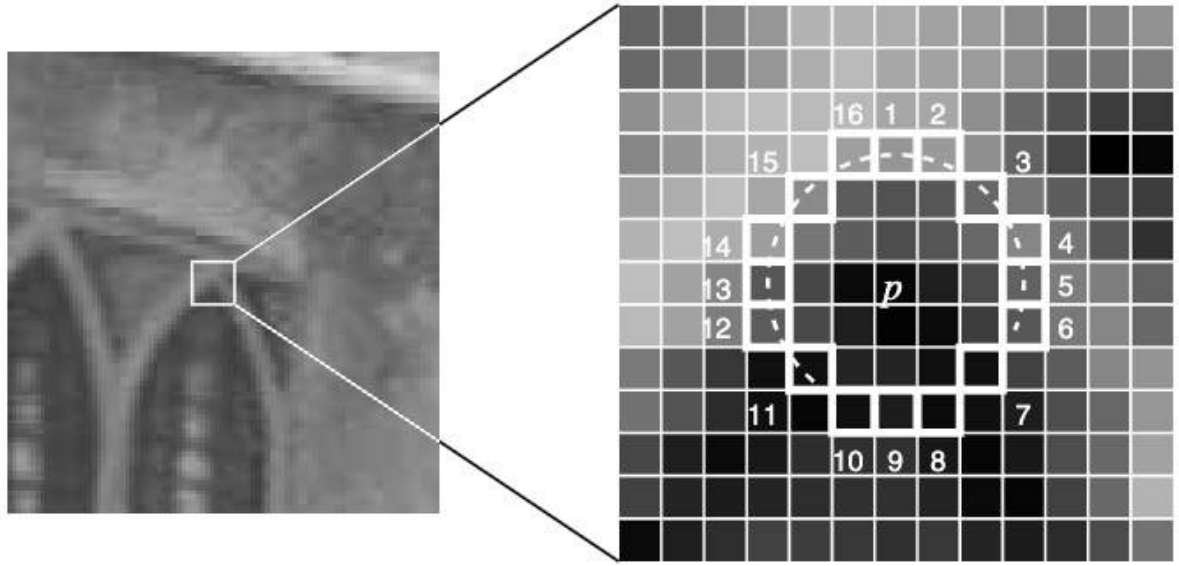
Corner detection is used as the first step of many vision tasks such as tracking, SLAM (simultaneous localisation and mapping), localisation, image matching and recognition. Hence, a large number of corner detectors exist in the literature. With so many already available it may appear unnecessary to present yet another detector to the community; however, we have a strong interest in real-time frame rate applications such as SLAM in which computational resources are at a premium. In particular, it is still true that when processing live video streams at full frame rate, existing feature detectors leave little if any time for further processing, even despite the consequences of Moore's Law.

Section 2 of this paper demonstrates how a feature detector described in earlier work can be redesigned employing a machine learning algorithm to yield a large speed increase. In addition, the approach allows the detector to be generalised, producing a suite of high-speed detectors which we currently use for real-time tracking [2] and AR label placement [3].

To show that speed can been obtained without necessarily sacrificing the quality of the feature detector we compare our detector, to a variety of well-known detectors. In Section 3 this is done using Schmid's criterion [1], that

FAST (2006)

- Underlying idea: find corners by comparing brightness values
  - At each candidate pixel p, consider a **circle of 16 pixels**
  - If **N contiguous pixels** in that circle are all **brighter or darker than the center** pixel p by a threshold, mark p as a **corner**. (typically N >= 12)



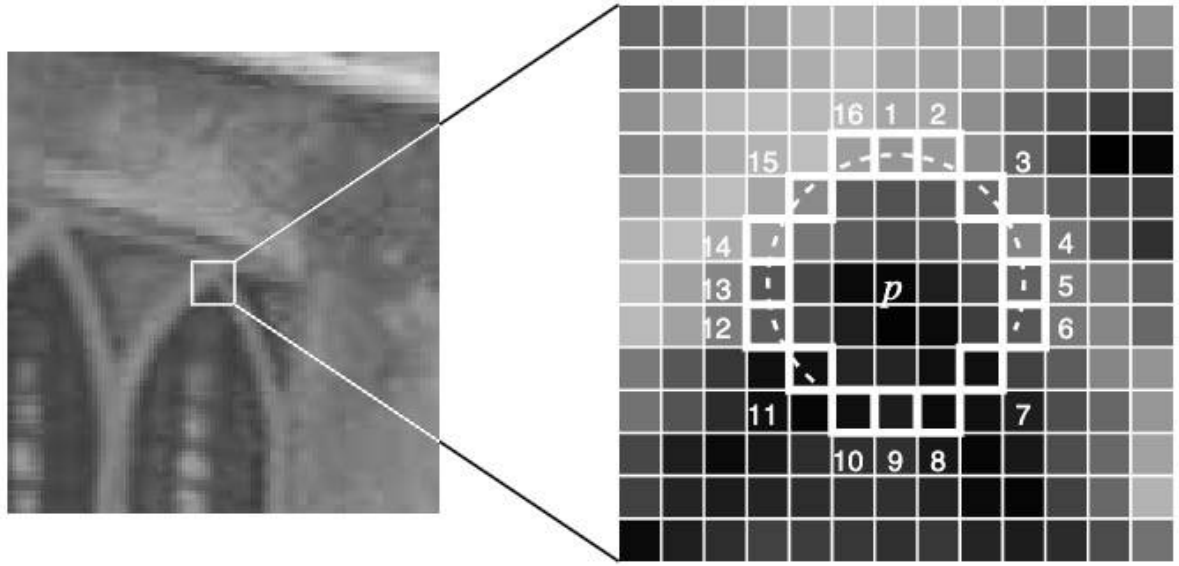
CRICOS No.00213J

Rosten, E., & Drummond, T. (2006). Machine learning for high-speed corner detection. In *European conference on computer vision*

FAST (2006)

- Underlying idea: find corners by comparing brightness values
  - At each candidate pixel p, consider a **circle of 16 pixels**
  - If **N contiguous pixels** in that circle are all **brighter or darker than the center** pixel p by a threshold, mark p as a **corner**. (typically N >= 12)

- A simple idea can speed this up:
  - Early **rejection test** checks only four pixels (pixels 1, 9, 5, 13)
  - If p is a corner, then three of them must be brighter or darker than p
  - Only if this test is passed, are the other pixels looked at

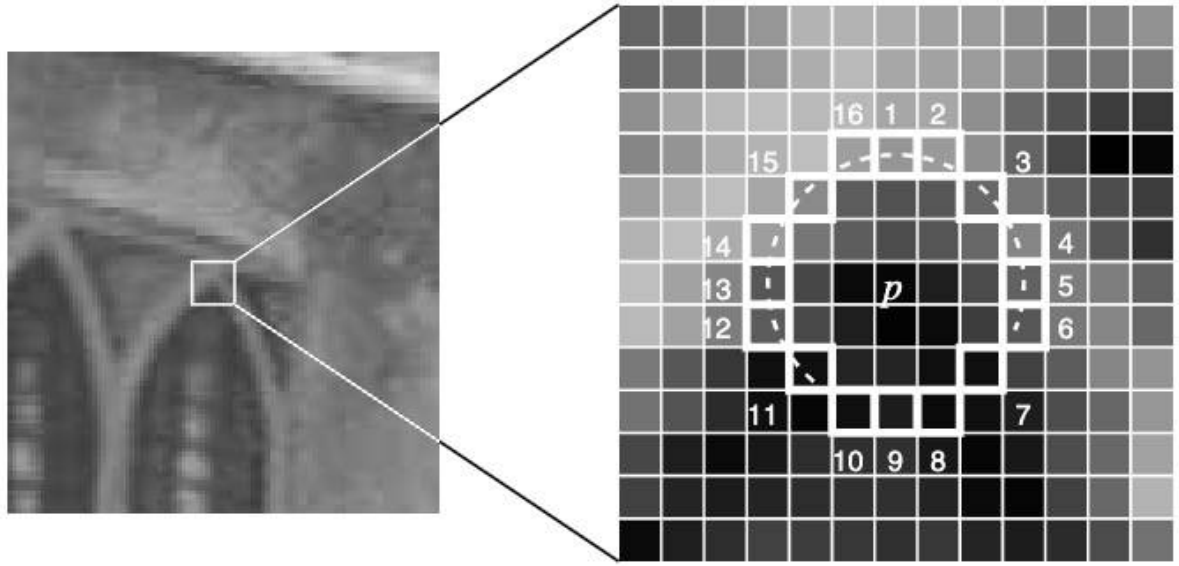


CRICOS No.00213J

Rosten, E., & Drummond, T. (2006). Machine learning for high-speed corner detection. In *European conference on computer vision*

FAST (2006)

- Underlying idea: find corners by comparing brightness values
- Even better:
  - Instead of hand-designed rules, **learn** which pixels to look at, and in which order!
  - Training dataset processed with the full variant
  - Then learn a decision tree
  - Compiled into a long list of nested if-then-else statements



CRICOS No.00213J

Rosten, E., & Drummond, T. (2006). Machine learning for high-speed corner detection. In *European conference on computer vision*

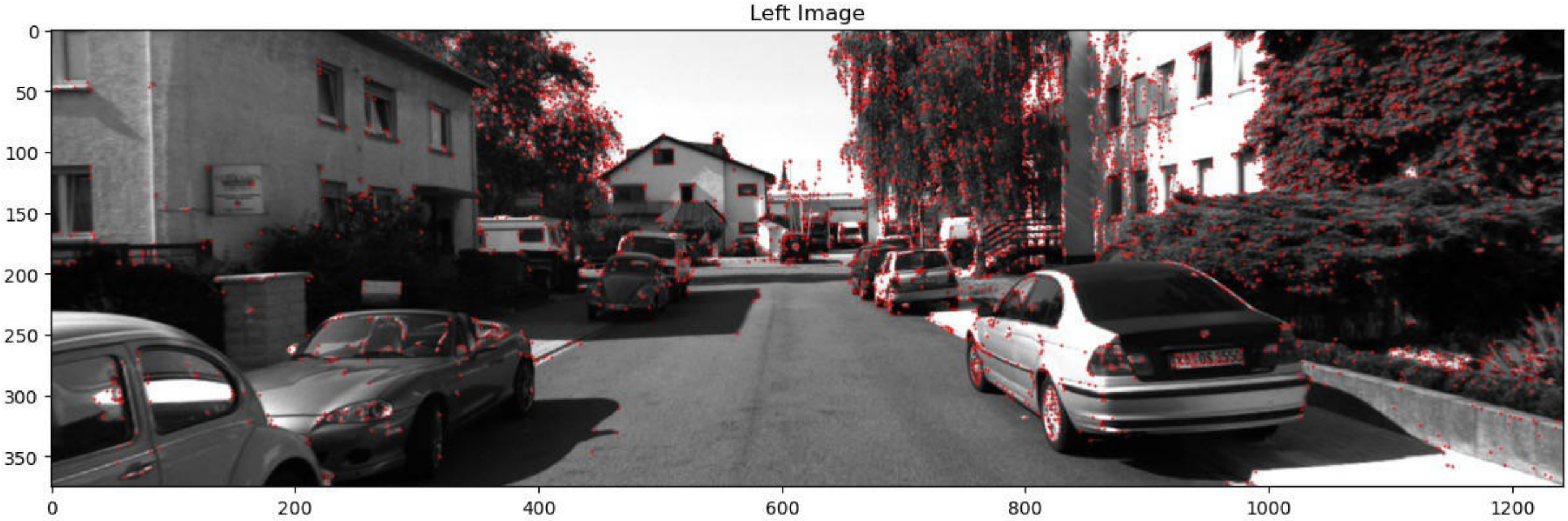
Why FAST is fast

- Uses **simple intensity comparisons**, not gradients or eigenvalues
- No floating-point computation, no convolution, not smoothing, no derivatives, no structure tensor …

- Summary:
  - **Decision tree learning** is used to optimise the **pixel test order**
  - Trained from data to **minimise the number of comparisons**
  - Achieves state-of-the-art speed while preserving accuracy

CRICOS No.00213J

Keypoints detected with FAST



CRICOS No.00213J

ORB

Combining FAST and BRIEF with Rotation and Scale Invariance



ORB (2011)

Abstract

Feature matching is at the base of many computer vision problems, such as object recognition or structure from motion. Current methods rely on costly descriptors for detection and matching. In this paper, we propose a very fast binary descriptor based on BRIEF, *called* ORB, *which is rotation invariant and resistant to noise. We demonstrate through experiments how* ORB *is at two orders of magnitude faster than* SIFT, *while performing as well in many situations. The efficiency is tested on several real-world applications, including object detection and patch-tracking on a smart phone.*

CRICOS No.00213J

ORB: an efficient alternative to SIFT or SURF

Ethan Rublee Vincent Rabaud Kurt Konolige Gary Bradski
Willow Garage, Menlo Park, California
{erublee}{vrabaud}{konolige}{bradski}@willowgarage.com

Abstract

Feature matching is at the base of many computer vision problems, such as object recognition or structure from motion. Current methods rely on costly descriptors for detection and matching. In this paper, we propose a very fast binary descriptor based on BRIEF, *called* ORB, *which is rotation invariant and resistant to noise. We demonstrate through experiments how* ORB *is at two orders of magnitude faster than* SIFT, *while performing as well in many situations. The efficiency is tested on several real-world applications, including object detection and patch-tracking on a smart phone.*

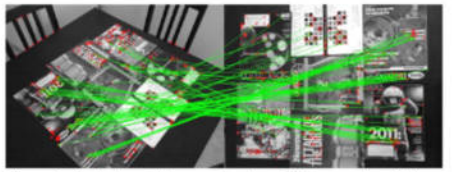


Figure 1. Typical matching result using ORB on real-world images with viewpoint change. Green lines are valid matches; red circles indicate unmatched points.

1. Introduction

The SIFT keypoint detector and descriptor [17], although over a decade old, have proven remarkably successful in a number of applications using visual features, including object recognition [17], image stitching [28], visual mapping [25], etc. However, it imposes a large computational burden, especially for real-time systems such as visual odometry, or for low-power devices such as cellphones. This has led to an intensive search for replacements with lower computation cost; arguably the best of these is SURF [2]. There has also been research aimed at speeding up the computation of SIFT, most notably with GPU devices [26].

In this paper, we propose a computationally-efficient replacement to SIFT that has similar matching performance, is less affected by image noise, and is capable of being used for real-time performance. Our main motivation is to enhance many common image-matching applications, e.g., to enable low-power devices without GPU acceleration to perform panorama stitching and patch tracking, and to reduce the time for feature-based object detection on standard PCs. Our descriptor performs as well as SIFT on these tasks (and better than SURF), while being almost two orders of magnitude faster.

Our proposed feature builds on the well-known FAST keypoint detector [23] and the recently-developed BRIEF descriptor [6]; for this reason we call it ORB (Oriented FAST and Rotated BRIEF). Both these techniques are attractive because of their good performance and low cost. In this paper, we address several limitations of these techniques vis-a-vis SIFT, most notably the lack of rotational invariance in BRIEF. Our main contributions are:

- The addition of a fast and accurate orientation component to FAST.
- The efficient computation of oriented BRIEF features.
- Analysis of variance and correlation of oriented BRIEF features.
- A learning method for de-correlating BRIEF features under rotational invariance, leading to better performance in nearest-neighbor applications.

To validate ORB, we perform experiments that test the properties of ORB relative to SIFT and SURF, for both raw matching ability, and performance in image-matching applications. We also illustrate the efficiency of ORB by implementing a patch-tracking application on a smart phone. An additional benefit of ORB is that it is free from the licensing restrictions of SIFT and SURF.

2. Related Work

Keypoints FAST and its variants [23, 24] are the method of choice for finding keypoints in real-time systems that match visual features, for example, Parallel Tracking and Mapping [13]. It is efficient and finds reasonable corner keypoints, although it must be augmented with pyramid

2011 IEEE International Conference on Computer Vision 2564
978-1-4577-1102-2/11/$26.00 ©2011 IEEE
Authorized licensed use limited to: Queensland University of Technology. Downloaded on July 20,2023 at 07:13:56 UTC from IEEE Xplore. Restrictions apply.

ORB Oriented FAST and Rotated BRIEF

ORB = FAST Detector + BRIEF Descriptor

- But both made orientation invariant

CRICOS No.00213J

ORB Oriented FAST and Rotated BRIEF

ORB = FAST Detector + BRIEF Descriptor

- But both made orientation invariant

- oFAST (Oriented FAST)
  - Borrows ideas from SIFT: scale pyramid
  - Borrows ideas from Harris: corner measure to avoid keypoints on lines
  - Like SIFT, it finds the dominant orientation, but much simpler:
    - Find centroid of pixel values around the keypoint coordinate, then the vector from the keypoint to the centroid is the orientation

CRICOS No.00213J

ORB Oriented FAST and Rotated BRIEF

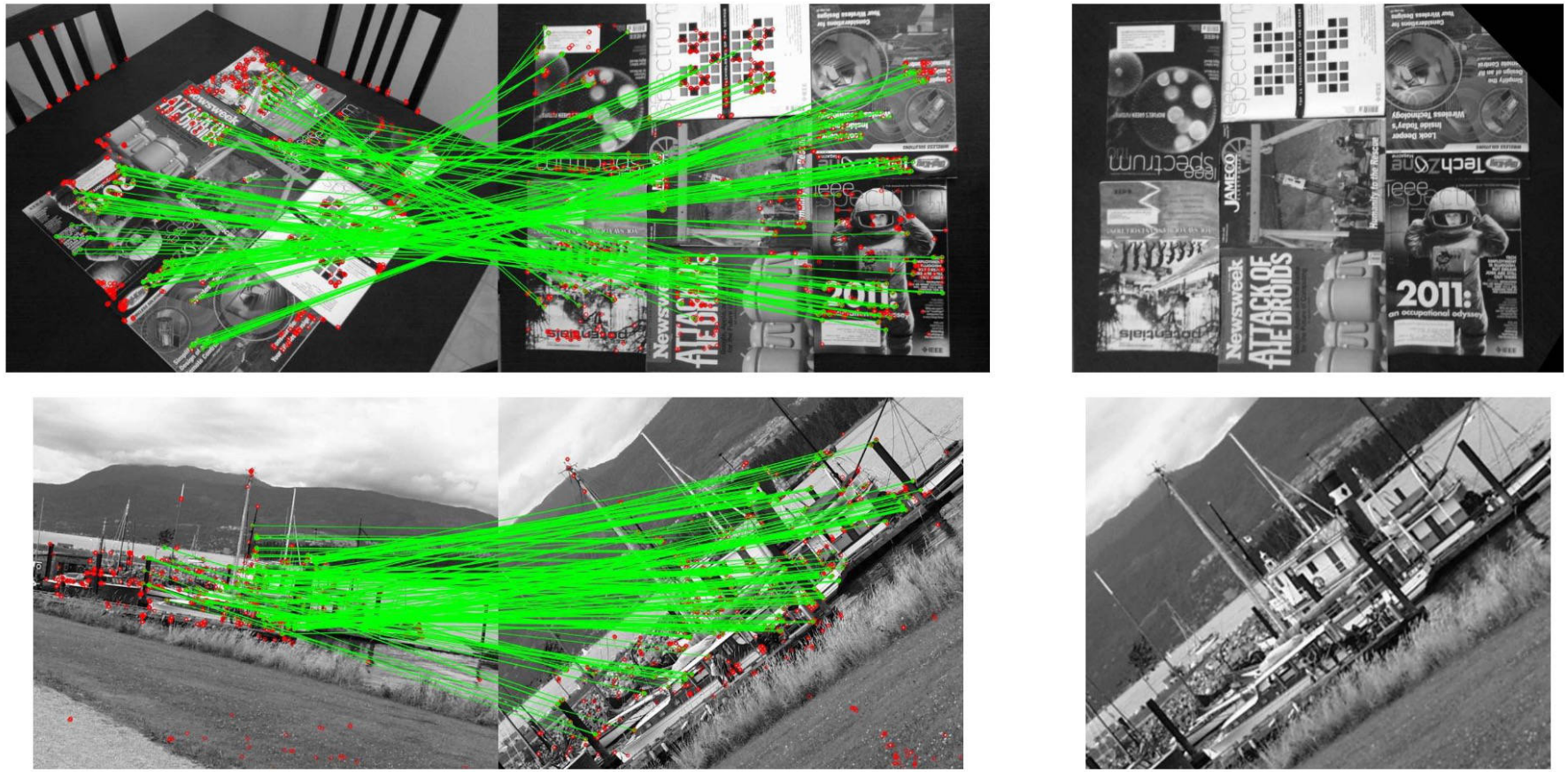
ORB = FAST Detector + BRIEF Descriptor

- But both made orientation invariant

- rBRIEF (Rotation-Aware BRIEF)
  - Use the orientation from oFAST to simply rotate the coordinates of compared patch locations

CRICOS No.00213J

ORB Oriented FAST and Rotated BRIEF



CRICOS No.00213J

Rublee, E., Rabaud, V., Konolige, K., & Bradski, G. (2011).
ORB: An efficient alternative to SIFT or SURF. In *2011 International Conference on Computer Vision*

Feature Matching

Achieving robust matches



Feature Matching

- Simplest scheme: brute force matching
  - For each descriptor in Image A, find the most similar in Image B

- Similarity metric
  - Euclidean distance
  - Hamming distance (for binary features like BRIEF)

CRICOS No.00213J

Feature Matching – Lowe's Ratio Test

- Sometimes the best and second match are almost equally similar
- What to do?
  - Simply discard such matches as unreliable

Ratio Test: $\dfrac{\text{best difference}}{2^{\text{nd}}\ \text{best difference}} < \theta\ ?$

$$\frac{A - B_1}{A - B_2} < \theta$$

CRICOS No.00213J

Feature Matching – Cross Check

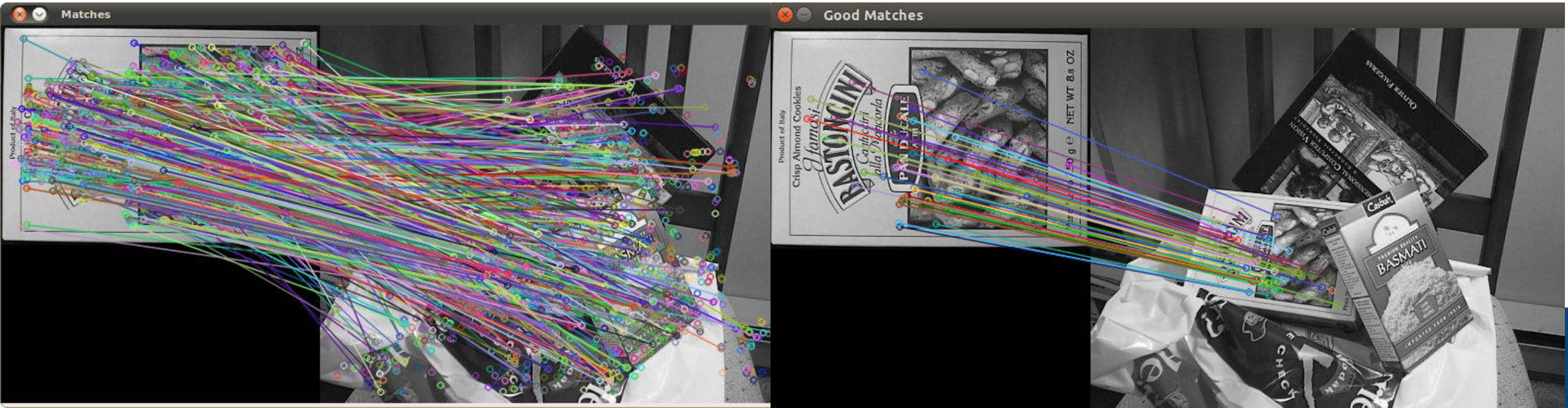
- Sometimes B_1 is the most similar feature for A_1, but not vice versa
  - i.e. there is a A_2 that is even more similar to B_1 than $A_1$is

- Is there an A_2 so that $A_2 - B_1 < A_1 - B_1$?
- In that situation, discard the match (A_1, B_1)

CRICOS No.00213J

Feature Matching – Avoid pure Brute Force

- In general, always include some level of refinement step
- Avoid using the pure brute force matching scheme
- We will learn another way of identifyign false matches in the coming weeks (using multiple view geometry)



Learned Feature Matching Schemes

Traditional matching treats each feature largely in isolation:

"Find the descriptor in the second image that looks most similar."

- This can fail when:
  - several regions look alike
  - individually plausible matches are inconsistent with the rest of the scene

The real task is therefore not just **descriptor comparison**. It is a **joint assignment problem**

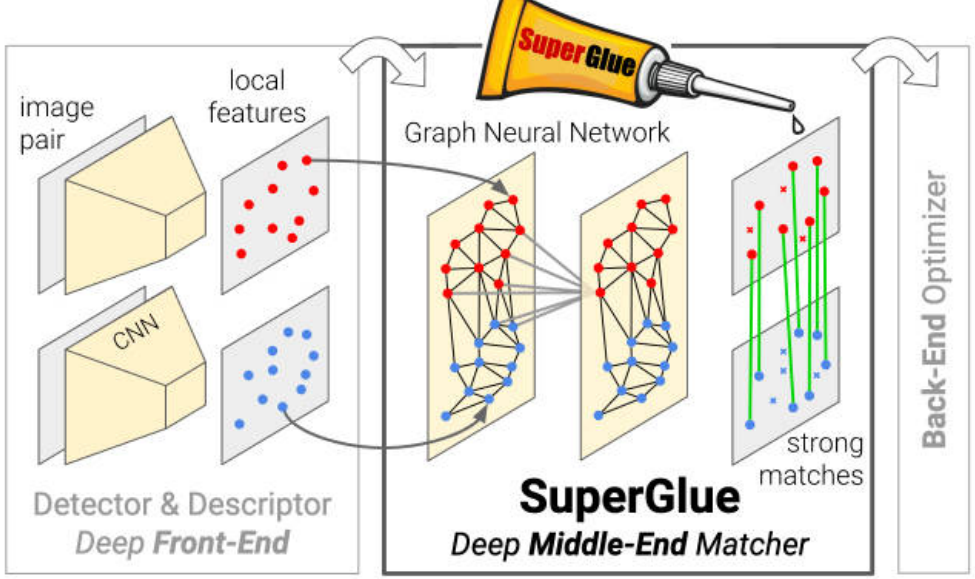
CRICOS No.00213J

Learned Feature Matching Schemes

Traditional matching	Learned matching
Compares features mostly independently	Reasons about all features jointly
Relies on rules such as ratio test and cross-check	Learns which matches are mutually consistent
Rejects ambiguity using fixed thresholds	Predicts both correspondences and unmatched points
Similar computation for every image pair	Can adapt computation to difficulty (e.g. LightGlue)

CRICOS No.00213J

SuperGlue



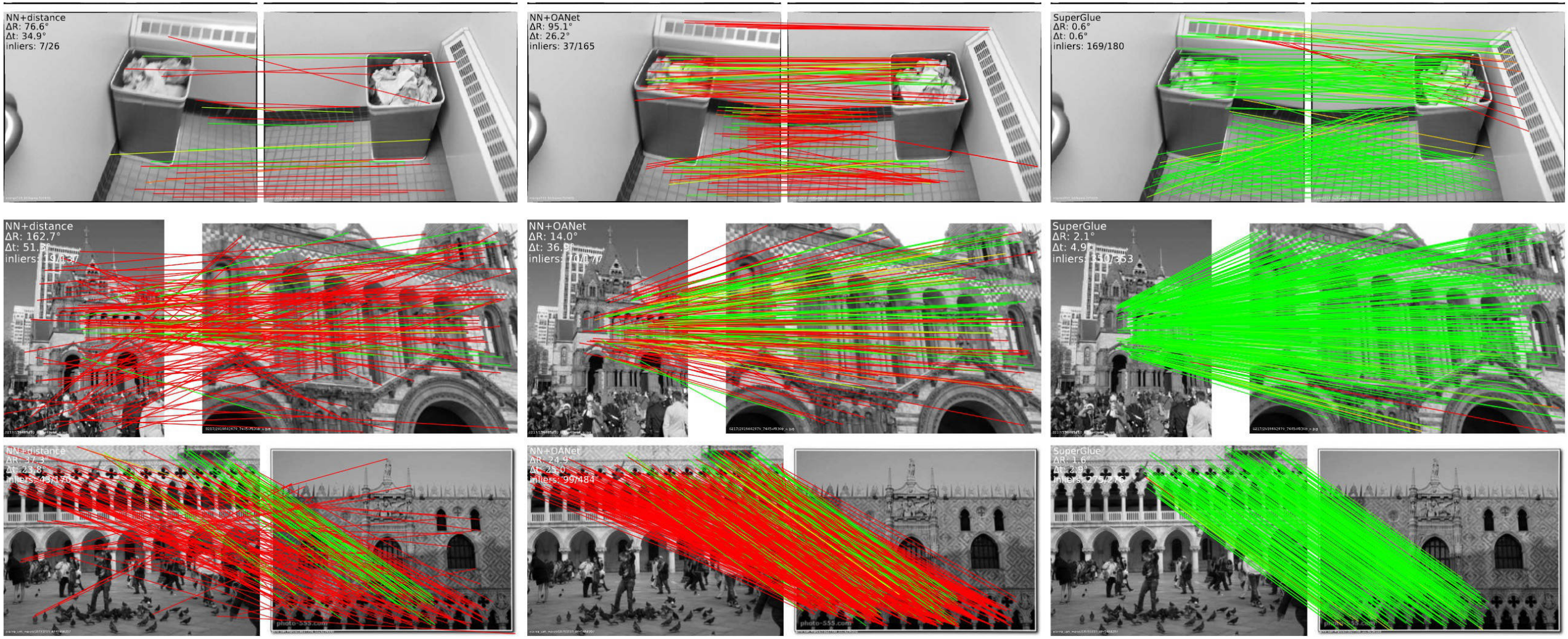
Core idea: reason about all features together.

- Rather than matching each descriptor independently, SuperGlue allows features to exchange information:
  - within each image, to understand surrounding context
  - across the two images, to identify likely correspondences
- It then jointly produces:
  - accepted feature correspondences
  - rejected features with no reliable match

Key intuition:

- A match becomes more convincing when it agrees with the broader pattern of matches across the two images.
- SuperGlue therefore replaces several hand-designed matching rules with a learned matching stage.

SuperGlue


NN+distance
ΔR: 76.6°
Δt: 34.9°
inliers: 7/26
NN+OANet
ΔR: 95.1°
Δt: 26.2°
inliers: 37/165
SuperGlue
ΔR: 0.6°
Δt: 0.6°
inliers: 169/180
NN+distance
ΔR: 162.7°
NN+OANet
ΔR: 14.0°
SuperGlue
ΔR: 2.1°
NN+distance
NN+OANet
ΔR: 24.9°
SuperGlue
ΔR: 1.6°

LightGlue – A more efficient SuperGlue

You can try it in the notebook for this week!

Abstract

We introduce LightGlue, a deep neural network that learns to match local features across images. We revisit multiple design decisions of SuperGlue, the state of the art in sparse matching, and derive simple but effective improvements. Cumulatively, they make LightGlue more efficient – in terms of both memory and computation, more accurate, and much easier to train. One key property is that LightGlue is adaptive to the difficulty of the problem: the inference is much faster on image pairs that are intuitively easy to match, for example because of a larger visual overlap or limited appearance change. This opens up exciting prospects for deploying deep matchers in latency-sensitive applications like 3D reconstruction. The code and trained models are publicly available at `github.com/cvg/LightGlue`.

1. Introduction

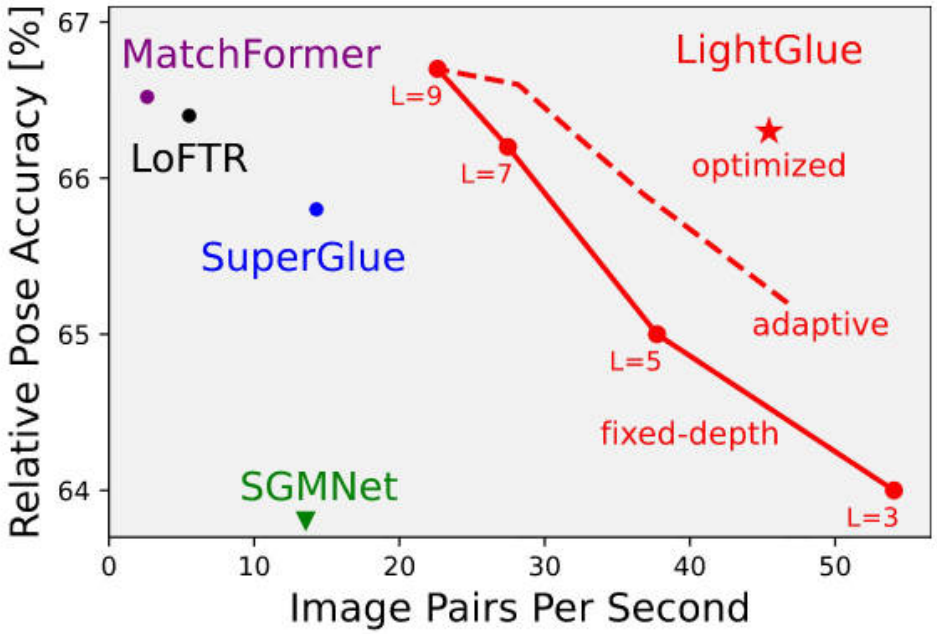
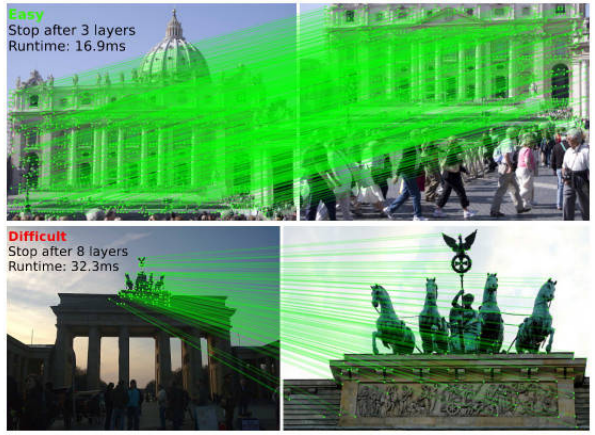


Figure 1. **LightGlue matches sparse features faster and better** than existing approaches like SuperGlue. Its adaptive stopping mechanism gives a fine-grained control over the speed vs. accuracy trade-off. Our final, optimized model ★ delivers an accuracy closer to the dense matcher LoFTR at an 8× higher speed, here in typical outdoor conditions.



Summary



Noteworthy Detectors and Descriptors

Name	Type	Invariant to	Speed	Notes
Harris	Detector	Rotation (partly)	Medium	Uses eigenvalues of gradient tensor
FAST	Detector	None (original)	**Very fast**	Pixel intensity pattern
SIFT	Detector + Descriptor	Scale, rotation	Slow	Very robust, patented (expired)
SURF	Detector + Descriptor	Scale, rotation	Faster than SIFT	Uses integral images
ORB	Detector + Descriptor	Rotation, scale (limited)	**Real-time**	Combines FAST + BRIEF
BRIEF	Descriptor	Not rotation invariant	Very fast	Binary descriptor

Feature Detection, Description, and Matching in the Age of Deep Learning

- Not surprisingly, a number of deep learning-based approaches have been developed in the recent past, and we list a few noteworthy ones here. Check out their papers, project websites and code!

  - Lift: Learned invariant feature transform An early approach that learned the complete local-feature pipeline: keypoint detection, orientation estimation, and feature description
  - DISK: Learning local features with policy gradient Learns keypoints and descriptors together, using reinforcement-learning ideas to directly reward features that produce many correct matches..
  - SuperPoint Jointly predicts interest points and local descriptors in a single network. It uses self-supervised training to learn repeatable features without requiring manually labelled keypoints.

  - SuperGlue is a learned feature-matching method that reasons about all potential correspondences jointly. It predicts reliable matches while rejecting points that do not have a valid correspondence.
  - LightGlue is a faster and more efficient successor to SuperGlue. It adapts its computation to the difficulty of the image pair, spending less effort on easy matching problems and more on difficult ones. You can use it in the ENN583 Jupyter server environment.

Most Recent Developments

Trend:

- The field is moving away from separately hand-designing sparse detectors, descriptors, and matching rules
- toward **jointly learned correspondence systems** that produce richer, often semi-dense or dense features and reason globally across both images.

CRICOS No.00213J

Most Recent Developments

Sparse learned pipelines

- Learn to detect keypoints, compute learned descriptors, and use learned matchers.
- can still improve substantially through larger datasets, models, and training. LoMa: Local Feature Matching Revisited (ECCV 2026)

Detector-free or dense pipelines

- avoid committing to a small set of keypoints
- compute feature maps over much more of the image and infer correspondences directly, which can help in low-texture areas and under difficult viewpoint changes.
- Recent dense methods explicitly position dense matching as a major modern direction



Practical Considerations – Which method should I use?

- Consider available hardware
  - (CPU, GPU, embedded processor)
  - Memory and power constraints
  - Required processing rate or latency
  - Expected number of features or image resolution
- Consider image conditions
  - Expected scale, rotation, viewpoint change, blur etc.

- Classical or Learned?
  - Classical methods such as SIFT, ORB, remain useful when they meet the application requirements more simply and reliably than a learned alternative.

CRICOS No.00213J

Practical Considerations

- **Practical starting point:**
  - try ORB first as a fast, lightweight baseline
  - consider SIFT if challenging conditions justify the additional computational cost.

- **Why ORB is a good starting point**
  - fast on CPUs, low memory footprint available directly in OpenCV, binary descriptors allow efficient Hamming-distance matching
  - suitable for real-time and embedded systems
- **When to try SIFT**
  - substantial scale changes, larger viewpoint changes, weaker or less distinctive texture
  - demanding image registration tasks
  - a need for fewer but more reliable matches
  - trade-off: more computationally expensive

CRICOS No.00213J

Practical Considerations

Stage	Good starting point	When to try something else
Detection	ORB	Try SIFT if you need better robustness to scale or viewpoint change
Description	ORB descriptor	Try SIFT if ORB descriptors are not distinctive enough
Matching	Brute-force nearest neighbour with the correct distance metric	Try approximate nearest neighbour search when the descriptor set becomes large
Filtering	Ratio test or mutual cross-check	Add geometric verification with RANSAC when false matches remain

A sensible baseline is ORB detection and description, Hamming-distance matching, followed by a ratio test or cross-check.
In following weeks we learn to use RANSAC to remove geometrically inconsistent matches.

CRICOS No.00213J

Where to go from here?

- Need a review of the lecture? Use the Echo360 recording
- Read the material on Canvas BEFORE the Tutorial on Friday
- Practical: Put it to use, implement a number of detectors, descriptors, matching algorithms

CRICOS No.00213J

Questions?

Remember: Read the paper before the Tutorial on Friday!